Non-Adaptive Programmable PRFs and Applications to Stacked Garbling

Vipul Goyal[*] David Heath[†] Abhishek Jain[‡] Yibin Yang[§]

April 25, 2026

Abstract

Garbled circuits are a fundamental primitive in cryptography. While the size of garbled circuits in Yao's original scheme grows linearly with the circuit size, a recent line of work on stacked garbling (SGC) [Heath-Kolesnikov, CRYPTO'20] has achieved near-sublinear size for *branching computations*, based only on one-way functions. Specifically, these schemes achieve garbled size growing only with the size of a single branch and the *total input length* to all the branches. Due to the latter dependence, these results are best suited to "small" input settings.

We present a stacked garbling scheme for "large" input settings based on one-way functions. The garbled size in our scheme grows only with the size of a single branch and its input length (up to logarithmic factors), plus an additive term in the number of branches (as in prior SGC).

To obtain our result, we uncover a connection between stacked garbling and the notion of (adaptive) programmable pseudorandom functions (apPRFs) [Boneh-Lewi-Wu, PKC'17]. While existing apPRF constructions either rely on stronger assumptions (e.g., learning with errors or indistinguishability obfuscation) or incur noticeable security losses under weaker assumptions, we identify a relaxed notion of *non-adaptive* programmable PRFs (napPRFs) that suffices for our result, and establish its feasibility based on one-way functions. Interestingly, we build on techniques from the SGC literature to construct napPRFs with our desired efficiency, and then apply napPRFs back to SGC to obtain our main result.

Along the way, as an additional result of independent interest, we provide the first construction of (adaptive) programmable PRFs for polynomial size domains based on one-way functions.

Keywords: Garbled Circuit · Stacked Garbling · Programmable PRF

[*]NTT Research, vipul@vipulgoyal.org.
[†]University of Illinois Urbana-Champaign, daheath@illinois.edu.
[‡]NTT Research and Johns Hopkins University, abhishek@cs.jhu.edu.
[§]NTT Research, yibiny@ece.utoronto.ca.

Contents

1 Introduction **1**
1.1 Our Results . . . 1
1.2 Related Work . . . 4

2 Technical Overview **4**
2.1 Non-adaptive Programmable PRFs . . . 4
2.2 Stacked Garbling: A Concise Overview . . . 6
2.3 Demultiplexers from napPRFs for Stacked Garbling . . . 6
2.4 Black-box Equivalence between napPRFs and Two-party DPFs . . . 8
2.5 Constructing Short-key Offline-online napPRFs . . . 9

3 Preliminaries **12**
3.1 Notation . . . 12
3.2 Distributed Point Functions . . . 13
3.3 Pseudorandom Functions . . . 13
3.4 Programmable PRFs . . . 14

4 Definitions of Non-adaptive Programmable PRFs **15**
4.1 Non-adaptive vs. Adaptive Programmable PRFs . . . 16
4.2 Suffix Invariance . . . 17

5 Relationship Between napPRFs and Two-Party DPFs **17**
5.1 Individual Pseudorandomness for DPFs . . . 18
5.2 napPRF $\Rightarrow$ DPF . . . 18
5.3 DPF $\Rightarrow$ napPRF . . . 19
5.4 Programmable DPF $\Rightarrow$ apPRF . . . 20

6 Toward Short-key Offline-online napPRFs **20**
6.1 Garbling-based Short-key Compiler . . . 21
6.2 Suffix-invariant napPRFs via Recursively Stacked Offline Keys . . . 21

7 Improving SGC via Short-key Offline-online napPRFs **25**
7.1 Off-the-shelf Use of Our Offline-online napPRF . . . 25
7.2 Tailoring Our Offline-online napPRF for Better Integration . . . 27

A Pseudorandomness of Master Evaluation Procedures **33**
A.1 Superpolynomial-size Domains . . . 33
A.2 Polynomial-size Domains . . . 34

B Deferred Proofs **34**
B.1 Proof of Lemma 2 . . . 34
B.2 Proof of Theorem 6 . . . 37
B.3 Proof of Theorem 7 . . . 38
B.4 Proof of Theorem 8 . . . 39
B.5 Proof of Theorem 9 . . . 39

C Formalism of Garbling-based Short-key Compiler **44**
C.1 Garbling Schemes 44
C.2 Formalism 45

D Tailored Offline-online napPRF for LogStack **46**

E The Improved LogStack Garbling Scheme **49**

1 Introduction

Garbled circuits (GCs) [Yao86] are a fundamental primitive in cryptography. In Yao's classical construction based on one-way functions (OWFs), given a Boolean circuit $\mathcal{C} : \{0,1\}^I \to \{0,1\}$, a *garbler* can generate a garbled circuit $\widehat{\mathcal{C}}$ together with short input encodings that associate *garbled labels* with each input wire. Given $\widehat{\mathcal{C}}$ and the labels corresponding to an input $x \in \{0,1\}^I$, an *evaluator* can compute an output label that decodes to $\mathcal{C}(x)$, while learning nothing beyond $\mathcal{C}(x)$. Standard instantiations of GCs have size $\mathcal{O}(|\mathcal{C}| \cdot \lambda)$ and evaluation uses I λ-bit input labels, where λ is the security parameter.

While GCs provide a generic mechanism for securely evaluating *circuits*, many real-world computations are more naturally expressed as programs with data-dependent control flow. Among these, a particularly notable class is *branching computation*, where an intermediate step computes an n-bit selector $\alpha \in \{0,1\}^n$ and then executes one out of $B = 2^n$ candidate subcircuits (or branches). Concretely, let $\mathcal{C}_0, \ldots, \mathcal{C}_{B-1}$ be subcircuits, each of size C and with a common interface $\mathcal{C}_j : \{0,1\}^I \to \{0,1\}$, and consider computing $y \leftarrow \mathcal{C}_\alpha(x)$. A naïve approach evaluates all B candidate circuits, then uses α to *multiplex* (i.e., select) the output of circuit $\mathcal{C}_\alpha$. When garbled, this computation has size $\mathcal{O}(BC\lambda)$ bits.

Stacked garbling (SGC) [HK20a, HK21] provides smaller garbled encodings for this problem. At a high level, SGC observes that the garbler can *stack* — by XOR-combining corresponding components — the B garblings of $\mathcal{C}_0, \ldots, \mathcal{C}_{B-1}$ into a single garbled artifact. The evaluator can then *unstack* locally once it learns (obliviously) the randomness used for all but the active (i.e., the α-th) branch. Building on this observation, [HK20a] achieves garbled size $\mathcal{O}(BI\lambda + C\lambda)$ at the cost of $\mathcal{O}(B^2)$ computation. The improved scheme LogStack [HK21] retains the same size/communication profile, while reducing party computation to $\mathcal{O}(B \log B)$.

SGC eliminates the *branch-duplication* blowup — stacking reduces the garbled size for B branches from $\mathcal{O}(BC\lambda)$ to $\mathcal{O}(C\lambda)$ — but incurs an additive $\mathcal{O}(BI\lambda)$ term, intuitively the cost of *privately routing* an I-bit input to the selected branch. Consequently, SGC is most beneficial when each branch performs substantial computation relative to its input length. In contrast, when I is comparable to (or larger than[1]) C, the improvement over the naïve approach becomes marginal and may even disappear.

This regime is indeed common in practice: many systems implement a large *dispatch* in which an n-bit "mode" selector α chooses one of 2^n simple behaviors, and the selected branch then performs only light processing on a large input. Notable examples include network packet filtering, where α selects one of many filtering rules and the system outputs accept/drop based on a few header fields while the payload may be large. Similarly, in ML inference pipelines, a model index α selects one of many lightweight prediction modules (e.g., a small linear classifier) applied to a large embedding.

This leaves open a natural question:

Can we retain SGC's $\mathcal{O}(C\lambda)$ benefit while decoupling the dependence on B from the input length I?

1.1 Our Results

We answer this affirmatively. Within the standard OWF-based (Yao) garbling framework, we give a new SGC scheme with garbled size $\mathcal{O}(I \log B \cdot \lambda + B\lambda + C\lambda)$ and party computation $\mathcal{O}(B \log B)$. This asymptotically improves the prior state of the art LogStack [HK21], which has size $\mathcal{O}(BI\lambda + C\lambda)$ with the same computation.

[1] This is possible because different branches may receive different subsets of the inputs.

Scheme	Offline key size (bits)	Online key size (bits)	Black-box in OWFs	Suffix invariance	Super-poly domains
v1, Section 5.3	0	$nm\lambda + 2nm + m\lambda$	✓	✗	✓
v2, Section 6.1	$\mathcal{O}(nm \cdot \mathsf{poly}(\lambda))$	$(n+m)\lambda$	✗	✗	✓
v3, Section 6.2	$8nm\lambda + 4n^2\lambda - 2n\lambda + m$	$(n+m)\lambda$	✓	✓	✗

Table 1: Overview of the napPRF constructions from OWFs in this work. All schemes are over domain $\{0,1\}^n$ and range $\{0,1\}^m$. v1 is a napPRF without offline-online separation. For the SGC application, we require suffix invariance but do not require support for superpolynomial domains; thus, *only* v3 applies.

Theorem 1 (Improved SGC, informal). *Let $\mathcal{C}_0, \ldots, \mathcal{C}_{B-1}$ be Boolean circuits of size C with a common interface $\{0,1\}^I \to \{0,1\}$. There exists an SGC scheme for computing $y \leftarrow \mathcal{C}_\alpha(x)$ from (black-box) OWFs whose garbled size is $\mathcal{O}(I \log B \cdot \lambda + B\lambda + C\lambda)$, and computation of each party is $\mathcal{O}(B \log B)$.*

Our improvement stems from a new perspective on the $\mathcal{O}(BI\lambda)$ bottleneck in SGC. We show that the relevant *demultiplexing* gadget (see next) can be cast as a one-point "programming" task closely related to *programmable PRFs* [BLW17, BKM17, CC17, PS18], and that the SGC setting only needs a weaker non-adaptive variant. This motivates a new primitive — *non-adaptive programmable PRFs (napPRFs)* — which we formalize, construct from OWFs, and use to optimize SGC. We sketch this connection next, and defer a more detailed overview to Section 2.

Demultiplexing as one-point programming. The $\mathcal{O}(BI\lambda)$ overhead in prior SGC schemes comes from the *demultiplexer*: a garbled gadget that, on a hidden selector $\alpha \in \{0,1\}^n$, must provide the evaluator with the I input-wire labels for branch α, while producing *dummy* labels for every $t \neq \alpha$. These dummy labels must (i) be independent of the real input x (so G can precompute the inactive-branch behavior) and (ii) be indistinguishable from real GC labels (so the active branch remains hidden). Existing constructions satisfy these constraints by essentially hard-coding B branch outcomes, incurring $\mathcal{O}(BI\lambda)$ size.

Non-adaptive programmable PRFs (napPRFs). We observe that demultiplexing is naturally a *one-point programming* task, reminiscent of *(adaptive) programmable PRFs* (apPRFs) [BLW17, BKM17, CC17, PS18]. An apPRF consists of a PRF family $F_{\mathsf{msk}}(\cdot)$ together with a *programming* procedure that, given $(\mathsf{msk}, \alpha, \beta)$, outputs a *programmed key* $\mathsf{sk}_{\alpha \mapsto \beta}$ defining a related function $F_{\mathsf{sk}_{\alpha \mapsto \beta}}(\cdot)$. This function satisfies $F_{\mathsf{sk}_{\alpha \mapsto \beta}}(\alpha) = \beta$, while for every $t \neq \alpha$ we have $F_{\mathsf{sk}_{\alpha \mapsto \beta}}(t) \equiv F_{\mathsf{msk}}(t)$, i.e., it behaves like the underlying PRF off the programmed point. Moreover, when β is uniform, the programmed key should hide the programmed index α (*index-hiding*). In the SGC setting, we take the payload to be the active-branch input-label vector, and the off-point requirement corresponds to outputs that are independent of β and indistinguishable from real GC labels.

Unfortunately, known index-hiding apPRFs rely on comparatively strong assumptions (e.g., LWE [CC17, BTVW17, PS18] or $i\mathcal{O}$ [BLW17]), and their programming procedure is not obviously GC-friendly. For SGC, however, full adaptive programmability is unnecessary: it suffices that off-point behavior is independent of β (though it may depend on α), and that α is hidden when β is uniform. This motivates napPRFs, which admit efficient OWF-based realizations and GC-friendly offline-online forms, and which we use to construct SGC demultiplexers (Theorem 1).

Our technical contribution. Motivated by the SGC application, we systematically study the napPRF primitive. Our main technical contribution includes:

- **A new primitive: napPRF.** We formalize non-adaptive programmable PRFs (napPRFs), capturing single-point programming with β-independent off-point behavior and index hiding under uniform programming.

  We also define an offline-online variant that decomposes the programmed key into an (α, β)-independent offline key (which can be used as garbled material) and a short online key produced by the (garbled) programming procedure.

  We introduce *suffix invariance*, a correctness property needed for SGC.

- **Black-box equivalence with DPFs.** We establish a black-box equivalence between napPRFs and two-party distributed point functions (DPFs), yielding OWF-based napPRFs for superpolynomial domains via known DPF constructions [GI14, BGI15]. However, these generic instantiations *do not suffice for our SGC application*: they lack suffix invariance or GC-friendly programming.

  Theorem 2 (napPRF $\Leftrightarrow$ DPF, informal). *A correct, index-hiding napPRF black-box implies a correct, secure two-party DPF, and vice versa.*

  Corollary 1 (OWF-based DPF $\Rightarrow$ napPRF, informal). *Assuming one-way functions (via black-box use), there exists a correct and index-hiding napPRF over $\{0,1\}^n \to \{0,1\}^m$ for $n \leq \mathsf{poly}(\lambda)$. Moreover, its programmed key has length $nm\lambda + 2nm + m\lambda$ bits, and it is* not *suffix-invariant.*

- **Short online keys with suffix invariance from black-box OWFs.** For polynomial-size domains, we construct black-box OWF-based offline-online napPRFs with short online keys and built-in suffix invariance.

  Interestingly, our constructions are inspired by stacked garbling: the offline key is stackable, and the online key consists solely of standard Yao garbled labels encoding (α, β), making the programming procedure inherently GC-friendly.

  Theorem 3 (Short-key offline-online napPRFs, informal). *Assuming one-way functions (via black-box use), there exists a correct and index-hiding offline-online napPRF over $\{0,1\}^n \to \{0,1\}^m$ for $2^n \leq \mathsf{poly}(\lambda)$. Moreover, its offline programmed key has length $8nm\lambda + 4n^2\lambda - 2n\lambda + m$ bits, whereas its online programmed key has length $(n+m)\lambda$ bits, and it is suffix-invariant.*

- **Improved stacked garbling.** We instantiate LogStack's demultiplexer using our offline-online napPRFs, obtaining garbled size $\mathcal{O}(I \log B \cdot \lambda + B\lambda + C\lambda)$ while preserving LogStack's $\mathcal{O}(B \log B)$ computation (cf. Theorem 1).

- **apPRFs from black-box OWFs.** As an additional result of independent interest, we show that *programmable* DPFs [BGIK22b] imply *adaptive* programmable PRFs; this yields the first OWF-based apPRF construction (for poly-size domains). In contrast, prior apPRF constructions either target superpolynomial domains under LWE [CC17, BTVW17, PS18] or $i\mathcal{O}$ [BLW17], or are OWF-based for polynomial domains but incur a $1/\mathsf{poly}(\lambda)$ security loss [GZSP25].

  Theorem 4 (Programmable DPF $\Rightarrow$ apPRF, informal). *Assuming one-way functions (via black-box use), there exists a correct and index-hiding apPRF over $\{0,1\}^n \to \{0,1\}^m$ for $2^n \leq \mathsf{poly}(\lambda)$.*

Table 1 summarizes the OWF-based napPRF constructions present in this work.

1.2 Related Work

Stacked garbling. The technique of *stacking* for oblivious branching was introduced in [HK20b] in the GC-ZK [JKO13] setting (i.e., privacy-free garbling), and later extended to standard Yao garbling in [HK20a, HK21]. It has since been applied and extended in several settings: [HKP20, GHAHJ22] extend it to MPC branching; [HHK+22, NK25] use it to build garbled PIR; [HKN24] uses it for garbled lookup tables; and [YPHK23] applies it to build a constant-round 2PC system for RAM programs. We advance stacked garbling by reducing the dominant garbled-size overhead that scales with the product of the number of branches and the per-branch input length.

Adaptive programmable PRFs. Programmable PRFs augment a standard PRF with a *one-point programming* capability: from a key, one can derive a programmed key whose evaluation at a designated index α is forced to a chosen value β, while evaluations on all other inputs remain unchanged [BW13, BGI14]. A central refinement is *index hiding* (also called *programming privacy*, or privately programmable PRFs, PPPRFs), which requires that the programmed key hide the programmed point α as long as the programmed value β is uniform [BLW17, BKM17, CC17, PS18]. Without index hiding, programmable PRFs follow easily from one-way functions via the classical puncturable PRF, essentially by embedding (α, β) into the punctured key [BW13, BGI14, GGM86], yielding simple and concretely efficient schemes. In contrast, with programming privacy, known constructions rely on substantially stronger assumptions — notably lattice-based assumptions such as LWE [CC17, BTVW17, PS18] or indistinguishability obfuscation [BLW17] — and are considerably more involved, typically layering homomorphic-encryption machinery. In this work, we use "(adaptive) programmable PRF" to mean an index-hiding construction.

A recent attempt to obtain an OWF-based index-hiding construction for polynomial-size domains appears in [GZSP25], but incurs a $1/\mathsf{poly}(\lambda)$ security loss. While our focus is on *non-adaptive* programmable PRFs, we also obtain the first *adaptive* OWF-based construction for polynomial-size domains *without* this inverse-polynomial loss (cf. Section 5.4).

Distributed point functions. The first two-party DPF construction from one-way functions was given by Gilboa and Ishai [GI14], with subsequent works improving efficiency and concrete performance [BGI15, BGI16, BGH+25]. In this work, we use the classical OWF-based two-party DPF constructions [GI14, BGI15] as black boxes.

Beyond the two-party setting, recent work [GWW25] gives the first OWF-based multiparty DPF construction. Earlier works from public-key assumptions (e.g., DCR and LPN) achieve succinct communication for a fixed number of parties [CK24], later extended to arbitrarily many parties [ARS24, CKY25]. Separately, [BGIK22a] studies information-theoretic DPFs for three and four parties tolerating one corruption.

2 Technical Overview

We present our techniques in sufficient detail to understand our contribution. Let $\mathcal{X} := \{0,1\}^n$ denote a domain and let $\mathcal{Y} := \{0,1\}^m$ denote a range. We often refer to elements $\alpha \in \mathcal{X}$ as *indexes* or *points*, and to elements $\beta \in \mathcal{Y}$ as *values*.

2.1 Non-adaptive Programmable PRFs

A non-adaptive programmable PRF (napPRF) consists of three PPT procedures:

- KeyGen samples a master key msk;
- Prog takes as input a master key msk, an index $\alpha \in \mathcal{X}$, and a value $\beta \in \mathcal{Y}$. It outputs a programmed key sk on input $(\mathsf{msk}, \alpha, \beta)$, where intuitively sk's index α is programmed to output β.
- Eval takes as input a programmed key sk, a point $x \in \mathcal{X}$ and outputs a value.

It satisfies the following informal correctness and security guarantees:

- **On-point correctness:** $\mathsf{Eval}(\mathsf{sk}, \alpha) = \beta$, i.e., the evaluation at α outputs β.
- **Off-point value independence (correctness):** For every unprogrammed point $x \neq \alpha$, all β_1, β_2, $\mathsf{Eval}(\mathsf{Prog}(\mathsf{msk}, \alpha, \beta_1), x) = \mathsf{Eval}(\mathsf{Prog}(\mathsf{msk}, \alpha, \beta_2), x)$, i.e., at $x \neq \alpha$, the evaluation output is fixed by α (and msk), but not by β.
- **Index-hiding security:** For uniform β, the programmed key sk hides α.

We will later prove that these guarantees already imply that for any fixed α and uniform β, the induced function $x \mapsto \mathsf{Eval}(\mathsf{Prog}(\mathsf{msk}, \alpha, \beta), x)$ is a PRF.

Offline-online napPRF. We also consider an *offline-online* variant of napPRFs, where the programmed key sk splits as $(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}})$. Here the offline key $\mathsf{sk}_{\mathsf{off}}$ is sampled by KeyGen and is independent of (α, β), while the online key $\mathsf{sk}_{\mathsf{on}}$ is the (α, β)-dependent part produced by Prog. That is, $\mathsf{KeyGen}(1^\lambda)$ outputs $(\mathsf{msk}, \mathsf{sk}_{\mathsf{off}})$, $\mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$ outputs $\mathsf{sk}_{\mathsf{on}}$, and Eval takes $\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}}, x$ as inputs.

This variant's correctness and security notions carry over verbatim. This split is w.l.o.g.: any napPRF is offline-online by setting $\mathsf{sk}_{\mathsf{off}} = \perp$, and any offline-online napPRF is a standard napPRF by folding $\mathsf{sk}_{\mathsf{off}}$ into msk.

The main motivation for considering this variant is online efficiency: $\mathsf{sk}_{\mathsf{off}}$, which may be large, can be transmitted before (α, β) is known. Then in the online phase, once (α, β) is fixed, only the short key $\mathsf{sk}_{\mathsf{on}}$ needs to be transmitted.

Key metrics. We measure efficiency mainly by (i) the total programmed-key size $|\mathsf{sk}|$ and (ii) the online part $|\mathsf{sk}_{\mathsf{on}}|$. We require $|\mathsf{sk}| \leq \mathsf{poly}(n, m, \lambda)$, and call an offline-online napPRF *short-key* if $|\mathsf{sk}_{\mathsf{on}}|$ is linear in $n+m$ (up to $\mathsf{poly}(\lambda)$ factors).

Difference from adaptive programmable PRFs. An apPRF (see, e.g., [PS18, GZSP25]) has a *single* master function $F_{\mathsf{msk}}(x)$ (fixed by msk) that programming must preserve: for any programmed point (α, β), evaluation agrees with F on all $x \neq \alpha$. In contrast, a napPRF only guarantees β-*invariance* off the programmed point: for fixed α, changing β does not affect outputs on $x \neq \alpha$, but those off-point outputs may still depend arbitrarily on α. Thus, apPRF correctness enforces cross-α consistency, while a generic napPRF need not.

Suffix invariance. We also introduce a structural correctness property, *suffix invariance*, needed for stacked garbling applications. For any off-point query $x \neq \alpha$, let ℓ be the first position where bitstrings x and α differ. Then $\mathsf{Eval}(\mathsf{sk}, x)$ may depend on α only in terms of the first ℓ bits of α. In other words, evaluation $\mathsf{Eval}(\mathsf{sk}, x)$ is invariant under changes to the suffix $\alpha_{\ell+1..n}$. Hence, for any fixed x, as α ranges over $\{0,1\}^n \setminus \{x\}$, there are only n possible off-point outputs (one per ℓ), rather than 2^n.

For instance, when $n = 3$ and $x = 000 \neq \alpha$, the value at x has exactly three cases depending on whether $\alpha \in \{001\}$, $\{010, 011\}$, or $\{100, 101, 110, 111\}$.

See Section 4 for formal definitions.

2.2 Stacked Garbling: A Concise Overview

Stacked garbling (SGC) [HK20a, HK21] optimizes *oblivious branching*: a computation selects one of $B = 2^n$ subcircuits $\mathcal{C}_0, \ldots, \mathcal{C}_{B-1}$ (with $B \leq \mathsf{poly}(\lambda)$), where each $\mathcal{C}_j : \{0,1\}^I \to \{0,1\}$ has C gates. We consider subcircuits with one-bit output for simplicity. The evaluator E holds labels encoding a selector $\alpha \in \{0,1\}^n$ and input labels encoding $x \in \{0,1\}^I$ (possibly arising as intermediate values), and must get output label for $\mathcal{C}_\alpha(x)$. Naïvely garbling all branches costs $\mathcal{O}(BC\lambda)$.

Heath and Kolesnikov [HK20a] show that the garbler G can *stack* the B garblings into a single artifact by simply XORing them; later, E can *unstack* locally once it obliviously learns seed randomness for all but the active branch $\mathcal{C}_\alpha$. This yields garbled size $\mathcal{O}(BI\lambda + C\lambda)$, with the computation improved to $\mathcal{O}(B \log B)$ in [HK21].

The $\mathcal{O}(BI\lambda)$ term comes from a garbling gadget called the **demultiplexer**. Unstacking produces B candidate branch GCs, only one of which (the α-th) is well-formed, and E cannot tell which. To evaluate candidate branch t, E must also obtain the corresponding I input-wire labels for that branch. The demultiplexer provides these labels: on the active branch it outputs the real labels encoding x, and on inactive branches it outputs *dummy* labels.

These dummy labels must satisfy two constraints: (i) they must be independent of x (since G does not know x but must precompute/merge the inactive-branch behavior), and (ii) they must be indistinguishable from fresh GC labels, so E learns nothing about which branch is active. Prior schemes implement this by naïvely hard-coding B branch outcomes, each consisting of $\mathcal{O}(I)$ λ-bit labels.

2.3 Demultiplexers from napPRFs for Stacked Garbling

We outline how an (offline-online) napPRF yields the *demultiplexer* gadget at the heart of SGC. Conceptually, the demultiplexer enables E to derive a candidate input-wire label vector for *every* branch. On the *active* branch, this candidate must equal the *true* input-wire labels consistent with the garbled input; on every *inactive* branch, it should look like an unrelated dummy label vector, independent of the garbled input and indistinguishable from the active one.

- To start with, a direct approach — used by [HK20a, HK21] — would communicate input-wire labels separately for each branch (e.g., via a per-branch garbled selection table). With $B = 2^n$ branches and input length I, this incurs $\Theta(BI)$ overhead — precisely the cost we seek to remove.
- **A tempting but insufficient compression attempt.** One might hope to generate all dummy label vectors from a short PRF key, so E can derive them locally. However, this runs into an immediate obstacle: on the active branch, the correct labels necessarily depend on the garbled input, so they cannot be fixed independently of the input. Still, this attempt is instructive: on every inactive branch, the PRF output looks pseudorandom (hence indistinguishable from the active branch) and is independent of the garbled input.
- **Programmability resolves the obstacle.** We instead use a *programmable* PRF: we generate a key that is "patched" at exactly one point so that its output equals the *correct* input-wire labels on the active branch,

while its outputs on all other branches remain pseudorandom. Crucially, the programmed key hides the patched point and hence does not reveal the active branch identity.

- **Non-adaptive programmability is sufficient.** At this point, a full programmable PRF would impose more structure than SGC needs. In SGC, inactive-branch label vectors can depend on *which* branch is active; we only require that they be independent of the garbled input. That is, once the active branch is fixed, changing the programmed output should not affect the PRF output on any other branch — this is exactly a napPRF.

- **Garbled programmer gadget.** Programming requires (a) the identity of the active branch and (b) the input-wire labels for that branch. In our setting, neither party can simply write these values down in the clear; they are available only as garbled labels. Our solution is to realize the programming procedure itself as a small GC (a "programmer gadget") that takes the selector labels together with the input-wire labels and outputs a programmed key.

- **Offline-online napPRF.** In particular, we use an *offline-online* napPRF so that the programmed key decomposes into an *offline* part that is independent of both the active branch and the input, and a short *online* part that depends on them. The garbled programmer gadget outputs only the online part, while the offline part can be sent in the clear. This is strictly beneficial: outputting a long offline key from a garbled circuit would bloat the gadget (one output wire per bit), yet hiding it buys no security.

- **Two remaining challenges.** We have two further major challenges.

  1. Garbling the napPRF programming procedure would potentially lead to non-black use of the underlying cryptographic primitives (including OWFs): something we are trying to avoid.
  2. At first glance, the programmer gadget seems to need access to the input-wire labels of GC for *every* branch, since it is not known in advance which branch will be programmed. Storing these labels explicitly would make the gadget (and hence the garbling) too large, essentially reintroducing a $\Theta(BI)$ cost. A natural workaround is to generate all per-branch label vectors from a short PRG seed, and have the gadget store only this seed. However, this pushes the burden into the gadget: it must now *expand* the PRG (and thus rely on its internal structure) inside the garbled circuit, which is exactly the kind of non-black-box use we seek to avoid.

We next preview how to address the remaining challenges; but even before doing so, if Prog has small Boolean circuit complexity (polynomial in n, m, λ, where $n = \log B$ and $m = I\lambda$ here), this blueprint yields an SGC demultiplexer of garbled size $\mathsf{poly}(\log B, I, \lambda)$, already improving over prior $\mathcal{O}(BI\lambda)$-size demultiplexers.

Resolving the challenges via our offline-online napPRFs. In this work, we construct a *short-key* (and *suffix-invariant*) offline-online napPRF over a polynomial-size domain from OWFs in a black-box way. In our construction, Prog essentially selects among $n + m$ standard Yao label pairs, so Challenge 1 can be handled using standard garbled truth tables (black-box in OWFs). Plugging this napPRF into the blueprint yields an SGC demultiplexer of garbled size $\mathcal{O}(nI \cdot \mathsf{poly}(\lambda) + n^2\lambda)$, and total SGC size $\mathcal{O}(nI \cdot \mathsf{poly}(\lambda) + B\lambda + C\lambda)$, where the $\mathcal{O}(B\lambda)$ term is inherited from SGC's oblivious seed-delivery gadget. We overview this napPRF construction in Section 2.5.

We further tailor this offline-online napPRF to SGC by *coupling* its internal GGM/PRG expansions with SGC's label generation, so the programmed value need not be materialized as an explicit $I\lambda$-bit string inside

a garbled circuit. That is, the tailored napPRF directly outputs GC labels (e.g., when $I = 1$, programming $(\alpha, 0)$ makes $\mathsf{Eval}(\mathsf{sk}, \alpha)$ output the *actual* SGC λ-bit wire label encoding 0 on branch α, and similarly for 1). This makes Challenge 2 fully black-box with respect to OWFs as well, yielding an SGC demultiplexer of garbled size $\mathcal{O}(nI\lambda + B\lambda)$ and total SGC size $\mathcal{O}(nI\lambda + B\lambda + C\lambda)$.

Toward preserving state-of-the-art SGC computation complexity. Beyond reducing garbled size, we also preserve LogStack's $\mathcal{O}(B \log B)$ computation for both parties [HK21]. This relies on our offline-online napPRFs as follows.

- **Garbler.** The garbler runs KeyGen, incurring $\mathcal{O}(B)$ computation.
- **Evaluator.** The evaluator requires outputs for all $t \in \{0,1\}^n$; rather than $B = 2^n$ separate calls to Eval, our napPRFs support a FullEval procedure that evaluates the napPRF on the entire domain in $\mathcal{O}(B \log B)$ time.
- **Suffix invariance.** We remark that suffix invariance is crucial for preserving LogStack's $\mathcal{O}(B \log B)$ computation. At a high level, during a subsequent *garbage collection* step, the garbler must predict, for each branch $t \in \{0,1\}^n$, the dummy output that E obtains on that (inactive) branch in order to combine B candidate outputs into a single garbled label. Without suffix invariance, the dummy output on a fixed t could vary with the full identity of α, yielding up to B possibilities per branch and total work $\mathcal{O}(B^2)$ — the bottleneck in the original SGC scheme [HK20a]. Suffix invariance restricts this dependence to only $\mathcal{O}(n)$ possible dummy outcomes, so the total work is $\mathcal{O}(B \log B)$.

See Section 7 for full details. The remainder of this section focuses on constructing various (offline-online) napPRFs.

2.4 Black-box Equivalence between napPRFs and Two-party DPFs

Recall that a two-party DPF encodes a point function $P_{\alpha,\gamma} : \mathcal{X} \to \mathcal{Y}$ — which outputs γ on input α and 0^m elsewhere — into two short keys (k_1, k_2). Given k_b and x, party $b \in \{1,2\}$ locally outputs y_b such that $y_1 \oplus y_2 = P_{\alpha,\gamma}(x)$, while each individual key hides (α, γ).

It is easy to see that a napPRF implies a two-party DPF in a black-box way.

Here, we focus on the converse direction. Consider $\mathcal{Y} = \{0,1\}$ and a DPF key pair $(k_1, k_2) \leftarrow \mathsf{Gen}(1^\lambda, \alpha, 1)$. By DPF correctness, the two local DPF evaluations coincide at every $x \neq \alpha$, while at $x = \alpha$ they differ by 1 (i.e., one evaluates to 0 and the other to 1). To obtain a correct napPRF (including off-point correctness), it suffices to output as the programmed key whichever of k_1 and k_2 evaluates to β at α. This DPF-based trick echoes the one-point equivocation underlying *somewhere equivocal encryption* [HJO+16], and we draw inspiration from it.

This approach ensures index-hiding security for the induced napPRF. At a high level, when $\beta \xleftarrow{\$} \{0,1\}$ is uniform, Prog outputs k_1 or k_2 with probability exactly $1/2$ each. By DPF security, each share distribution is simulatable without α, and hence so is their half-half mixture.

To support range $\{0,1\}^m$ for any m, we simply repeat this procedure independently for each output bit and concatenate the resulting keys.

See Section 5 for formal details of this equivalence. Instantiating it with the well-known OWF-based DPF constructions [GI14, BGI15] yields a napPRF from black-box OWFs with total key size $nm\lambda + 2nm + m\lambda$ bits. However, this particular OWF-based instantiation has two drawbacks:

1. **No meaningful offline-online split.** The programmed key is generated entirely as a function of (α, β), i.e., there is no nontrivial (α, β)-independent offline component (and the total key size scales with nm).

2. **Missing suffix invariance.** The induced napPRF does *not* satisfy suffix invariance: off-point behavior depends on the full identity of α.

These limitations persist for the constructions of [GI14, BGI15] absent substantial modifications. Intuitively, the so-called *correction words* underlying these constructions depend on α, so the resulting key structure varies with α.

Finally, we highlight an additional result of independent interest. In Section 5.4, we demonstrate that programmable DPFs [BGIK22b] imply *adaptive* programmable PRFs via the same black-box reduction, giving the first OWF-based construction (for polynomial-size domains) of apPRFs.[2]

2.5 Constructing Short-key Offline-online napPRFs

We discuss how to construct short-key offline-online napPRFs. If we allow non-black-box techniques, there is a straightforward compiler: given any napPRF $\Pi = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ whose Prog has small Boolean circuit complexity, we garble Prog [Yao86] and place the garbling in $\mathsf{sk}_{\mathsf{off}}$, so that $\mathsf{sk}_{\mathsf{on}}$ consists only of the input labels for (α, β) and has size $|\mathsf{sk}_{\mathsf{on}}| = (n + m)\lambda$.

However, this generic compilation is not ideal. First, it is inherently non-black-box: when instantiated from OWF-based (DPF-induced) napPRFs, it requires garbling large subcircuits (notably the $\mathcal{O}(nm)$ PRG expansions), leading to substantial overhead in $\mathsf{sk}_{\mathsf{off}}$. Second, this compilation does not endow suffix invariance: if the underlying napPRF lacks it, so does the compiled one.

To this end, we give a nontrivial construction of *short-key and suffix-invariant* offline-online napPRFs over polynomial-size domains from black-box OWFs. Interestingly, this construction is inspired by the stacked garbling technique.

A warm-up: extending the domain by one bit. Before presenting the full construction, it is helpful to isolate an instructive subproblem. Suppose we have an offline-online napPRF $\Pi^{(k)} : \{0,1\}^k \to \{0,1\}^m$ whose online key consists of only $k+m$ Yao labels (e.g., as obtained by the generic garbling-based compiler). Can we construct an offline-online napPRF $\Pi^{(k+1)} : \{0,1\}^{k+1} \to \{0,1\}^m$ — i.e., extend the domain by one bit — treating $\Pi^{(k)}$ as a black-box building block?

Here is **a natural first (but flawed) attempt**. The offline key simply contains *two independent copies* of the k-bit napPRF's offline key, one meant for inputs whose first bit is 0 and the other for inputs whose first bit is 1.

To program a point $\alpha = \alpha_1 \| \alpha'$ with value β, we proceed as follows. If $\alpha_1 = 0$, we run the k-bit programming procedure on the "0-copy" at index α' with value β and take the resulting $k+m$ labels as the online key. If $\alpha_1 = 1$, we do the same but on the "1-copy." In either case, the online key consists of exactly $k+m$ garbled labels; the only difference is which copy they belong to.

To evaluate at an input $x = x_1 \| x'$, we route to the corresponding copy: we run the k-bit evaluation procedure using the offline key of the x_1-copy, together with the online key and the suffix x'.

It is easy to see that this attempt satisfies on-point correctness: when $x = \alpha$, the output of the (routed) k-bit evaluation procedure is exactly β.

[2] We note that [GZSP25] gives an attempt, but it incurs a $1/\mathsf{poly}(\lambda)$ security loss.

This attempt fails as it only *partially* satisfies off-point value independence. To see this, w.l.o.g. assume $\alpha = 0^{k+1}$, so programming is performed on the 0-copy at point $\alpha' = 0^k$. When $x = 0\|x'$, we do achieve off-point value independence for this α: evaluation always invokes the 0-copy, so it reduces to the k-bit napPRF's off-point value independence. However, when $x = 1\|x'$, we fail even for this α: evaluation invokes the 1-copy but is given an *ill-formed* online key for that copy. Indeed, the online key consists of labels produced for the 0-copy and varies with β, so we cannot appeal to the k-bit napPRF off-point correctness of the 1-copy.

This suggests a simple patch. During the $(k+1)$-bit programming procedure, we additionally include a uniformly sampled *dummy* online key for the $\overline{\alpha}_1$-copy (i.e., $k+m$ random garbled labels). The online key now contains two parts, one for each copy, and evaluation routes to the appropriate part. Crucially, when evaluation goes into the $\overline{\alpha}_1$-copy, its online key is independent of β.

We briefly examine the index-hiding security of this $(k+1)$-bit construction. Both components hide α': the real one by the index-hiding of $\Pi^{(k)}$, and the dummy one by uniformity. Thus, the only remaining potential leakage is α_1, via distinguishing which online key is real. Ensuring this indistinguishability depends on how we instantiate $\Pi^{(k)}$, so we defer it to the end.

Garble-then-recurse. Given a $(k+1)$-bit offline-online napPRF, the natural next step is to extend the domain further to $(k+2)$ bits by applying the above idea once more. However, an immediate issue arises: the $(k+1)$-bit online key already contains two components (one per copy), and thus has size proportional to $2(k+m)$. Repeating the same transformation would double the number of components again, causing the online key to grow exponentially with the recursion depth. (A similar issue arises for the offline key; we address it shortly.)

The fix is simple: we further apply the garbling-based compiler idea to the $(k+1)$-bit construction, thereby *compressing* its online key. Concretely, we represent the $(k+1)$-bit programming procedure itself as a small garbled circuit stored in the offline key, so that the online key consists only of the input labels for (α, β) and has size $((k+1) + m)\lambda$. More explicitly, we proceed as follows:

- In the master key, sample $(k+1) + m$ Yao label pairs corresponding to the bits of $\alpha \in \{0,1\}^{k+1}$ and $\beta \in \{0,1\}^m$. The online key is simply the selection of one label per bit, determined by (α, β).
- In the offline key, include a garbling of the "programmer gadget" implementing the $(k+1)$-bit programming procedure described above. Given the input labels for (α, β), this garbled circuit outputs the two *internal* online keys (one real and one dummy) for the two k-bit copies.

Crucially, the $(k+1)$-bit programming procedure performs only simple selection among garbled labels from the two k-bit copies, together with dummy padding. Hence, the programmer gadget can be implemented via simple garbled tables.

For example, consider the case $m = 1$, so $\beta \in \{0,1\}$ is a single bit. $\mathsf{KeyGen}^{(k+1)}$ samples Yao labels for the selector bit α_1 and the programmed bit β:

$$\mathcal{L}^{(0)}_{\alpha_1}, \mathcal{L}^{(1)}_{\alpha_1}, \mathcal{L}^{(0)}_{\beta}, \mathcal{L}^{(1)}_{\beta} \in \{0,1\}^\lambda$$

The two k-bit copies come with their own β-labels:

$$\mathcal{L}^{(0)}_{\beta,0}, \mathcal{L}^{(1)}_{\beta,0} \in \{0,1\}^\lambda \text{ (0-copy)} \qquad \mathcal{L}^{(0)}_{\beta,1}, \mathcal{L}^{(1)}_{\beta,1} \in \{0,1\}^\lambda \text{ (1-copy)}$$

To implement the programmer gadget for β, $\mathsf{KeyGen}^{(k+1)}$ samples a dummy label $\mathcal{J}_\beta$ and garbles the 2×2 translation table as below:

$$\begin{array}{ll} \mathsf{Enc}_{\mathcal{L}^{(0)}_{\alpha_1}, \mathcal{L}^{(0)}_{\beta}}(\mathcal{L}^{(0)}_{\beta,0} \parallel \mathcal{J}_\beta) & \mathsf{Enc}_{\mathcal{L}^{(0)}_{\alpha_1}, \mathcal{L}^{(1)}_{\beta}}(\mathcal{L}^{(1)}_{\beta,0} \parallel \mathcal{J}_\beta) \\ \mathsf{Enc}_{\mathcal{L}^{(1)}_{\alpha_1}, \mathcal{L}^{(0)}_{\beta}}(\mathcal{J}_\beta \parallel \mathcal{L}^{(0)}_{\beta,1}) & \mathsf{Enc}_{\mathcal{L}^{(1)}_{\alpha_1}, \mathcal{L}^{(1)}_{\beta}}(\mathcal{J}_\beta \parallel \mathcal{L}^{(1)}_{\beta,1}) \end{array}$$

(with the usual random permutation / point-and-permute indexing). Given the online labels $(\mathcal{L}^{(\alpha_1)}_{\alpha_1}, \mathcal{L}^{(\beta)}_{\beta})$, one can decrypt one entry and obtain a pair consisting of the *real* label for copy α_1 and a dummy label for the other copy — exactly what we need. For instance, if $(\alpha_1, \beta) = (0, 0)$, decryption yields $\mathcal{L}^{(0)}_{\beta,0} \| \mathcal{J}_\beta$.

We can now obtain the $(k{+}2)$-bit construction by applying the above transformation to the compressed-online-key $(k{+}1)$-bit construction. More generally, repeating this step yields a $(k{+}n)$-bit construction for any $n \geq 1$ whose online key has size $((k{+}n) + m)\lambda$, *once we can control the growth of the offline key.*

Stacking the child offline keys. We now show how to avoid exponential growth in the offline key. So far, the $(k{+}1)$-bit construction stores two child offline keys (two k-bit copies); if repeated naïvely, the offline key would double at each level. Inspired by the stacked garbling technique, we instead store only a *stacked* representation of the two child offline keys (i.e., their XOR). During evaluation, the napPRF evaluator can "unstack" and recover the offline key for the α_1-copy using randomness revealed for the other copy.

In more detail, $\mathsf{KeyGen}^{(k+1)}$ runs $\mathsf{KeyGen}^{(k)}$ twice with seeds seed_0 and seed_1 (PRG-expanded if needed), producing two child-copy offline keys $\mathsf{sk}_{\mathsf{off},0}$ and $\mathsf{sk}_{\mathsf{off},1}$. It then sets its offline key $\mathsf{sk}^{(k+1)}_{\mathsf{off}}$ to include:

- The stacked child offline key $\mathsf{sk}_{\mathsf{stack}} := \mathsf{sk}_{\mathsf{off},0} \oplus \mathsf{sk}_{\mathsf{off},1}$.
- A tiny garbled table that reveals the *other* seed (i.e., the unprogrammed half):

$$\mathsf{Enc}_{\mathcal{L}^{(0)}_{\alpha_1}}(\mathsf{seed}_1) \quad \mathsf{Enc}_{\mathcal{L}^{(1)}_{\alpha_1}}(\mathsf{seed}_0) \qquad \text{(randomly permuted)}$$

Given $\mathcal{L}^{(\alpha_1)}_{\alpha_1}$ in the online key, $\mathsf{Eval}^{(k+1)}$ decrypts the table and learns $\mathsf{seed}_{\overline{\alpha}_1}$. It then recomputes $\mathsf{sk}_{\mathsf{off},\overline{\alpha}_1}$ from $\mathsf{seed}_{\overline{\alpha}_1}$ and unstacks by XORing with $\mathsf{sk}_{\mathsf{stack}}$:

$$\mathsf{sk}_{\mathsf{off},\alpha_1} \;=\; \mathsf{sk}_{\mathsf{stack}} \oplus \mathsf{sk}_{\mathsf{off},\overline{\alpha}_1}$$

Finally, it invokes $\mathsf{Eval}^{(k)}$ using the recovered child offline key.

We briefly revisit off-point value independence. Note that, regardless of x, the unstacked offline key is always $\mathsf{sk}_{\mathsf{off},\alpha_1}$. In particular, when $x_1 \neq \alpha_1$, $\mathsf{Eval}^{(k+1)}$ pairs $\mathsf{sk}_{\mathsf{off},\alpha_1}$ with the *dummy* internal online key — both are independent of β.

Finally, with stacked offline keys in place, the garble-then-recurse blueprint extends to any depth. Concretely, for any $n \geq 1$ it yields an offline-online napPRF $\Pi^{(k+n)}$ whose online key has size $((k{+}n){+}m)\lambda$ and whose offline key has size $\mathcal{O}(nm\lambda + n^2\lambda + |\mathsf{sk}^{(k)}_{\mathsf{off}}|)$. Crucially, the construction treats $\Pi^{(k)}$ as a black box and uses only black-box OWFs.

Our construction: starting from $\Pi^{(0)}_{\mathsf{napPRF}}$. Our short-key (and suffix-invariant) offline-online napPRFs follow the above bit-by-bit extension blueprint, starting from a simple base case offline-online napPRF $\Pi^{(0)}_{\mathsf{napPRF}}$ for the singleton domain.

For the singleton domain $\{0,1\}^0 = \{\epsilon\}$, programming outputs m garbled labels. The offline key $\mathsf{sk}^{(0)}_{\mathsf{off}}$ contains one decoding bit per wire (via point-and-permute [BMR90]), so that $\mathsf{Eval}^{(0)}$ can recover $\beta \in \{0,1\}^m$ from the m labels in $\mathsf{sk}^{(0)}_{\mathsf{on}}$. We define the base case in this form to match the interface of our blueprint.

Then $\mathsf{sk}^{(0)}_{\mathsf{off}}$ defines schemes $\{\Pi^{(k)}_{\mathsf{napPRF}}\}_{k=0}^{n}$, where $\Pi^{(k)}_{\mathsf{napPRF}}$ has domain $\{0,1\}^k$.

See Section 6 for formal details; we conclude this section with several remarks:

- **Suffix invariance.** Our offline-online napPRFs satisfy suffix invariance, and the proof is by a simple induction on k. Consider the $(k{+}1)$-bit construction. If $x_1 \neq \alpha_1$, evaluation uses the α_1-copy offline key (recovered by unstacking) together with the dummy labels; neither depends on the programmed suffix α', so the output is invariant under changing α'. If $x_1 = \alpha_1$, evaluation uses the same α_1-copy offline key together with the real labels, and suffix invariance reduces to that of the k-bit instance. Since $\Pi^{(0)}_{\mathsf{napPRF}}$ is trivially suffix-invariant, it follows that $\Pi^{(k)}_{\mathsf{napPRF}}$ is suffix-invariant for all k (including $k = n$).

- **Security.** We briefly discuss why our offline-online napPRFs satisfy index-hiding. Recall that, in the $(k{+}1)$-bit step, the only new leakage channel is the first programmed bit α_1 via distinguishing the real from the dummy.

  In particular, fix the $(k{+}1)$-bit construction. First, the *seed table* reveals only the $\overline{\alpha}_1$-copy seed; the entry containing the α_1-copy seed is undecryptable. Second, the *translation table* reveals only the entry decrypted by the online labels, and in that entry exactly one half contains the *real* child labels (for α_1-copy) while the other half is dummy. Thus, the adversary's view is consistent with the recovered α_1-copy offline key together with a candidate internal online key. Consequently, distinguishing whether this candidate internal online key is *well-formed* (real) or uniformly random reduces to a k-bit guarantee: given the corresponding offline key, the online key output by $\mathsf{Prog}^{(k)}$ is indistinguishable from a uniformly random bundle of $k{+}m$ garbled labels.

  The base case $k = 0$ is immediate when β is uniformly sampled: $\mathsf{sk}^{(0)}_{\mathsf{on}}$ is just a uniformly random choice among the two labels on each wire (up to public permutation bits), hence indistinguishable from uniform. Assuming the claim for level k, it follows that the real internal online key at level $(k{+}1)$ can be replaced by a uniform dummy without changing the adversary's view; this hides which component is real and therefore hides α_1.

- **Polynomial-size domains.** Finally, we remark that this construction targets polynomial-size domains (sufficient for our SGC application). Conceptually, the bit-by-bit extension unfolds into a full binary recursion tree of depth n, with 2^n leaves, and thus incurs $\Theta(2^n)$ work in $\mathsf{KeyGen}^{(n)}$ and $\mathsf{Eval}^{(n)}$.

3 Preliminaries

3.1 Notation

Throughout, λ denotes the security parameter and $\approx_c$ denotes computational indistinguishability (for ensembles of random variables or oracles). We write n and m for the input (domain) and output (range)

lengths (often suppressing their dependence on λ), and let $B := 2^n$. Unless stated otherwise, $\mathcal{X} := \{0,1\}^n$ and $\mathcal{Y} := \{0,1\}^m$ are public parameters known to all algorithms (including simulators). We write $\xleftarrow{\$}$ for uniform sampling, use 1-based indexing for vectors unless noted, and let $[k] = \{1, 2, \ldots, k\}$ with $[0] := \emptyset$. When we quantify the computation cost of a construction, we count the number of invocations on the dominating cryptographic primitive.

3.2 Distributed Point Functions

A (two-party) *distributed point function* (DPF) [BGI15, GI14] enables a dealer to secret share a point function as two compacts keys.

Point functions. Let $\mathcal{X}$ denote a finite domain and $\mathcal{Y}$ an abelian group. For $\alpha \in \mathcal{X}$ and $\beta \in \mathcal{Y}$, define the point function $P_{\alpha,\beta} : \mathcal{X} \to \mathcal{Y}$ by $P_{\alpha,\beta}(x) = \beta$ if $x = \alpha$, and $P_{\alpha,\beta}(x) = 0$ otherwise.

Definition 1 (Two-party DPF). *A two-party **distributed point function** (DPF) scheme consists of two PPT algorithms:*
- $\mathsf{Gen}(1^\lambda, \alpha, \beta)$ *outputs a pair of keys* (k_1, k_2)*, and*
- $\mathsf{Eval}(k, x)$ *outputs a value in* $\mathcal{Y}$ *for* $k \in \{k_1, k_2\}$ *and* $x \in \mathcal{X}$*.*

*The scheme is **correct** if for all* $\lambda \in \mathbb{N}$*, all* $\alpha \in \mathcal{X}$*, all* $\beta \in \mathcal{Y}$*, and all* $x \in \mathcal{X}$*, the following equality holds with probability* 1*:*

$$\mathsf{Eval}(k_1, x) + \mathsf{Eval}(k_2, x) = P_{\alpha,\beta}(x) \;\Big|\; (k_1, k_2) \leftarrow \mathsf{Gen}(1^\lambda, \alpha, \beta)$$

Security. A two-party DPF scheme $\Pi = (\mathsf{Gen}, \mathsf{Eval})$ is *secure* if for each $b \in \{1, 2\}$ there exists a PPT simulator Sim_b such that for all $\alpha \in \mathcal{X}$ and $\beta \in \mathcal{Y}$, the following ensembles are computationally indistinguishable:

$$\left\{k_b \;\middle|\; (k_1, k_2) \leftarrow \mathsf{Gen}(1^\lambda, \alpha, \beta)\right\}_\lambda \approx_c \left\{\mathsf{Sim}_b(1^\lambda)\right\}_\lambda$$

Lemma 1 (DPFs from OWFs [GI14, BGI15]). *Assuming one-way functions, there exists a correct and secure two-party DPF scheme* Π_{DPF} *for functions* $\{0,1\}^n \to \{0,1\}$*, with only black-box use of OWFs, such that each share* k_b *(for* $b \in [2]$*) has length* $n\lambda + 2n + \lambda$ *bits. Moreover, both* Gen *and* Eval *run in time* $\mathcal{O}(n)$ *and make* $\mathcal{O}(n)$ *calls to a length-doubling PRG.*

3.3 Pseudorandom Functions

A *pseudorandom function* (PRF) [GGM86] is an efficiently computable function family that is computationally indistinguishable from a truly random function.

Definition 2 (PRF). *Let* λ *be the security parameter. A function family* $F = \{F_k : \mathcal{X} \to \mathcal{Y}\}_{k \in \mathcal{K}}$ *is a **pseudorandom function** (PRF) if there exists a PPT key generation algorithm* $\mathsf{KeyGen}(1^\lambda)$ *that outputs a key* $k \in \mathcal{K}$*, and a deterministic poly-time evaluation algorithm s.t. the following oracles are indistinguishable:*

$$\left\{F_k(\cdot) \;\middle|\; k \leftarrow \mathsf{KeyGen}(1^\lambda)\right\}_\lambda \approx_c \left\{R(\cdot) \;\middle|\; R \leftarrow \mathsf{Rand}(\mathcal{X}, \mathcal{Y})\right\}_\lambda$$

Here, $\mathsf{Rand}(\mathcal{X}, \mathcal{Y})$ *denotes the uniform distribution over all functions mapping* $\mathcal{X}$ *to* $\mathcal{Y}$*, and* $\mathcal{A}$ *is given oracle access to either* $F_k(\cdot)$ *or a truly random function* $R(\cdot)$*.*

3.4 Programmable PRFs

We recall the definition of (adaptive) programmable PRFs from [PS18, GZSP25].

Definition 3 (apPRF). *Fix input length n and output length m, and let $\mathcal{X} := \{0,1\}^n$ and $\mathcal{Y} := \{0,1\}^m$. An **(adaptive) programmable PRF** (apPRF) for a* single *point is a tuple of PPT algorithms $\Pi = (\mathsf{KeyGen}, \mathsf{MEval}, \mathsf{Prog}, \mathsf{Eval})$ with the following interfaces (we omit 1^λ when clear):*

$$\begin{aligned} \mathsf{msk} &\leftarrow \mathsf{KeyGen}(1^\lambda) \\ y &:= \mathsf{MEval}(\mathsf{msk}, x) \in \mathcal{Y} && \textit{(deterministic)} \\ \mathsf{sk} &:= \mathsf{Prog}(\mathsf{msk}, \alpha, \beta) && \textit{(deterministic)} \\ y &:= \mathsf{Eval}(\mathsf{sk}, x) \in \mathcal{Y} && \textit{(deterministic)} \end{aligned}$$

*The scheme is **correct** if for all $\lambda \in \mathbb{N}$, all $\mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda)$, all indices $\alpha \in \mathcal{X}$, all values $\beta \in \mathcal{Y}$, let $\mathsf{sk} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$, the following properties hold.*

1. ***Correct programming at** α. That is, $\mathsf{Eval}(\mathsf{sk}, \alpha) = \beta$.*
2. ***Off-point value unchanged.** $\forall x \in \mathcal{X} \setminus \{\alpha\}$, $\mathsf{Eval}(\mathsf{sk}, x) = \mathsf{MEval}(\mathsf{msk}, x)$.*

Security (index hiding). An apPRF scheme $\Pi = (\mathsf{KeyGen}, \mathsf{MEval}, \mathsf{Prog}, \mathsf{Eval})$ is *index hiding* if there exists a PPT simulator Sim such that for every index $\alpha \in \mathcal{X}$ the following ensembles are computationally indistinguishable:

$$\left\{ \mathsf{sk} \;\middle|\; \begin{array}{l} \mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda), \\ \beta \xleftarrow{\$} \mathcal{Y}, \\ \mathsf{sk} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta) \end{array} \right\}_\lambda \approx_c \left\{ \mathsf{Sim}(1^\lambda) \right\}_\lambda$$

That is, when programmed value β is uniform, the distribution of sk can be efficiently simulated without the knowledge of programmed index α.

Security (PRF under uniform programming). An apPRF scheme $\Pi = (\mathsf{KeyGen}, \mathsf{MEval}, \mathsf{Prog}, \mathsf{Eval})$ is *PRF-secure under uniform programming* if for every $\alpha \in \mathcal{X}$, the following oracle ensembles are computationally indistinguishable:

$$\left\{ \mathsf{Eval}(\mathsf{sk}, \cdot) \;\middle|\; \begin{array}{l} \mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda), \\ \beta \xleftarrow{\$} \mathcal{Y}, \\ \mathsf{sk} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta) \end{array} \right\}_\lambda \approx_c \left\{ R(\cdot) \xleftarrow{\$} \mathsf{Rand}(\mathcal{X} \to \mathcal{Y}) \right\}_\lambda$$

where $R : \mathcal{X} \to \mathcal{Y}$ is a uniformly random function. That is, when the programmed value is uniform, the induced oracle $x \mapsto \mathsf{Eval}(\mathsf{sk}, x)$ is a PRF.

Remarks. We make several remarks on the above definitions.

- The index-hiding notion is also referred to as *private programmability* (or *privately programmable*) in prior work [PS18, GZSP25].
- Our results in Section 4 show that *correctness* together with *index hiding* already suffices to derive *PRF security under uniform programming*. To the best of our knowledge, this implication has not been explicitly observed before.

- When the domain size $|\mathcal{X}|$ is superpolynomial, PRF security under uniform programming further implies that $\mathsf{MEval}(\mathsf{msk}, \cdot)$ can be used as a standard PRF. In contrast, for polynomial-size domains, this implication fails in general. We provide a proof and an explicit counterexample in Appendix A.
- Existing constructions of apPRFs with index hiding rely on strong assumptions (e.g., LWE and/or indistinguishability obfuscation). Although our main focus is on napPRFs, in Section 5.4, we show an apPRF scheme with index hiding can be built only from one-way functions for a polynomial-size domain.

4 Definitions of Non-adaptive Programmable PRFs

Definition 4 (napPRF). *Fix input length n and output length m, and let $\mathcal{X} := \{0,1\}^n$ and $\mathcal{Y} := \{0,1\}^m$. A **non-adaptive programmable PRF** (napPRF) for a single point is a tuple of PPT algorithms $\Pi = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ with the following interfaces (we omit 1^λ when clear):*

$$\begin{aligned} \mathsf{msk} &\leftarrow \mathsf{KeyGen}(1^\lambda) \\ \mathsf{sk} &:= \mathsf{Prog}(\mathsf{msk}, \alpha, \beta) && \textit{(deterministic)} \\ y &:= \mathsf{Eval}(\mathsf{sk}, x) \in \mathcal{Y} && \textit{(deterministic)} \end{aligned}$$

where msk *is the* master secret key*,* sk *is the* programmed key*. We view* Eval *as defining a keyed function* $F_{\mathsf{sk}}(\cdot) : \mathcal{X} \to \mathcal{Y}, F_{\mathsf{sk}}(x) := \mathsf{Eval}(\mathsf{sk}, x)$.

*The scheme is **correct** if for all $\lambda \in \mathbb{N}$, all $\mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda)$, all indices $\alpha \in \mathcal{X}$, and all values $\beta, \beta_1, \beta_2 \in \mathcal{Y}$, the following properties hold.*

1. ***Correct programming at** α. Let* $\mathsf{sk} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$, $\mathsf{Eval}(\mathsf{sk}, \alpha) = \beta$.
2. ***Off-point value independence.** Let* $\mathsf{sk}^{(i)} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta_i)$ *for* $i \in \{1, 2\}$*. Then for every* $x \in \mathcal{X} \setminus \{\alpha\}$, $\mathsf{Eval}(\mathsf{sk}^{(1)}, x) = \mathsf{Eval}(\mathsf{sk}^{(2)}, x)$.

Security (index hiding). A napPRF scheme $\Pi = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ is *index hiding* if there exists a PPT simulator Sim such that for every index $\alpha \in \mathcal{X}$ the following ensembles are computationally indistinguishable:

$$\left\{ \mathsf{sk} \;\middle|\; \begin{array}{l} \mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda), \\ \beta \xleftarrow{\$} \mathcal{Y}, \\ \mathsf{sk} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta) \end{array} \right\}_\lambda \approx_c \left\{ \mathsf{Sim}(1^\lambda) \right\}_\lambda$$

That is, when programmed value β is uniform, the distribution of sk can be efficiently simulated without the knowledge of programmed index α.

Security (PRF under uniform programming). A napPRF $\Pi = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ is *PRF-secure under uniform programming* if for every $\alpha \in \mathcal{X}$, the following oracle ensembles are computationally indistinguishable:

$$\left\{ \mathsf{Eval}(\mathsf{sk}, \cdot) \;\middle|\; \begin{array}{l} \mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda), \\ \beta \xleftarrow{\$} \mathcal{Y}, \\ \mathsf{sk} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta) \end{array} \right\}_\lambda \approx_c \left\{ R(\cdot) \xleftarrow{\$} \mathsf{Rand}(\mathcal{X} \to \mathcal{Y}) \right\}_\lambda$$

where $R : \mathcal{X} \to \mathcal{Y}$ is a uniformly random function. That is, when the programmed value is uniform, the induced oracle $x \mapsto \mathsf{Eval}(\mathsf{sk}, x)$ is a PRF.

Perhaps surprisingly, in Section 5 we show that correctness and index hiding already imply PRF security under uniform programming, that is:

Theorem 5 (Index-hiding napPRF $\Rightarrow$ PRF under uniform programming). *Let* $\Pi_{\mathsf{napPRF}} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ *be a napPRF. If* Π_{napPRF} *is* correct *and* index hiding*, then it is* PRF-secure under uniform programming.

Definition 5 (Offline-online napPRF). *An* offline-online napPRF *is a tuple of PPT algorithms* $\Pi = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ *such that* $\mathsf{KeyGen}(1^\lambda)$ *samples* $(\mathsf{msk}, \mathsf{sk_{off}})$, $\mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$ *outputs* $\mathsf{sk_{on}}$*, and* $\mathsf{Eval}(\mathsf{sk_{off}}, \mathsf{sk_{on}}, x)$ *outputs* $y \in \mathcal{Y}$*. The programmed key is* $\mathsf{sk} := (\mathsf{sk_{off}}, \mathsf{sk_{on}})$*, where* $\mathsf{sk_{off}}$ *is independent of* (α, β)*.*

All correctness/security notions extend by viewing the programmed key as $\mathsf{sk} = (\mathsf{sk_{off}}, \mathsf{sk_{on}})$. This is w.l.o.g.: a standard napPRF is the special case $\mathsf{sk_{off}} = \perp$, and conversely an offline-online napPRF can be viewed as a standard one by folding $\mathsf{sk_{off}}$ into msk and having Prog output it. Henceforth, we write napPRF for the standard syntax and *offline-online* napPRF when we use the split-key syntax.

Primary efficiency metrics and short-key offline-online napPRFs. Our primary efficiency measures are (i) the total programmed-key size $|\mathsf{sk}|$ and (ii) the online-key size $|\mathsf{sk_{on}}|$. We require $|\mathsf{sk}| \leq \mathsf{poly}(n, m, \lambda)$. We call an offline-online napPRF *short-key* if $|\mathsf{sk_{on}}|$ is linear in $n + m$ (up to $\mathsf{poly}(\lambda)$ factors).

Evaluation over the full domain. We also introduce a full-domain evaluation procedure. For any programmed key sk (possibly $\mathsf{sk} = (\mathsf{sk_{off}}, \mathsf{sk_{on}})$), define

$$\mathsf{FullEval}(\mathsf{sk}) \in (\{0,1\}^m)^{2^n}$$

as the vector indexed by $x \in \{0,1\}^n$ whose x-th entry is $\mathsf{Eval}(\mathsf{sk}, x)$.

Note that an implementation of $\mathsf{FullEval}$ is not required to compute the vector by explicitly invoking Eval on every input; in particular, a construction may admit a more efficient procedure for generating the full-domain evaluation.

4.1 Non-adaptive vs. Adaptive Programmable PRFs

We clarify the distinction between napPRFs and apPRFs (cf. Section 3.4).

Indeed, any apPRF $\Pi_{\mathsf{apPRF}} = (\mathsf{KeyGen}, \mathsf{MEval}, \mathsf{Prog}, \mathsf{Eval})$ that is correct and index-hiding immediately yields a (standard) napPRF in a black-box way by simply dropping the master evaluation algorithm MEval. Correctness and index hiding of Π_{apPRF} immediately carry over.

Note that Theorem 5 also applies to the napPRF induced from an apPRF as above. In particular, under this embedding, we have $\Pi_{\mathsf{napPRF}}.\mathsf{Eval} \equiv \Pi_{\mathsf{apPRF}}.\mathsf{Eval}$. This implies that for an apPRF, *correctness* together with *index hiding* already suffices to derive *PRF security under uniform programming* (cf. Section 3.4).

We now consider the converse direction and explain why, in general, a correct and index-hiding napPRF does *not* yield a correct and index-hiding apPRF via a black-box transformation. Interestingly, the obstruction is not security but *correctness*: an apPRF requires the existence of a single "master" evaluation procedure (i.e., MEval) that is preserved under programming.

To see this, recall that in an apPRF we have a master function $F(x) := \Pi_{\mathsf{apPRF}}.\mathsf{MEval}(\mathsf{msk}, x)$ such that programming at (α, β) produces a key sk whose induced oracle agrees with F everywhere except at α: $\forall x \neq \alpha, \Pi_{\mathsf{apPRF}}.\mathsf{Eval}(\mathsf{sk}, x) = F(x)$. That is, the off-point value is independent of *both* α and β.

In contrast, napPRF correctness guarantees only *off-point invariance with respect to the programmed value*: For every fixed α and all β, β', the evaluations on $x \neq \alpha$ are identical, but they may still depend arbitrarily on α. That is, while a napPRF can ensure that changing β does not affect off-point outputs, it does not enforce that programming different indices α's is consistent with a single underlying master function. This missing cross-α consistency is exactly what prevents a generic black-box construction of an apPRF from a napPRF.

Definition 6 (Cross-α consistency). *A napPRF* $\Pi = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ *is* cross-α consistent *if for all* $\lambda \in \mathbb{N}$*, all* $\mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda)$*, all indices* $\alpha_1, \alpha_2 \in \mathcal{X}$*, and all* $x \in \mathcal{X} \setminus \{\alpha_1, \alpha_2\}$ *the following property holds:*

$$\mathsf{Eval}\big(\mathsf{Prog}(\mathsf{msk}, \alpha_1, 0), x\big) = \mathsf{Eval}\big(\mathsf{Prog}(\mathsf{msk}, \alpha_2, 0), x\big)$$

Indeed, any napPRF that is correct, index-hiding, and cross-α consistent immediately yields a correct and index-hiding apPRF via a black-box construction. The transformation mirrors the trivial embedding of apPRFs into napPRFs, except for one caveat: we must define a master evaluation algorithm $\Pi_{\mathsf{apPRF}}.\mathsf{MEval}$. Cross-$\alpha$ consistency allows us to do so unambiguously: to evaluate at x, pick any $\alpha \in \mathcal{X} \setminus \{x\}$, and output $\Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\Pi_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha, 0), x)$.

In Section 5.4, we show how to construct a correct, index-hiding, and cross-α consistent napPRF over a polynomial-size domain from one-way functions. By the discussion above, this immediately yields a correct and index-hiding apPRF from one-way functions in the same regime, which (to our knowledge) was not previously known. We emphasize, however, that since the domain is only polynomial-size, this does not by itself imply that the resulting apPRF's master evaluation MEval is a standard PRF (cf. Sections 3.4 and 5.4).

4.2 Suffix Invariance

We introduce an additional correctness property of napPRFs that we will need for our application to stacked garbling. See Section 2.1 for a concise explanation.

Definition 7 (Suffix invariance). *Let* $\Pi = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ *be a napPRF over* $\mathcal{X} = \{0,1\}^n$ *and* $\mathcal{Y}$*. For* $x, \alpha \in \{0,1\}^n$ *with* $x \neq \alpha$*, define the* divergence

$$\div(x, \alpha) \;:=\; \min\{\, i \in [n] : x_i \neq \alpha_i \,\} \in [n]$$

and (optionally) set $\div(x, x) := n + 1$.

We say that Π *is* suffix-invariant *if for all* $\lambda \in \mathbb{N}$*, all* $\mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda)$*, all* $x \in \{0,1\}^n$*, all* $\alpha, \alpha' \in \{0,1\}^n \setminus \{x\}$*, and all* $\beta, \beta' \in \mathcal{Y}$*,*

$$\div(x, \alpha) = \div(x, \alpha') \implies \mathsf{Eval}\big(\mathsf{Prog}(\mathsf{msk}, \alpha, \beta), x\big) = \mathsf{Eval}\big(\mathsf{Prog}(\mathsf{msk}, \alpha', \beta'), x\big)$$

It is straightforward that suffix-invariant correctness strictly strengthens napPRF correctness while remaining strictly weaker than apPRF correctness.

5 Relationship Between napPRFs and Two-Party DPFs

In this section, we formalize a *black-box* equivalence between napPRFs and two-party DPFs. Specifically, we provide black-box reductions in both directions.

Π_{DPF}

- $\underline{\mathsf{Gen}(1^\lambda, \alpha, \beta)}$. Sample $\mathsf{msk} \leftarrow \Pi_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda)$ and $r \xleftarrow{\$} \mathcal{Y}$. Compute

$$\mathsf{sk}^{(1)} := \Pi_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha, r) \quad \text{and} \quad \mathsf{sk}^{(2)} := \Pi_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha, r \oplus \beta)$$

  Output $k_1 := \mathsf{sk}^{(1)}$ and $k_2 := \mathsf{sk}^{(2)}$.
- $\underline{\mathsf{Eval}(k_b, x) \text{ for } b \in \{1, 2\}}$. Output $\Pi_{\mathsf{napPRF}}.\mathsf{Eval}(k_b, x)$.

Figure 1: Constructing Π_{DPF} from $\Pi_{\mathsf{napPRF}} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ as a black box.

5.1 Individual Pseudorandomness for DPFs

We record a useful consequence of standard DPF security (cf. Section 3.2).

Individual pseudorandomness. A two-party DPF scheme $(\mathsf{Gen}, \mathsf{Eval})$ satisfies *individual pseudorandomness* if each individual key induces a function that is computationally indistinguishable from a uniformly random function. Formally, for each $b \in \{1, 2\}$ and for all $\alpha \in \mathcal{X}$ and $\beta \in \mathcal{Y}$, the following oracle distributions are computationally indistinguishable:

$$\left\{ \mathsf{Eval}(k_b, \cdot) \;\middle|\; (k_1, k_2) \leftarrow \mathsf{Gen}(1^\lambda, \alpha, \beta) \right\}_\lambda \approx_c \left\{ R(\cdot) \;\middle|\; R \leftarrow \mathsf{Rand}(\mathcal{X} \to \mathcal{Y}) \right\}_\lambda$$

where $R : \mathcal{X} \to \mathcal{Y}$ is a uniformly random function. That is, each share k_b defines a PRF over domain $\mathcal{X}$ and range $\mathcal{Y}$. Interestingly, correctness and standard DPF security already imply individual pseudorandomness:

Lemma 2 (DPF security ⇒ Individual pseudorandomness). *Let $\Pi = (\mathsf{Gen}, \mathsf{Eval})$ be a two-party DPF scheme. Suppose Π is* correct *and* secure *in the sense of per-share simulation security for both shares (defined in Section 3.2). Then Π satisfies* individual pseudorandomness.

We defer the proof to Appendix B.1. Jumping ahead, we use Lemma 2 to justify why index hiding implies PRF security under uniform programming in napPRFs.

5.2 napPRF ⇒ DPF

Let $\Pi_{\mathsf{napPRF}} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ be a (standard) napPRF scheme over domain $\mathcal{X} = \{0, 1\}^{n(\lambda)}$ and range $\mathcal{Y} = \{0, 1\}^{m(\lambda)}$ (where $\mathcal{Y}$ is viewed as an abelian group under $\oplus$). We construct a two-party DPF scheme $\Pi_{\mathsf{DPF}} = (\mathsf{Gen}, \mathsf{Eval})$ by using Π_{napPRF} as a black box, as depicted in Figure 1. We have:

Theorem 6 (napPRF ⇒ DPF). *If $\Pi_{\mathsf{napPRF}} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ is a* correct *and* index-hiding *napPRF for a single point over domain $\mathcal{X}$ and range $\mathcal{Y}$, then the construction in Figure 1 yields a two-party DPF $\Pi_{\mathsf{DPF}} = (\mathsf{Gen}, \mathsf{Eval})$ over domain $\mathcal{X}$ and range $\mathcal{Y}$ that is* correct *and* secure *(defined in Section 3.2).*

The intuition behind Theorem 6 is natural. DPF correctness follows since at $x = \alpha$ we have outputs r and $r \oplus \beta$, whose XOR equals β, while for every $x \neq \alpha$, off-point value independence gives $\Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(1)}, x) = \Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(2)}, x)$, so the XOR is 0. DPF security follows because both r and $r \oplus \beta$ are uniform in $\mathcal{Y}$; hence each share key k_b is distributed as a napPRF key programmed with a uniform

Π_{napPRF}

- $\underline{\mathsf{KeyGen}(1^\lambda)}$. Sample $\mathsf{msk} \xleftarrow{\$} \{0,1\}^{mR}$. Output msk.
- $\underline{\mathsf{Prog}(\mathsf{msk}, \alpha, \beta)}$. Parse $\beta = \beta_1 \| \cdots \| \beta_m$ where each $\beta_i \in \{0,1\}$. Parse $\mathsf{msk} = \mathsf{msk}_1 \| \cdots \| \mathsf{msk}_m$ where each $\mathsf{msk}_i \in \{0,1\}^R$. For each $i \in [m]$:
  1. Compute $(k_1^{(i)}, k_2^{(i)}) := \Pi_{\mathsf{DPF}}.\mathsf{Gen}(1^\lambda, \alpha, 1; \mathsf{msk}_i)$, i.e., generate DPF shares of the point function $P_{\alpha,1}$ using random tape fixed as msk_i.
  2. Let $b_i := \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1^{(i)}, \alpha)$. If $b_i = \beta_i$, set $k^{(i)} := k_1^{(i)}$; else, set $k^{(i)} := k_2^{(i)}$.

  Output $\mathsf{sk} := k^{(1)} \| \cdots \| k^{(m)}$.
- $\underline{\mathsf{Eval}(\mathsf{sk}, x)}$. Parse $\mathsf{sk} = k^{(1)} \| \cdots \| k^{(m)}$. Compute $y_i := \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k^{(i)}, x)$ for each $i \in [m]$. Output $y := y_1 \| \cdots \| y_m$.

Figure 2: Constructing Π_{napPRF} from $\Pi_{\mathsf{DPF}} = (\mathsf{Gen}, \mathsf{Eval})$ as a black box.

value, which is simulatable by napPRF's index hiding and thus hides (α, β). We defer the full proof to Appendix B.2.

We conclude this section with a corollary of Theorem 6 and Lemma 2, which we formalize in Theorem 5. By Theorem 6, our construction turns any correct and index-hiding napPRF into a two-party DPF that is correct and secure; by Lemma 2, this DPF moreover satisfies *individual pseudorandomness*.

Unwinding the construction, we obtain the following statement about the underlying napPRF. Fix any index $\alpha \in \mathcal{X}$ and let $\beta \xleftarrow{\$} \mathcal{Y}$ be uniform. Sample $\mathsf{msk} \leftarrow \Pi_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda)$ and set $\mathsf{sk} := \Pi_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$. Then the induced oracle $\mathsf{Eval}(\mathsf{sk}, \cdot) : \mathcal{X} \to \mathcal{Y}$ is computationally indistinguishable from a uniformly random function. That is, when the programmed value is uniform, $\Pi_{\mathsf{napPRF}}.\mathsf{Eval}$ behaves like a PRF for any fixed α.

5.3 DPF ⇒ napPRF

Let $\Pi_{\mathsf{DPF}} = (\mathsf{Gen}, \mathsf{Eval})$ be a DPF scheme over domain $\mathcal{X} = \{0,1\}^{n(\lambda)}$ and range $\{0,1\}$. Assume that $\Pi_{\mathsf{DPF}}.\mathsf{Gen}$ uses at most $R(\lambda)$ random bits. We construct a napPRF scheme $\Pi_{\mathsf{napPRF}} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ (over domain $\mathcal{X}$ and range $\mathcal{Y} = \{0,1\}^m$) by using Π_{DPF} as a black box, as depicted in Figure 2. We have:

Theorem 7 (DPF ⇒ napPRF). *If $\Pi_{\mathsf{DPF}} = (\mathsf{Gen}, \mathsf{Eval})$ is a* correct *and* secure *two-party DPF over domain $\mathcal{X}$ and range $\{0,1\}$, then the construction in Figure 2 yields a napPRF $\Pi_{\mathsf{napPRF}} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ over domain $\mathcal{X}$ and range $\mathcal{Y}$ that is* correct *and* index-hiding *(defined in Section 4).*

We note that the intuition behind Theorem 7 already appears in Section 2.4; we defer the full proof to Appendix B.3.

Instantiating with OWF-based DPF. Plugging the standard OWF-based DPF constructions [GI14, BGI15] (cf. Lemma 1) into Theorem 7 yields an OWF-based napPRF whose programmed key has size $nm\lambda + 2nm + m\lambda$ bits. In this instantiation, the programmed key is generated entirely as a function of (α, β) (i.e., there is no offline part), and it also fails suffix invariance (cf. Definition 7), even with non-black-box access to the constructions of [GI14, BGI15] (absent substantial modifications). Intuitively, in these DPFs the so-called *correction words* are α-dependent, so changing α changes the resulting key structure.

5.4 Programmable DPF ⇒ apPRF

Instantiating the transformation in Section 5.3 with a correct and secure *programmable* DPF [BGIK22b] yields a napPRF that is not only correct and index hiding, but also *cross-α consistent*. Consequently, by Section 4.1, this napPRF further implies a correct and index-hiding apPRF via a black-box construction.

Programmable DPFs. At a high level, a programmable DPF is a DPF scheme whose key-generation algorithm Gen can be decomposed into two stages, Gen_1 and Gen_2. In Gen_1, the first key k_1 is sampled without knowledge of (α, β); then Gen_2 produces the second key k_2 as a (randomized) function of k_1 and (α, β). In particular, once the randomness of Gen is fixed, the induced function $x \mapsto \mathsf{DPF.Eval}(k_1, x)$ is fully determined for all $x \in \mathcal{X}$.

Cross-α consistency of the induced napPRF. Recall that cross-α consistency requires that for all (distinct) $\alpha_1, \alpha_2 \in \mathcal{X}$ and all $x \in \mathcal{X} \setminus \{\alpha_1, \alpha_2\}$,

$$\mathsf{Eval}\big(\mathsf{Prog}(\mathsf{msk}, \alpha_1, 0), x\big) = \mathsf{Eval}\big(\mathsf{Prog}(\mathsf{msk}, \alpha_2, 0), x\big)$$

Consider our napPRF in Section 5.3 and fix any coordinate $i \in [m]$. For each α, Prog computes $(k_1^{(i)}, k_2^{(i)}) \leftarrow \Pi_{\mathsf{DPF}}.\mathsf{Gen}(1^\lambda, \alpha, 1; \mathsf{msk}_i)$ (using fixed randomness msk_i), and then outputs either $k_1^{(i)}$ or $k_2^{(i)}$. For any $x \neq \alpha$, DPF correctness implies $\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1^{(i)}, x) = \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2^{(i)}, x)$, and hence the i-th output bit of $\mathsf{Eval}(\mathsf{sk}, x)$ equals $\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1^{(i)}, x)$, regardless of if Prog chose $k_1^{(i)}$ or $k_2^{(i)}$.

Now take two indices α_1, α_2 and a point $x \notin \{\alpha_1, \alpha_2\}$. In the programmable DPF, the first key $k_1^{(i)}$ produced with randomness msk_i is independent of α, so the same $k_1^{(i)}$ is used in both executions with α_1 and with α_2. Therefore, the i-th output bit at x is identical in both cases, and concatenating over $i \in [m]$ gives cross-α consistency for the induced napPRF.

Building apPRF from [BGIK22b]. Boyle *et al.* [BGIK22b] construct programmable DPFs for polynomial-size domains from one-way functions. Combining their OWF-based programmable DPF with the implications above, we obtain (for polynomial-size domains) a correct and index-hiding apPRF from one-way functions via a black-box reduction; to our knowledge, such an OWF-based construction was not previously available even in this restricted regime.[3]

We emphasize that, since the domain is only polynomial-size, the resulting apPRF guarantee pertains to *PRF security under uniform programming*. That is, it implies pseudorandomness of the programmed evaluation oracle $\mathsf{Eval}(\mathsf{sk}, \cdot)$ when the programmed value is uniform, but does not by itself imply that the master evaluation $\mathsf{MEval}(\mathsf{msk}, \cdot)$ is a standard PRF (cf. Section 3.4). It remains to identify additional conditions or instantiations under which the induced master evaluation MEval also enjoys standard PRF security. It is plausible that the specific OWF-based programmable DPF construction of [BGIK22b] already yields such a guarantee; we leave a careful investigation to future work.

6 Toward Short-key Offline-online napPRFs

In this section, we construct *short-key offline-online* napPRFs (cf. Definition 5) from one-way functions. Recall that in this syntax the programmed key splits as $\mathsf{sk} = (\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}})$, where $(\mathsf{msk}, \mathsf{sk}_{\mathsf{off}}) \leftarrow \mathsf{KeyGen}(1^\lambda)$

[3] We note that [GZSP25] gives an attempt, but it incurs a $1/\mathsf{poly}(\lambda)$ security loss.

is independent of (α, β) and $\mathsf{sk}_{\mathsf{on}} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$ is the only *online* material that depends on the programmed point. Our goal is to make $\mathsf{sk}_{\mathsf{on}}$ short — i.e., $|\mathsf{sk}_{\mathsf{on}}|$ is linear in $n + m$ (up to $\mathsf{poly}(\lambda)$ factors) — while keeping the total key size $|\mathsf{sk}| \leq \mathsf{poly}(n, m, \lambda)$.

We give two routes: a generic garbling-based compilation that yields short online keys (assuming a standard napPRF whose Prog is a small circuit), and a main black-box OWF-based construction that also satisfies suffix invariance.

6.1 Garbling-based Short-key Compiler

We begin with a simple approach for building a short-key offline-online napPRF from a standard napPRF whose Prog has small Boolean circuit complexity. We use Yao garbling to *compile* the programming algorithm Prog into offline key material $\mathsf{sk}_{\mathsf{off}}$. That is, we garble a circuit for Prog and include the resulting GC in $\mathsf{sk}_{\mathsf{off}}$. The online key $\mathsf{sk}_{\mathsf{on}}$ then consists only of the input labels encoding α and β. Since this requires $(n + m)$ labels, we obtain $|\mathsf{sk}_{\mathsf{on}}| = (n + m)\lambda$.

This compiler is conceptually simple; we defer the formalism to Appendix C.

Instantiation via our DPF-based napPRF. The napPRF from Section 5.3 (instantiated with OWF-based DPFs [GI14, BGI15]) indeed admits a Prog procedure with small circuit complexity: Prog is dominated by $\mathcal{O}(nm)$ calls to a length-doubling PRG. Thus, via non-black-box use of cryptography (i.e., garbling $\mathcal{O}(nm)$ PRG expansions), the above compiler yields a short-key OWF-based offline-online napPRF with total key size $|\mathsf{sk}| = \mathcal{O}(nm \cdot \mathsf{poly}(\lambda))$.

This instantiation supports super-poly domains, but is non-black-box and does not provide the suffix-invariance needed for our SGC application.

6.2 Suffix-invariant napPRFs via Recursively Stacked Offline Keys

In this section, we construct suffix-invariant short-key offline-online napPRF schemes for poly-size domains from one-way functions, using all underlying primitives in a black-box manner. An intuitive overview appears in Section 2.5. Specifically, we define a sequence $\Pi^{(0)}_{\mathsf{napPRF}}, \ldots, \Pi^{(n)}_{\mathsf{napPRF}}$ where (i) $\Pi^{(0)}_{\mathsf{napPRF}}$ is a correct, index-hiding offline-online napPRF over domain $\{0,1\}^0 = \{\epsilon\}$ and range $\{0,1\}^m$, and (ii) for each $k \in \{1, \ldots, n\}$, the k-th level scheme $\Pi^{(k)}_{\mathsf{napPRF}}$ treats $\Pi^{(k-1)}_{\mathsf{napPRF}}$ purely as a black box and forms a correct, index-hiding offline-online napPRF over domain $\{0,1\}^k$ and range $\{0,1\}^m$. Looking ahead, the final scheme $\Pi^{(n)}_{\mathsf{napPRF}}$ has $|\mathsf{sk}_{\mathsf{off}}| = 8nm\lambda + 4n^2\lambda - 2n\lambda + m$ and $|\mathsf{sk}_{\mathsf{on}}| = (n + m)\lambda$.

PRG (used by all levels). Let $G : \{0,1\}^\lambda \to \{0,1\}^*$ be a PRG (from OWFs).

Encryption schemes (used by all levels). Let $F : \{0,1\}^\lambda \times \{0,1\}^* \to \{0,1\}^{2\lambda}$ be a PRF (from OWFs). All ciphertexts below have length λ or 2λ, and every garbled-table entry is associated with a public *unique* tweak $\tau \in \{0,1\}^*$.

- **Single-key encryption.** For messages $M \in \{0,1\}^\lambda$ define

$$\mathsf{Enc}^{(1)}_K(\tau, M) := M \oplus F_K(\tau).\mathsf{first} \qquad \mathsf{Dec}^{(1)}_K(\tau, C) := C \oplus F_K(\tau).\mathsf{first}$$

  where $F_K(\tau).\mathsf{first}$ denotes the first λ bits of $F_K(\tau) \in \{0,1\}^{2\lambda}$. By PRF security, if K is uniform and hidden, then $\mathsf{Enc}^{(1)}_K(\tau, M)$ is computationally indistinguishable from uniform over $\{0,1\}^\lambda$ (even given τ and M).

$\Pi^{(0)}_{\mathsf{napPRF}}$

- $\underline{\mathsf{KeyGen}(1^\lambda)}$. Sample a master seed $s_{\mathsf{master}} \xleftarrow{\$} \{0,1\}^\lambda$. Use $G(s_{\mathsf{master}})$ to sample

$$\{\mathcal{L}^{(0)}_{\beta,i}, \mathcal{L}^{(1)}_{\beta,i} \in \{0,1\}^\lambda\}_{i\in[m]}$$

  Point-and-permute constraint: overwrite the last bits so that

$$\forall i \in [m], \mathsf{Lsb}(\mathcal{L}^{(1)}_{\beta,i}) := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\beta,i}) \oplus 1$$

  (All other bits remain as sampled, so $\mathcal{L}^{(1)}$ is not derivable from $\mathcal{L}^{(0)}$.)
  Output $\mathsf{msk} := \{\mathcal{L}^{(0)}_{\beta,i}, \mathcal{L}^{(1)}_{\beta,i}\}_{i\in[m]}$ and $\mathsf{sk}_{\mathsf{off}} := \{\, b_i := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\beta,i}) \,\}_{i\in[m]}$.
- $\underline{\mathsf{Prog}(\mathsf{msk}, \epsilon, \beta \in \{0,1\}^m)}$. Parse $\beta = \beta_1 \| \cdots \| \beta_m$. Output $\mathsf{sk}_{\mathsf{on}} := \{\mathcal{L}^{(\beta_i)}_{\beta,i}\}_{i\in[m]}$.
- $\underline{\mathsf{Eval}(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}}, \epsilon)}$. Parse $\mathsf{sk}_{\mathsf{off}} = \{b_i\}_{i\in[m]}$ and $\mathsf{sk}_{\mathsf{on}} = \{\mathcal{L}_{\beta,i}\}_{i\in[m]}$. For each $i \in [m]$, compute $\beta_i := \mathsf{Lsb}(\mathcal{L}_{\beta,i}) \oplus b_i$. Output $\beta = \beta_1 \| \cdots \| \beta_m$.

Figure 3: $\Pi^{(0)}_{\mathsf{napPRF}}$ for domain $\{0,1\}^0 = \{\epsilon\}$ and range $\{0,1\}^m$.

- **Two-key encryption.** For messages $M \in \{0,1\}^{2\lambda}$ define

$$\mathsf{Enc}^{(2)}_{K_1,K_2}(\tau, M) \;:=\; M \oplus F_{K_1}(\tau\|0) \oplus F_{K_2}(\tau\|1)$$

$$\mathsf{Dec}^{(2)}_{K_1,K_2}(\tau, C) \;:=\; C \oplus F_{K_1}(\tau\|0) \oplus F_{K_2}(\tau\|1)$$

  By PRF security, if at least one of K_1, K_2 is uniform and hidden, then $\mathsf{Enc}^{(2)}_{K_1,K_2}(\tau, M)$ is computationally indistinguishable from uniform over $\{0,1\}^{2\lambda}$ (even given τ, M, and the other key).

Least significant bit. For $x \in \{0,1\}^\lambda$, $\mathsf{Lsb}(x)$ denotes its least significant bit.

Base case $\Pi^{(0)}_{\mathsf{napPRF}}$. For a singleton domain, an (offline-online) napPRF is immediate (one may take $\mathsf{sk} = \beta$). We nevertheless give a slightly more verbose formulation — decoding garbled labels — to match the interface of our recursive construction. The scheme $\Pi^{(0)}_{\mathsf{napPRF}}$ is depicted in Figure 3, and satisfies:

Theorem 8 (Base case). *Let G be a PRG. $\Pi^{(0)}_{\mathsf{napPRF}}$ (cf. Figure 3) is a correct and index-hiding offline-online napPRF over domain $\{0,1\}^0$ and range $\{0,1\}^m$.*

The proof is straightforward; we defer the full details to Appendix B.4.

Recursive definitions. Fix $k \geq 1$. We define $\Pi^{(k)}_{\mathsf{napPRF}} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ over domain $\mathcal{X} = \{0,1\}^k$ and range $\mathcal{Y} = \{0,1\}^m$. The construction of $\Pi^{(k)}_{\mathsf{napPRF}}$ makes *black-box* use of $\Pi^{(k-1)}_{\mathsf{napPRF}}$.

We write $\tau^{(k)}_{\mathsf{seed},r}$, $\tau^{(k)}_{\beta,i,r,c}$, and $\tau^{(k)}_{\alpha,i,r,c}$ for public, *unique* tweaks for the corresponding (garbled) table entries. For notational convenience, we write

$$(\mathsf{msk}, \mathsf{sk}_{\mathsf{off}}) := \Pi_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda; s)$$

$$\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda)$$

1. Sample a master seed $s_{\mathsf{master}} \xleftarrow{\$} \{0,1\}^\lambda$. Use $G(s_{\mathsf{master}})$ to sample

$$s_{\mathsf{left}}, s_{\mathsf{right}} \in \{0,1\}^\lambda \quad \{\mathcal{L}^{(0)}_{\alpha,i}, \mathcal{L}^{(1)}_{\alpha,i} \in \{0,1\}^\lambda\}_{i\in[k]} \quad \{\mathcal{L}^{(0)}_{\beta,i}, \mathcal{L}^{(1)}_{\beta,i} \in \{0,1\}^\lambda\}_{i\in[m]}$$

$$\{\mathcal{J}_{\alpha,i} \in \{0,1\}^\lambda\}_{i\in[k-1]} \quad \{\mathcal{J}_{\beta,i} \in \{0,1\}^\lambda\}_{i\in[m]}$$

   Point-and-permute constraint: overwrite the last bits so that

$$\forall i \in [k], \mathsf{Lsb}(\mathcal{L}^{(1)}_{\alpha,i}) := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\alpha,i}) \oplus 1 \qquad \forall i \in [m], \mathsf{Lsb}(\mathcal{L}^{(1)}_{\beta,i}) := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\beta,i}) \oplus 1$$

   (All other bits remain as sampled, so $\mathcal{L}^{(1)}$ is not derivable from $\mathcal{L}^{(0)}$.)

2. **Generate two child instances.** For $j \in \{\mathsf{left}, \mathsf{right}\}$, compute $(\mathsf{msk}_j, \mathsf{sk}_{j,\mathsf{off}}) := \Pi^{(k-1)}_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda; s_j)$. By construction, msk_j includes the child wire labels $\{\mathcal{L}^{(0)}_{\alpha,j,i}, \mathcal{L}^{(1)}_{\alpha,j,i}\}_{i\in[k-1]}$ and $\{\mathcal{L}^{(0)}_{\beta,j,i}, \mathcal{L}^{(1)}_{\beta,j,i}\}_{i\in[m]}$, whereas $\mathsf{sk}_{j,\mathsf{off}}$ is the level-$(k-1)$ offline key output by the recursive call (in a fixed canonical serialization).

3. Output $\mathsf{msk} := \{\mathcal{L}^{(0)}_{\alpha,i}, \mathcal{L}^{(1)}_{\alpha,i}\}_{i\in[k]}, \{\mathcal{L}^{(0)}_{\beta,i}, \mathcal{L}^{(1)}_{\beta,i}\}_{i\in[m]}$ and $\mathsf{sk}_{\mathsf{off}}$ that contains:

   - **Stacked child offline keys:** $\mathsf{sk}_{\mathsf{stack}} := \mathsf{sk}_{\mathsf{left,off}} \oplus \mathsf{sk}_{\mathsf{right,off}}$.
   - **Seed table $\mathcal{T}_{\mathsf{seed}}$.** Let $r := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\alpha,1})$. Define

$$\mathcal{T}_{\mathsf{seed}}[r] := \mathsf{Enc}^{(1)}_{\mathcal{L}^{(0)}_{\alpha,1}}(\tau^{(k)}_{\mathsf{seed},r},\ s_{\mathsf{right}}) \quad \mathcal{T}_{\mathsf{seed}}[r\oplus 1] := \mathsf{Enc}^{(1)}_{\mathcal{L}^{(1)}_{\alpha,1}}(\tau^{(k)}_{\mathsf{seed},r\oplus 1},\ s_{\mathsf{left}})$$

   - **α-translation tables $\{\mathcal{T}_{\alpha,i}\}_{i\in[k-1]}$.** Let $r := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\alpha,1})$.
     For each $i \in [k-1]$: Let $j := i+1, c := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\alpha,j})$ and define

$$\begin{aligned}
\mathcal{T}_{\alpha,i}[r][c] &:= \mathsf{Enc}^{(2)}_{\mathcal{L}^{(0)}_{\alpha,1},\mathcal{L}^{(0)}_{\alpha,j}}(\tau^{(k)}_{\alpha,i,r,c},\ \mathcal{L}^{(0)}_{\alpha,\mathsf{left},i}\|\mathcal{J}_{\alpha,i})\\
\mathcal{T}_{\alpha,i}[r][c\oplus 1] &:= \mathsf{Enc}^{(2)}_{\mathcal{L}^{(0)}_{\alpha,1},\mathcal{L}^{(1)}_{\alpha,j}}(\tau^{(k)}_{\alpha,i,r,c\oplus 1},\ \mathcal{L}^{(1)}_{\alpha,\mathsf{left},i}\|\mathcal{J}_{\alpha,i})\\
\mathcal{T}_{\alpha,i}[r\oplus 1][c] &:= \mathsf{Enc}^{(2)}_{\mathcal{L}^{(1)}_{\alpha,1},\mathcal{L}^{(0)}_{\alpha,j}}(\tau^{(k)}_{\alpha,i,r\oplus 1,c},\ \mathcal{J}_{\alpha,i}\|\mathcal{L}^{(0)}_{\alpha,\mathsf{right},i})\\
\mathcal{T}_{\alpha,i}[r\oplus 1][c\oplus 1] &:= \mathsf{Enc}^{(2)}_{\mathcal{L}^{(1)}_{\alpha,1},\mathcal{L}^{(1)}_{\alpha,j}}(\tau^{(k)}_{\alpha,i,r\oplus 1,c\oplus 1},\ \mathcal{J}_{\alpha,i}\|\mathcal{L}^{(1)}_{\alpha,\mathsf{right},i})
\end{aligned}$$

   - **β-translation tables $\{\mathcal{T}_{\beta,i}\}_{i\in[m]}$.** Let $r := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\alpha,1})$.
     For each $i \in [m]$: Let $c := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\beta,i})$ and define

$$\begin{aligned}
\mathcal{T}_{\beta,i}[r][c] &:= \mathsf{Enc}^{(2)}_{\mathcal{L}^{(0)}_{\alpha,1},\mathcal{L}^{(0)}_{\beta,i}}(\tau^{(k)}_{\beta,i,r,c},\ \mathcal{L}^{(0)}_{\beta,\mathsf{left},i}\|\mathcal{J}_{\beta,i})\\
\mathcal{T}_{\beta,i}[r][c\oplus 1] &:= \mathsf{Enc}^{(2)}_{\mathcal{L}^{(0)}_{\alpha,1},\mathcal{L}^{(1)}_{\beta,i}}(\tau^{(k)}_{\beta,i,r,c\oplus 1},\ \mathcal{L}^{(1)}_{\beta,\mathsf{left},i}\|\mathcal{J}_{\beta,i})\\
\mathcal{T}_{\beta,i}[r\oplus 1][c] &:= \mathsf{Enc}^{(2)}_{\mathcal{L}^{(1)}_{\alpha,1},\mathcal{L}^{(0)}_{\beta,i}}(\tau^{(k)}_{\beta,i,r\oplus 1,c},\ \mathcal{J}_{\beta,i}\|\mathcal{L}^{(0)}_{\beta,\mathsf{right},i})\\
\mathcal{T}_{\beta,i}[r\oplus 1][c\oplus 1] &:= \mathsf{Enc}^{(2)}_{\mathcal{L}^{(1)}_{\alpha,1},\mathcal{L}^{(1)}_{\beta,i}}(\tau^{(k)}_{\beta,i,r\oplus 1,c\oplus 1},\ \mathcal{J}_{\beta,i}\|\mathcal{L}^{(1)}_{\beta,\mathsf{right},i})
\end{aligned}$$

Note that entries in each table are stored in the canonical order.

Figure 4: $\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{KeyGen}$ **for domain** $\{0,1\}^k$ **and range** $\{0,1\}^m$**.**

$$\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha \in \{0,1\}^k, \beta \in \{0,1\}^m)$$

Parse $\alpha = \alpha_1 \| \cdots \| \alpha_k$, $\beta = \beta_1 \| \cdots \| \beta_m$. Output $\mathsf{sk_{on}} := \{\mathcal{L}^{(\alpha_i)}_{\alpha,i}\}_{i\in[k]} \parallel \{\mathcal{L}^{(\beta_i)}_{\beta,i}\}_{i\in[m]}$.

$$\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk_{off}}, \mathsf{sk_{on}}, x \in \{0,1\}^k)$$

Parse $x = x_1 \| \cdots \| x_k$, $\mathsf{sk_{off}} = (\mathsf{sk_{stack}}, \mathcal{T}_{\mathsf{seed}}, \{\mathcal{T}_{\alpha,i}\}_{i\in[k-1]}, \{\mathcal{T}_{\beta,i}\}_{i\in[m]})$ and $\mathsf{sk_{on}} = (\{\mathcal{L}_{\alpha,i}\}_{i\in[k]}, \{\mathcal{L}_{\beta,i}\}_{i\in[m]})$.

1. **Recover seed.** Let $r := \mathsf{Lsb}(\mathcal{L}_{\alpha,1})$. Compute $s := \mathsf{Dec}^{(1)}_{\mathcal{L}_{\alpha,1}}(\tau^{(k)}_{\mathsf{seed},r},\ \mathcal{T}_{\mathsf{seed}}[r])$.
2. **Re-derive inactive offline key.** Compute $(\widehat{\mathsf{msk}}, \widehat{\mathsf{sk_{off}}}) := \Pi^{(k-1)}_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda; s)$.
3. **Decrypt translation for α and β.** Let $r := \mathsf{Lsb}(\mathcal{L}_{\alpha,1})$.
   For each $i \in [k-1]$: Let $c := \mathsf{Lsb}(\mathcal{L}_{\alpha,i+1})$ and compute
   $$\mathcal{L}_{\alpha,0,i} \| \mathcal{L}_{\alpha,1,i} := \mathsf{Dec}^{(2)}_{\mathcal{L}_{\alpha,1},\mathcal{L}_{\alpha,i+1}}(\tau^{(k)}_{\alpha,i,r,c},\ \mathcal{T}_{\alpha,i}[r][c])$$
   For each $i \in [m]$: Let $c := \mathsf{Lsb}(\mathcal{L}_{\beta,i})$ and compute
   $$\mathcal{L}_{\beta,0,i} \| \mathcal{L}_{\beta,1,i} := \mathsf{Dec}^{(2)}_{\mathcal{L}_{\alpha,1},\mathcal{L}_{\beta,i}}(\tau^{(k)}_{\beta,i,r,c},\ \mathcal{T}_{\beta,i}[r][c])$$
4. **Derive next-level offline and online keys then recurse.** Define
   $$\mathsf{sk}^{(\mathsf{next})}_{\mathsf{off}} := \mathsf{sk_{stack}} \oplus \widehat{\mathsf{sk_{off}}} \quad \text{and} \quad \mathsf{sk}^{(\mathsf{next})}_{\mathsf{on}} := \{\mathcal{L}_{\alpha,x_1,i}\}_{i\in[k-1]} \parallel \{\mathcal{L}_{\beta,x_1,i}\}_{i\in[m]}$$
   Output $\Pi^{(k-1)}_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(\mathsf{next})}_{\mathsf{off}}, \mathsf{sk}^{(\mathsf{next})}_{\mathsf{on}}, x_2 \| \cdots \| x_k)$.

Figure 5: $\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{Prog}$ **and** $\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{Eval}$ **for domain** $\{0,1\}^k$ **and range** $\{0,1\}^m$**.**

to denote running $\Pi_{\mathsf{napPRF}}.\mathsf{KeyGen}$ with randomness set as $s \in \{0,1\}^\lambda$. In particular, we will ensure that each KeyGen uses only a λ-bit random tape. The scheme $\Pi^{(k)}_{\mathsf{napPRF}}$ is depicted in Figures 4 and 5, and satisfies:

Theorem 9 (Main). *Let G be a PRG and let F be a PRF. For every $k \geq 1$, the scheme $\Pi^{(k)}_{\mathsf{napPRF}}$ (cf. Figures 4 and 5) is a correct and index-hiding offline-online napPRF over domain $\{0,1\}^k$ and range $\{0,1\}^m$.*

We note that the intuition behind Theorem 9 already appears in Section 2.5; we defer the full proof to Appendix B.5.

Suffix invariance. We remark that this construction is suffix-invariant (cf. Definition 7); see Section 2.5 for an intuitive justification.

Efficiency. We briefly discuss the efficiency of our final scheme $\Pi^{(n)}_{\mathsf{napPRF}}$.

At each recursive step, $\mathsf{sk_{off}}$ grows only by the garbled tables introduced at that level: the two child offline keys are stacked (via XOR), so lower-level offline material is not duplicated across the recursion. Therefore, $|\mathsf{sk_{off}}| = 8nm\lambda + 4n^2\lambda - 2n\lambda + m$. Moreover, $\mathsf{sk_{on}}$ consists of n λ-bit α-related labels together with m λ-bit β-related label, giving $|\mathsf{sk_{on}}| = (n+m)\lambda$.

We next analyze the computational complexity. Our metric is the number of calls to $\mathsf{Enc}^{(1/2)}$ and $\mathsf{Dec}^{(1/2)}$, which dominates the running time. Specifically:

- $\mathsf{KeyGen}^{(n)}$ makes $(2^n - 1)(4m + 6) - 4n$ calls;
- $\mathsf{Prog}^{(n)}$ makes 0 calls; and
- $\mathsf{Eval}^{(n)}$ makes $(2^n - n - 1)(4m + 6) + n(m + 1) - \frac{3}{2}n(n - 1)$ calls.

The appearance of 2^n terms explains why the construction is intended for poly-size domains. Finally, we remark that $\Pi^{(n)}_{\mathsf{napPRF}}$ admits a more efficient $\mathsf{FullEval}$ by reusing the inactive-branch offline key $\widehat{\mathsf{sk}_{\mathsf{off}}}$ (and decrypted components):

- $\mathsf{FullEval}^{(n)}$ makes $2^{n+1}nm - 3(2^n - 1)m + 2^n(3n - 8) + 3n + 8$ calls.

7 Improving SGC via Short-key Offline-online napPRFs

In this section, we show how our short-key and suffix-invariant offline-online napPRF from Section 6.2 can be used — both in an off-the-shelf and a tailored manner — to improve the state-of-the-art stacked garbling (SGC) scheme $\mathsf{LogStack}$ [HK21], which optimizes (oblivious) branching in Yao garbled circuits.

Concretely, consider oblivious branching among $B = 2^n$ Boolean subcircuits $\mathcal{C}_0, \ldots, \mathcal{C}_{B-1}$, where each $\mathcal{C}_j : \{0,1\}^I \to \{0,1\}$ has size C.[4] Given selector-wire labels for $\alpha \in \{0,1\}^n$ and input-wire labels for $x \in \{0,1\}^I$, the evaluator must obtain the output label encoding $\mathcal{C}_\alpha(x)$. While $\mathsf{LogStack}$ achieves garbled size $\mathcal{O}(BI\lambda + C\lambda)$ with $\mathcal{O}(B \log B)$ computation for both parties, we reduce the $\mathcal{O}(BI\lambda)$ term without increasing the $\mathcal{O}(B \log B)$ computation.

A concise overview of stacked garbling appears in Section 2.2, and an overview of our napPRF-based improvement appears in Section 2.3.

7.1 Off-the-shelf Use of Our Offline-online napPRF

The additive $\mathcal{O}(BI\lambda)$ term in $\mathsf{LogStack}$'s garbled-circuit size arises from a garbling gadget called the *demultiplexer*. At a high level, the demultiplexer enables E, given wire labels for the selector bits $\alpha_1, \ldots, \alpha_n$ and the inputs $x_1, \ldots, x_I$, to obtain *candidate* input-wire labels for every branch $t \in \{0,1\}^n$. Crucially, on the *active* branch $t = \alpha := \alpha_1 \| \cdots \| \alpha_n$, the candidate labels must encode the underlying input values $x_1, \ldots, x_I$, while on every inactive branch $t \neq \alpha$ the candidate labels must be indistinguishable from the active ones. Moreover (for technical reasons in SGC), the candidate label vector on each inactive branch must be a fixed dummy value that is independent of x.

This is exactly the functionality provided by an offline-online napPRF.[5] Let $\beta \in \{0,1\}^{I\lambda}$ denote the concatenation of the I input-wire labels for the active branch (on x). In principle, if E were given a programmed key $\mathsf{sk} = (\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}})$ with $\mathsf{sk}_{\mathsf{on}} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$, then for every $t \in \{0,1\}^n$ it could compute a candidate label vector $\beta_t := \mathsf{Eval}(\mathsf{sk}, t)$. On-point correctness gives $\beta_\alpha = \beta$, while off-point correctness guarantees that for all $t \neq \alpha$ we have $\beta_t = \beta_\perp$ for a fixed dummy value $\beta_\perp$ that is independent of β (and hence of x) and distributed like valid labels. Since $\mathsf{sk}_{\mathsf{off}}$ depends only on KeyGen, G can simply include $\mathsf{sk}_{\mathsf{off}}$ as part of the garbled material. The remaining issue is how to deliver the online key $\mathsf{sk}_{\mathsf{on}}$ to E (via garbling).

[4] SGC supports multiple outputs; we focus on a single output.

[5] A standard napPRF can also offer similar functionality; see Section 2.3.

We exploit the internal structure of SGC (including LogStack): the active-branch input-wire labels are obtained by evaluating an n-level GGM tree along the selector path α and then expanding the resulting leaf seed via a PRG. Consequently, G can garble a circuit that (i) derives the leaf seed on the path $\alpha_1, \ldots, \alpha_n$, (ii) expands it into the I input-wire label pairs for that branch, and (iii) for each $i \in [I]$, selects the label consistent with the evaluator's input label x_i. Equivalently, from the selector-wire labels (encoding α) and the input-wire labels (encoding x), the circuit computes — in GC label form — the napPRF programmed value β for the active branch.

Finally, recall that in our construction $\Pi^{(n)}_{\mathsf{napPRF}}$ (cf. Section 6.2), the procedure Prog outputs precisely the GC labels that constitute $\mathsf{sk}_{\mathsf{on}}$ (i.e., the labels corresponding to the programmed index and the programmed value). Thus, once the circuit above computes the appropriate β, we can use standard garbled truth tables to let E get the corresponding $\mathsf{sk}_{\mathsf{on}}$ from its labels for $\alpha_1, \ldots, \alpha_n$ and β.

Garbled size. The dominant garbling components of this approach are:

1. Computing the GGM path seed costs $\mathcal{O}(n \cdot \mathsf{poly}(\lambda))$.
2. Expanding the leaf seed via PRG to get $\mathcal{O}(I)$ input labels costs $\mathcal{O}(I \cdot \mathsf{poly}(\lambda))$.
3. A garbled truth table that delivers the online key $\mathsf{sk}_{\mathsf{on}}$ given GC labels for (α, β). Here we use GC labels to encode (the bits of) $\beta \in \{0,1\}^{I\lambda}$, so this contributes $\mathcal{O}((n + I\lambda)\lambda)$.
4. The offline key $\mathsf{sk}_{\mathsf{off}}$. In our instantiation (cf. Section 6.2) with range length $m = I\lambda$, we have $|\mathsf{sk}_{\mathsf{off}}| = \mathcal{O}(nI\lambda^2 + n^2\lambda)$.

Overall, this yields a demultiplexer of size $\mathcal{O}(nI \cdot \mathsf{poly}(\lambda) + n^2\lambda)$, improving over the original $\mathcal{O}(BI\lambda)$-size construction. With this new demultiplexer in place, the overall garbled-circuit size for branching in LogStack becomes $\mathcal{O}(nI \cdot \mathsf{poly}(\lambda) + B\lambda + C\lambda)$, where the additive $\mathcal{O}(B\lambda)$ term is inherited from LogStack's oblivious seed-delivery gadget (called SortingHat).

Correctness and security. We use the napPRF guarantees as follows:

- **On-point correctness.** Correct programming ensures that E obtains the intended input-wire labels on the active branch $t = \alpha$.
- **Fixed off-point behavior.** For every $t \neq \alpha$, napPRF evaluation yields a fixed dummy value $\beta_\perp$ that is independent of β (and hence of x).
- **Index hiding.** Since β consists of GC labels (and thus is pseudorandom-looking), index hiding implies that sk does not reveal which branch α is active.

Computation complexity. We discuss how the (offline-online) napPRF-based demultiplexer affects the computation complexity of both G and E. On the garbler side, generating the napPRF keys and the additional garbled material (needed to let E recover $\mathsf{sk}_{\mathsf{on}}$) incurs work proportional to B (dominated by KeyGen). On the evaluator side, by our FullEval procedure, evaluating the napPRF over the entire domain $\{0,1\}^n$ incurs work proportional to $B \log B$.

Maintaining LogStack's overall computation complexity, however, requires more than matching these asymptotic bounds. In a subsequent *garbage-collection* step, G must predict, for each branch $t \in \{0,1\}^n$, the dummy output that E obtains on branch t (when it is inactive) in order to combine B candidate outputs into a single garbled label. Without additional structure, this dummy output for a fixed t could vary with the full identity of the active index α, yielding up to B possibilities per branch and total work $\mathcal{O}(B^2)$ (indeed,

this is the bottleneck in the original SGC scheme [HK20a]). LogStack avoids this blow-up by requiring *suffix invariance*: for each inactive branch $t \neq \alpha$, the dummy output on t depends on α only through the first divergence position ℓ between t and α (i.e., $t_{1..\ell} = \alpha_{1..\ell}$ but $t_{\ell+1} \neq \alpha_{\ell+1}$), which limits a fixed t to only $\mathcal{O}(n)$ possible dummy outcomes and keeps the total work at $\mathcal{O}(B \log B)$.

Our construction $\Pi^{(n)}_{\mathsf{napPRF}}$ from Section 6.2 satisfies this requirement when instantiated as the demultiplexer, because it is suffix-invariant.

Summary. We conclude with the following theorem.

Theorem 10. *Let $\mathcal{C}_0, \ldots, \mathcal{C}_{B-1}$ be Boolean circuits, each of size C, sharing a common interface $\{0,1\}^I \to \{0,1\}$, and suppose $B = 2^n = \mathsf{poly}(\lambda)$. Then there exists a (non-black-box) OWF-based garbling scheme — achieving the same correctness and security guarantees as* LogStack *[HK21] — such that, given garbled inputs for (α, x), an evaluator can compute $y = \mathcal{C}_\alpha(x)$ with total garbled size*

$$\mathcal{O}\big(nI \cdot \mathsf{poly}(\lambda) + B\lambda + C\lambda\big)$$

Moreover, both the garbler and evaluator run in time $\mathcal{O}(B \log B)$.

7.2 Tailoring Our Offline-online napPRF for Better Integration

The napPRF-based demultiplexer from Section 7.1 incurs a $\mathsf{poly}(\lambda)$ overhead, and it must garble the GGM/PRG label-generation procedure inside a circuit. In this section, we tailor our napPRF construction from Section 6.2 to LogStack, eliminating the need to garble GGM/PRG for a more efficient SGC integration.

Intuition. The $\mathsf{poly}(\lambda)$ overhead in Section 7.1 is an artifact of composition. To produce the online key $\mathsf{sk}_{\mathsf{on}}$, the demultiplexer must first recover the active-branch input-label vector β, represented as GC wires (i.e., labels encoding labels). In SGC, these labels are generated by evaluating a GGM tree on the selector α and then expanding the resulting leaf seed via PRG. Thus, the off-the-shelf approach ends up garbling this entire GGM/PRG label-generation process.

We can avoid this by exploiting that the same "GGM tree $\to$ PRG expansion $\to$ wire labels" structure already appears in our napPRF construction from Section 6.2. In $\Pi^{(n)}_{\mathsf{napPRF}}$, the master seed s_{master} deterministically induces a full binary tree of child seeds (e.g., $s_{\mathsf{left}}, s_{\mathsf{right}}$ at the top level), and each leaf seed is expanded via a PRG to generate the corresponding Yao wire labels. We therefore *couple* the randomness so that the napPRF's internal GC labels at the leaf corresponding to α are *exactly* the active-branch labels that LogStack would generate.

A strawman attempt. Suppose we instantiate $\Pi^{(n)}_{\mathsf{napPRF}}$ using the same top-level master seed and PRG as LogStack's label-generation procedure. If we then program the napPRF at index α with a *bit* $b \in \{0,1\}$ (as opposed to the GC label for b), the PRG expansion at the leaf corresponding to α already determines the GC wire label for b (prior to the final LSB-based decoding). In particular, evaluating the napPRF at α yields the desired active-branch wire label "for free," without any additional garbled PRG computation.

However, this naïve coupling raises two issues:

1. In $\Pi^{(0)}_{\mathsf{napPRF}}$, $\mathsf{sk}_{\mathsf{off}}$ includes output-decoding information that would allow E to decode a wire label (e.g., the GC label encoding the input of active branch) into the underlying bit. We can avoid this by simply omitting the decoding information from $\mathsf{sk}_{\mathsf{off}}$ and treating napPRF outputs purely as GC labels.

2. A second, more subtle issue arises from the SortingHat gadget in LogStack. To achieve the claimed $\mathcal{O}(B \log B)$ computation cost, SortingHat provides E with a *candidate seed* for every node of the balanced binary tree whose B leaves correspond to the branches. A node receives its *real* seed (from GGM) if and only if it is a sibling root of the active leaf α; every other node receives a fixed dummy seed that is indistinguishable from a real one. At a high level, this ensures that for any fixed inactive branch, as α varies, unstacking yields only $\mathcal{O}(n)$ (i.e., $\mathcal{O}(\log B)$) possible dummy GCs rather than $\mathcal{O}(B)$.

   These evaluator-visible seeds rule out the naive idea of reusing LogStack's master seed as the top-level seed of $\Pi^{(n)}_{\mathsf{napPRF}}$. Indeed, $\Pi^{(n)}_{\mathsf{napPRF}}.\mathsf{Eval}$ explicitly reveals a seed for the sibling subtree opposite the programmed path at the first divergence. If this seed were derived from the same master seed underlying SortingHat, then E could simply compare the seed leaked by Eval to the candidate seeds output by SortingHat to identify the corresponding sibling subtree (and, by iterating this argument down the tree, recover α). This would violate oblivious branching and break the security of SGC.

Absorbing SortingHat into the napPRF seed delivery. To address the second issue, we "pad" the seed information exposed by $\Pi^{(n)}_{\mathsf{napPRF}}$ with the SortingHat view already present in LogStack. In other words, we tailor our offline-online napPRF constructions in Section 6.2 by absorbing the SortingHat.

Concretely, rather than leak a single sibling seed via $\mathcal{T}_{\mathsf{seed}}$, we drop $\mathcal{T}_{\mathsf{seed}}$ altogether and let msk contain the seed material needed to emulate SortingHat: a full binary tree of GGM-expanded seeds together with an independent full binary tree of i.i.d. dummy seeds (sampled via the PRG). In Prog, we output the online key $\mathsf{sk}_{\mathsf{on}}$ consisting of (i) the usual garbled labels encoding the programmed pair (α, β) and (ii) a *candidate-seed tree*, populated so that precisely the sibling roots on the path to α carry the *real* GGM seeds, while every other node carries the corresponding dummy seed. As a result, the seeds that Eval uses to unstack $\mathsf{sk}_{\mathsf{off}}$ are delivered exactly as in SortingHat. Hence, the tailored napPRF-based demultiplexer subsumes SortingHat's candidate-seed view, and the new SGC no longer requires invoking it as a separate gadget.

We include formal descriptions of this LogStack-tailored napPRF in Appendix D. Here we briefly discuss its Eval, correctness, security, and efficiency.

Eval. On input $(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}})$ and $x \in \{0,1\}^n$, the Eval parses the online key $\mathsf{sk}_{\mathsf{on}}$ into (i) the programmed labels for (α, β) and (ii) the candidate-seed tree. It then follows the Eval procedure of $\Pi^{(n)}_{\mathsf{napPRF}}$, except that whenever Eval would recover an (inactive) seed from $\mathcal{T}_{\mathsf{seed}}$, it instead *reads* the corresponding sibling seed from the candidate-seed tree. Using this seed, Eval locally runs the next-level KeyGen routine to reconstruct an offline key $\mathsf{sk}_{\mathsf{off}}$ for the sibling subtree, and recurses into the other subtree as dictated by the first bit of x. Note, Eval never needs to determine which candidate seeds are "real" versus "dummy."

Correctness and suffix invariance. If $x = \alpha$, then at every level the sibling seed read from the candidate-seed tree is a *real* GGM seed, and hence the locally unstacked $\mathsf{sk}_{\mathsf{off}}$ coincides with the offline key that KeyGen would generate along the programmed path. The recursion therefore reaches the leaf corresponding to α, which yields exactly the active-branch Yao wire-label vector encoding β.

If $x \neq \alpha$, let ℓ be the first position where x and α differ, and consider the recursive call made after this divergence. At level ℓ, Eval reads a *dummy* sibling seed from the candidate-seed tree, sampled independently of β; consequently, the re-derived child offline key $\mathsf{sk}'_{\mathsf{off}}$ is independent of β. Moreover, the child online key passed down is also independent of β: at this level Eval decrypts the translation ciphertexts to obtain the next-level labels, but on the diverged side these decryptions yield only fixed junk labels, and in our LogStack-tailored variant, *all* candidate seeds below this node are dummy. Thus, the recursive invocation receives a programmed key $(\mathsf{sk}'_{\mathsf{off}}, \mathsf{sk}'_{\mathsf{on}})$ whose distribution is independent of β.

Note that the same reasoning above also establishes suffix invariance.

Security. Security follows by essentially the same hybrid argument (cf. Appendix B.5) as for $\Pi^{(n)}_{\mathsf{napPRF}}$ in Section 6.2, and by the security of SortingHat [HK21].

Efficiency. This LogStack-tailored napPRF incurs two additional overheads:

1. The online key $\mathsf{sk}_{\mathsf{on}}$ additionally carries a full SortingHat-style tree of candidate seeds of total size $\mathcal{O}(B)$; this is acceptable in our application since SGC (including LogStack) already accounts for $\mathcal{O}(B)$ seed material.

2. In FullEval, each level invokes next-level KeyGen twice (once per candidate seed) rather than once, but this does not affect the asymptotic running time; the total computation complexity remains proportional to $B \log B$.

Summary. We conclude with the following theorem.

Theorem 11. *Let $\mathcal{C}_0, \ldots, \mathcal{C}_{B-1}$ be Boolean circuits, each of size C, sharing a common interface $\{0,1\}^I \to \{0,1\}$, and suppose $B = 2^n = \mathsf{poly}(\lambda)$. Then there exists a (black-box) OWF-based garbling scheme — achieving the same correctness and security guarantees as* LogStack *[HK21] — such that, given garbled inputs for (α, x), an evaluator can compute $y = \mathcal{C}_\alpha(x)$ with total garbled size*

$$\mathcal{O}\big(nI\lambda + B\lambda + C\lambda\big)$$

Moreover, both the garbler and evaluator run in time $\mathcal{O}(B \log B)$.

Formal improved LogStack. With our tailored offline-online napPRFs, the improved LogStack is largely plug-and-play: it simply replaces LogStack's demultiplexer with our napPRF-based one. For completeness, following [HK20a, HK21], we formalize the resulting SGC as a *garbling scheme* [BHR12] in Appendix E.

Acknowledgments

David Heath was supported by NSF grant CNS-#2246353.

References

[ARS24] Damiano Abram, Lawrence Roy, and Peter Scholl. Succinct homomorphic secret sharing. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 301–330, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.

[BGH+25] Elette Boyle, Niv Gilboa, Matan Hamilis, Yuval Ishai, and Yaxin Tu. Improved constructions for distributed multi-point functions. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 2414–2432, San Francisco, CA, USA, May 12–15, 2025. IEEE Computer Society Press.

[BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudorandom functions. In Hugo Krawczyk, editor, *PKC 2014*, volume 8383 of *LNCS*, pages 501–519, Buenos Aires, Argentina, March 26–28, 2014. Springer Berlin Heidelberg, Germany.

[BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 337–367, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.

[BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1292–1303, Vienna, Austria, October 24–28, 2016. ACM Press.

[BGIK22a] Elette Boyle, Niv Gilboa, Yuval Ishai, and Victor I. Kolobov. Information-theoretic distributed point functions. In Dana Dachman-Soled, editor, *ITC 2022*, volume 230 of *LIPIcs*, pages 17:1–17:14, Cambridge, MA, USA, July 5–7, 2022. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik.

[BGIK22b] Elette Boyle, Niv Gilboa, Yuval Ishai, and Victor I. Kolobov. Programmable distributed point functions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 121–151, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.

[BHR12] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 784–796, Raleigh, NC, USA, October 16–18, 2012. ACM Press.

[BKM17] Dan Boneh, Sam Kim, and Hart William Montgomery. Private puncturable PRFs from standard lattice assumptions. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part I*, volume 10210 of *LNCS*, pages 415–445, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland.

[BLW17] Dan Boneh, Kevin Lewi, and David J. Wu. Constraining pseudorandom functions privately. In Serge Fehr, editor, *PKC 2017, Part II*, volume 10175 of *LNCS*, pages 494–524, Amsterdam, The Netherlands, March 28–31, 2017. Springer Berlin Heidelberg, Germany.

[BMR90] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *22nd ACM STOC*, pages 503–513, Baltimore, MD, USA, May 14–16, 1990. ACM Press.

[BTVW17] Zvika Brakerski, Rotem Tsabary, Vinod Vaikuntanathan, and Hoeteck Wee. Private constrained PRFs (and more) from LWE. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 264–302, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland.

[BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part II*, volume 8270 of *LNCS*, pages 280–300, Bengalore, India, December 1–5, 2013. Springer Berlin Heidelberg, Germany.

[CC17] Ran Canetti and Yilei Chen. Constraint-hiding constrained PRFs for NC^1 from LWE. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part I*, volume 10210 of *LNCS*, pages 446–476, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland.

[CK24] Geoffroy Couteau and Naman Kumar. 10-party sublinear secure computation from standard assumptions. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IX*, volume 14928 of *LNCS*, pages 39–73, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.

[CKKZ12] Seung Geol Choi, Jonathan Katz, Ranjit Kumaresan, and Hong-Sheng Zhou. On the security of the "free-XOR" technique. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 39–53, Taormina, Sicily, Italy, March 19–21, 2012. Springer Berlin Heidelberg, Germany.

[CKY25] Geoffroy Couteau, Naman Kumar, and Xiaxi Ye. Multiparty homomorphic secret sharing and more from LPN and MQ. In Benny Applebaum and Huijia (Rachel) Lin, editors, *TCC 2025, Part II*, volume 16269 of *LNCS*, pages 498–530, Aarhus, Denmark, December 1–5, 2025. Springer, Cham, Switzerland.

[GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, October 1986.

[GHAHJ22] Aarushi Goel, Mathias Hall-Andersen, Aditya Hegde, and Abhishek Jain. Secure multiparty computation with free branching. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 397–426, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.

[GI14] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 640–658, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.

[GWW25] Aarushi Goel, Mingyuan Wang, and Zhiheng Wang. Multiparty distributed point functions. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part IV*, volume 16003 of *LNCS*, pages 140–173, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland.

[GZSP25] Ashrujit Ghoshal, Mingxun Zhou, Elaine Shi, and Bo Peng. Pseudorandom functions with weak programming privacy and applications to private information retrieval. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VII*, volume 15607 of *LNCS*, pages 284–313, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.

[HHK+22] Abida Haque, David Heath, Vladimir Kolesnikov, Steve Lu, Rafail Ostrovsky, and Akash Shah. Garbled circuits with sublinear evaluator. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 37–64, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.

[HJO+16] Brett Hemenway, Zahra Jafargholi, Rafail Ostrovsky, Alessandra Scafuro, and Daniel Wichs. Adaptively secure garbled circuits from one-way functions. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 149–178, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.

[HK20a] David Heath and Vladimir Kolesnikov. Stacked garbling - garbled circuit proportional to longest execution path. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 763–792, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.

[HK20b] David Heath and Vladimir Kolesnikov. Stacked garbling for disjunctive zero-knowledge proofs. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 569–598, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.

[HK21] David Heath and Vladimir Kolesnikov. LogStack: Stacked garbling with $O(b \log b)$ computation. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part III*, volume 12698 of *LNCS*, pages 3–32, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.

[HKN24] David Heath, Vladimir Kolesnikov, and Lucien K. L. Ng. Garbled circuit lookup tables with logarithmic number of ciphertexts. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part V*, volume 14655 of *LNCS*, pages 185–215, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.

[HKP20] David Heath, Vladimir Kolesnikov, and Stanislav Peceny. MOTIF: (almost) free branching in GMW - via vector-scalar multiplication. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 3–30, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland.

[JKO13] Marek Jawurek, Florian Kerschbaum, and Claudio Orlandi. Zero-knowledge using garbled circuits: how to prove non-algebraic statements efficiently. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 955–966, Berlin, Germany, November 4–8, 2013. ACM Press.

[KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 486–498, Reykjavik, Iceland, July 7–11, 2008. Springer Berlin Heidelberg, Germany.

[NK25] Lucien K. L. Ng and Vladimir Kolesnikov. Toss: Garbled PIR from table-only stacking. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 4634–4648, Taipei, Taiwan, October 13–17, 2025. ACM Press.

[PS18] Chris Peikert and Sina Shiehian. Privately constraining and programming PRFs, the LWE way. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part II*, volume 10770 of *LNCS*, pages 675–701, Rio de Janeiro, Brazil, March 25–29, 2018. Springer, Cham, Switzerland.

[Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167, Toronto, Ontario, Canada, October 27–29, 1986. IEEE Computer Society Press.

[YPHK23] Yibin Yang, Stanislav Peceny, David Heath, and Vladimir Kolesnikov. Towards generic MPC compilers via variable instruction set architectures (VISAs). In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 2516–2530, Copenhagen, Denmark, November 26–30, 2023. ACM Press.

[ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.

A Pseudorandomness of Master Evaluation Procedures

In this section, we study adaptive programmable PRFs (cf. Section 3.4) and ask whether PRF-under-uniform-programming security implies that the corresponding master evaluation procedure MEval is itself a PRF.

Security (PRF, no programming). An apPRF scheme $\Pi = (\mathsf{KeyGen}, \mathsf{MEval}, \mathsf{Prog}, \mathsf{Eval})$ is *PRF-secure without programming* if the following oracle ensembles are computationally indistinguishable:

$$\left\{\mathsf{MEval}(\mathsf{msk}, \cdot) \;\middle|\; \mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda) \right\}_\lambda \approx_c \left\{R(\cdot) \xleftarrow{\$} \mathsf{Rand}(\mathcal{X} \to \mathcal{Y})\right\}_\lambda$$

where $R : \mathcal{X} \to \mathcal{Y}$ is a uniformly random function. That is, when the programmed value is uniform, the induced oracle $x \mapsto \mathsf{MEval}(\mathsf{msk}, x)$ is a PRF.

A.1 Superpolynomial-size Domains

When the domain size $|\mathcal{X}|$ is superpolynomial, PRF security under uniform programming further implies PRF security without programming.

Theorem 12 (PRF under uniform programming $\Rightarrow$ PRF without programming). *Let $\Pi_{\mathsf{apPRF}} = (\mathsf{KeyGen}, \mathsf{MEval}, \mathsf{Prog}, \mathsf{Eval})$ be a correct apPRF over a domain $\mathcal{X}$ of superpolynomial size in λ. If Π_{apPRF} is* PRF-secure under uniform programming*, it is* PRF-secure without programming*.*

Proof. Fix any PPT distinguisher $\mathcal{A}$ making at most $q(\lambda)$ oracle queries. Consider the standard PRF experiment for $F_{\mathsf{msk}}(\cdot) := \mathsf{MEval}(\mathsf{msk}, \cdot)$, where the oracle is either $\mathsf{MEval}(\mathsf{msk}, \cdot)$ for $\mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda)$ or a uniform random function $R \xleftarrow{\$} \mathsf{Rand}(\mathcal{X} \to \mathcal{Y})$. We build a PPT distinguisher $\mathcal{B}$ for the *PRF-under-uniform-programming* experiment as follows.

Distinguisher $\mathcal{B}^{O(\cdot)}(1^\lambda)$. Sample $\alpha \xleftarrow{\$} \mathcal{X}$.[6] Run $\mathcal{A}$ and answer each oracle query x of $\mathcal{A}$ as follows: if $x = \alpha$, then $\mathcal{B}$ aborts and outputs a uniform bit; otherwise, $\mathcal{B}$ forwards x to its own oracle $O(\cdot)$ and returns the reply to $\mathcal{A}$. If $\mathcal{A}$ halts and outputs b, then $\mathcal{B}$ outputs b.

In the *real* uniform-programming experiment, $\mathcal{B}$'s oracle is $O(\cdot) = \mathsf{Eval}(\mathsf{sk}, \cdot)$ for $\mathsf{msk} \leftarrow \mathsf{KeyGen}(1^\lambda)$, $\beta \xleftarrow{\$} \mathcal{Y}$, and $\mathsf{sk} := \mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$. In the *ideal* experiment, $O(\cdot) = R(\cdot)$ for $R \xleftarrow{\$} \mathsf{Rand}(\mathcal{X} \to \mathcal{Y})$.

Let Bad be the event that $\mathcal{A}$ ever queries α (i.e., that $\mathcal{B}$ aborts). We first bound $\Pr[\mathsf{Bad}]$ in either experiment. Conditioned on $\neg\mathsf{Bad}$ up to any point, all oracle answers returned to $\mathcal{A}$ are taken at inputs $x \neq \alpha$. By apPRF correctness (off-point unchanged), for every $x \neq \alpha$ we have $\mathsf{Eval}(\mathsf{sk}, x) = \mathsf{MEval}(\mathsf{msk}, x)$, so the answers seen by $\mathcal{A}$ up to that point are distributed independently of α. Therefore, for each query index i,

$$\Pr[x_i = \alpha \mid \neg\mathsf{Bad} \text{ on the first } i-1 \text{ queries}] \;=\; \frac{1}{|\mathcal{X}|}$$

and by a union bound,

$$\Pr[\mathsf{Bad}] \;\leq\; \frac{q(\lambda)}{|\mathcal{X}|} \tag{1}$$

Next, we relate $\mathcal{A}$'s view in the standard PRF experiment for MEval to $\mathcal{B}$'s view in the uniform-programming PRF experiment. In the *real* case, conditioned on $\neg\mathsf{Bad}$, every query made by $\mathcal{A}$ is some

[6] We use that PRF-security under uniform programming holds for every fixed $\alpha \in \mathcal{X}$; in particular, it holds for the value of α sampled above.

$x \neq \alpha$, hence by correctness $\mathsf{Eval}(\mathsf{sk}, x) = \mathsf{MEval}(\mathsf{msk}, x)$ for all answered queries; thus $\mathcal{A}$'s entire transcript is identically distributed to its transcript when given oracle access to $\mathsf{MEval}(\mathsf{msk}, \cdot)$. In the *ideal* case, conditioned on $\neg\mathsf{Bad}$, $\mathcal{A}$ receives answers from a uniformly random function in both experiments. Note, $\mathcal{B}$ outputs a random bit upon Bad, so

$$\mathsf{Adv}^{\mathsf{prf}}_{\mathsf{MEval}}(\mathcal{A}) \;\leq\; \mathsf{Adv}^{\mathsf{uProg\text{-}prf}}_{\Pi_{\mathsf{apPRF}}}(\mathcal{B}) \;+\; \Pr[\mathsf{Bad}] \;\leq\; \mathsf{Adv}^{\mathsf{uProg\text{-}prf}}_{\Pi_{\mathsf{apPRF}}}(\mathcal{B}) \;+\; \frac{q(\lambda)}{|\mathcal{X}|}$$

by (1). Finally, since Π_{apPRF} is PRF-secure under uniform programming, the first term is negligible. Since $q(\lambda)$ is polynomial and $|\mathcal{X}|$ is superpolynomial, $q(\lambda)/|\mathcal{X}|$ is negligible. Hence $\mathsf{Adv}^{\mathsf{prf}}_{\mathsf{MEval}}(\mathcal{A})$ is negligible for every PPT $\mathcal{A}$, so $\mathsf{MEval}(\mathsf{msk}, \cdot)$ is a standard PRF (i.e., Π is PRF-secure without programming). □

A.2 Polynomial-size Domains

In contrast, for polynomial-size domains, this implication can fail. We give a simple counterexample.

Let $\mathcal{X} = \{0,1\}^n$ with $2^n = \mathsf{poly}(\lambda)$ and $\mathcal{Y} = \{0,1\}$. Define KeyGen to sample a master key msk that encodes the full truth table of $\mathsf{MEval}(\mathsf{msk}, \cdot)$ as follows: choose the first $2^n - 1$ outputs uniformly at random, and set the last output so that the overall parity is zero, that is,

$$\bigoplus_{x \in \{0,1\}^n} \mathsf{MEval}(\mathsf{msk}, x) = 0$$

Then MEval is clearly *not* a PRF: a distinguisher can query all $x \in \{0,1\}^n$ and check whether the XOR of the replies is zero.

Now define the programming procedure $\mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$ to output a programmed key sk that is identical to msk except that the entry at position α is overwritten with β (and $\mathsf{Eval}(\mathsf{sk}, \cdot)$ simply reads the resulting table). If $\beta \xleftarrow{\$} \{0,1\}$ is uniform, then $\mathsf{Eval}(\mathsf{sk}, \cdot)$ is distributed exactly as a uniformly random truth table over $\{0,1\}^n \to \{0,1\}$: the parity constraint is destroyed by re-randomizing a single entry. Hence, Π can satisfy PRF-under-uniform-programming security even though MEval is not a PRF (i.e., not PRF-secure without programming).

B Deferred Proofs

B.1 Proof of Lemma 2

Lemma 2. Let $\Pi = (\mathsf{Gen}, \mathsf{Eval})$ be a two-party DPF scheme. Suppose Π is *correct* and *secure* in the sense of per-share simulation security for both shares (defined in Section 3.2). Then Π satisfies *individual pseudorandomness*.

Proof. Fix $b = 1$; the argument for $b = 2$ is symmetric. For a key k, write $F_k : \mathcal{X} \to \mathcal{Y}$ for the induced function $F_k(x) := \mathsf{Eval}(k, x)$.

Step 1a: Single-point shift invariance. Fix any $\alpha \in \mathcal{X}$ and $\beta \in \mathcal{Y}$. For each λ, sample $(k^{\alpha,\beta}_{1,\lambda}, k^{\alpha,\beta}_{2,\lambda}) \leftarrow \mathsf{Gen}(1^\lambda, \alpha, \beta)$, and define the induced (random) functions $F^{\alpha,\beta}_{1,\lambda}(x) := \mathsf{Eval}(k^{\alpha,\beta}_{1,\lambda}, x)$ and $F^{\alpha,\beta}_{2,\lambda}(x) := \mathsf{Eval}(k^{\alpha,\beta}_{2,\lambda}, x)$.

By correctness, for every λ, we have for all $x \in \mathcal{X}$:

$$F^{\alpha,\beta}_{1,\lambda}(x) + F^{\alpha,\beta}_{2,\lambda}(x) = P_{\alpha,\beta}(x)$$

and hence, as functions,

$$F_{2,\lambda}^{\alpha,\beta} = P_{\alpha,\beta} - F_{1,\lambda}^{\alpha,\beta} \tag{$*$}$$

By security of share 2, for all β,

$$\left\{k_{2,\lambda}^{\alpha,\beta}\right\}_\lambda \approx_c \left\{\mathsf{Sim}_2(1^\lambda)\right\}_\lambda$$

Since Eval is PPT, this lifts to oracle indistinguishability, and in particular,

$$\left\{F_{2,\lambda}^{\alpha,\beta}\right\}_\lambda \approx_c \left\{F_{2,\lambda}^{\alpha,0}\right\}_\lambda \quad \text{(as oracles)}$$

Combining with $(*)$ for β and for $\beta = 0$, we obtain

$$P_{\alpha,\beta} - \left\{F_{1,\lambda}^{\alpha,\beta}\right\}_\lambda \approx_c -\left\{F_{1,\lambda}^{\alpha,0}\right\}_\lambda \quad \text{(as oracles)}$$

or equivalently,

$$\left\{F_{1,\lambda}^{\alpha,\beta}\right\}_\lambda \approx_c \left\{F_{1,\lambda}^{\alpha,0}\right\}_\lambda + P_{\alpha,\beta} \quad \text{(as oracles)}$$

By security of share 1, $\left\{F_{1,\lambda}^{\alpha,\beta}\right\}_\lambda \approx_c \left\{F_{1,\lambda}^{\alpha,0}\right\}_\lambda$ as oracles, yielding

$$\left\{F_{1,\lambda}^{\alpha,0}\right\}_\lambda \approx_c \left\{F_{1,\lambda}^{\alpha,0}\right\}_\lambda + P_{\alpha,\beta} \quad \text{(as oracles)} \tag{1}$$

We remark that since α was arbitrary, (1) holds for every fixed $\alpha \in \mathcal{X}$ and $\beta \in \mathcal{Y}$.

Step 1b: Global shift invariance. Fix arbitrary $\alpha^\star \in \mathcal{X}$ and $\beta^\star \in \mathcal{Y}$, and let $F_\lambda^\star := F_{1,\lambda}^{\alpha^\star,\beta^\star}$. Now, for any $\alpha \in \mathcal{X}$ and $\beta \in \mathcal{Y}$, by per-share simulation security for share 1, we have

$$\{F_\lambda^\star\}_\lambda \approx_c \{F_{1,\lambda}^{\alpha,0}\}_\lambda \quad \text{(as oracles)} \tag{2}$$

Post-processing (2) by adding $P_{\alpha,\beta}$ yields

$$\{F_\lambda^\star\}_\lambda + P_{\alpha,\beta} \approx_c \{F_{1,\lambda}^{\alpha,0}\}_\lambda + P_{\alpha,\beta} \quad \text{(as oracles)} \tag{3}$$

Applying (1) and combining with (2) and (3) yields:

$$\{F_\lambda^\star\}_\lambda \approx_c \{F_{1,\lambda}^{\alpha,0}\}_\lambda \approx_c \{F_{1,\lambda}^{\alpha,0}\}_\lambda + P_{\alpha,\beta} \approx_c \{F_\lambda^\star\}_\lambda + P_{\alpha,\beta} \quad \text{(as oracles)} \tag{4}$$

I.e., the oracle ensemble induced by a single DPF share is computationally indistinguishable from the same ensemble after adding an arbitrary point function.

Step 2: Hybrid argument (adaptive queries). Fix a stateful PPT adversary $\mathcal{A}$ with oracle access, and let $Q(\lambda)$ be a polynomial upper bound on its total number of oracle queries. We define a sequence of hybrid oracles $\{H_{i,\lambda}\}_{i=0}^{Q(\lambda)}$. Each hybrid maintains a table $T : \mathcal{X} \to \mathcal{Y}$ (initially empty) and a counter d (initially 0) that counts the number of *distinct* query points seen so far. On a query $x \in \mathcal{X}$, the oracle responds as follows:

1. If $T[x]$ is defined, return $T[x]$.
2. Otherwise increment $d \leftarrow d + 1$ and:
   - if $d \leq i$, sample $T[x] \xleftarrow{\$} \mathcal{Y}$ and return $T[x]$;

- if $d > i$, set $T[x] := F^\star_\lambda(x)$ and return $T[x]$.

Equivalently, $H_{i,\lambda}$ answers the first i *distinct* queries according to a lazy-sampled truly random function table, and answers all later (new) points by $F^\star_\lambda$.

Let $\mathcal{V}_{i,\lambda}$ denote $\mathcal{A}$'s output bit in an interaction with $H_{i,\lambda}$ (including $\mathcal{A}$'s internal randomness).

Claim 2.1. For every $i \in \{1, \ldots, Q(\lambda)\}$, $\{\mathcal{V}_{i-1,\lambda}\}_\lambda \approx_c \{\mathcal{V}_{i,\lambda}\}_\lambda$.

Proof of Claim 2.1. Suppose for contradiction there exists an i and a distinguisher $\mathcal{A}$ such that $\{\mathcal{V}_{i-1,\lambda}\}_\lambda$ and $\{\mathcal{V}_{i,\lambda}\}_\lambda$ are distinguishable with non-negligible advantage. We construct a PPT distinguisher $\mathcal{D}$ that breaks the shift-invariance property (4). Distinguisher $\mathcal{D}$ is given oracle access to $O(\cdot)$, where O is either $F^\star_\lambda(\cdot)$ or $F^\star_\lambda(\cdot) + P_{\alpha,\beta}(\cdot)$ for some (α, β). $\mathcal{D}$ runs $\mathcal{A}$ and answers $\mathcal{A}$'s oracle queries by simulating an oracle $\widetilde{H}(\cdot)$ defined as follows. $\widetilde{H}$ maintains a table T and a counter d exactly as above. On a query x:

1. If $T[x]$ is defined, return $T[x]$.
2. Otherwise increment $d \leftarrow d + 1$.
   - If $d < i$, sample $T[x] \xleftarrow{\$} \mathcal{Y}$ and return $T[x]$.
   - If $d = i$, set $\alpha := x$ and sample $\beta \xleftarrow{\$} \mathcal{Y}$. Then query $y \leftarrow O(x)$, define $T[x] := y - \beta$, and return $T[x]$.
   - If $d > i$, query $y \leftarrow O(x)$, set $T[x] := y$, and return $T[x]$.

Finally, $\mathcal{D}$ outputs whatever bit $\mathcal{A}$ outputs.

We emphasize that (α, β) are fixed by $\mathcal{D}$ *prior to learning any information about the challenger's oracle* O. Concretely, until the moment $d = i$, $\mathcal{D}$ answers $\mathcal{A}$ using only its own fresh randomness (the values sampled into $T[\cdot]$) and does not receive any oracle reply from O. Thus the random variables (α, β) depend only on $\mathcal{A}$'s internal coins and on $\mathcal{D}$'s fresh uniform samples, and are therefore independent of whether the challenger sampled O as $F^\star_\lambda$ or as $F^\star_\lambda + P_{\alpha,\beta}$. This is crucial: it ensures that $\mathcal{D}$ indeed contradicts (4), which guarantees indistinguishability for every *fixed* (α, β) (and hence also for any (α, β) sampled independently of the challenge bit).

We now analyze the simulation under the two cases for O.

<u>Case 1: $O = F^\star_\lambda$.</u> For the first $i - 1$ distinct points, $\widetilde{H}$ returns independent uniform values, matching $H_{i,\lambda}$. On the i-th distinct point $x = \alpha$, $\widetilde{H}$ returns $F^\star_\lambda(\alpha) - \beta$, which is uniform over $\mathcal{Y}$ (since β is uniform and independent), and is stored for repeat-consistency. All later new points are answered by $F^\star_\lambda$. Thus $\widetilde{H}$ is distributed identically to $H_{i,\lambda}$, and hence $\mathcal{D}^{F^\star_\lambda}$ perfectly simulates $\mathcal{A}^{H_{i,\lambda}}$.

<u>Case 2: $O = F^\star_\lambda + P_{\alpha,\beta}$.</u> For the first $i - 1$ distinct points, the answers are again independent uniform values. On the i-th distinct point $x = \alpha$, we have $O(\alpha) = F^\star_\lambda(\alpha) + \beta$, and so $\widetilde{H}$ returns $O(\alpha) - \beta = F^\star_\lambda(\alpha)$. All later new points are answered by $O(x) = F^\star_\lambda(x)$ since $P_{\alpha,\beta}(x) = 0$ for $x \neq \alpha$. Thus $\widetilde{H}$ is distributed identically to $H_{i-1,\lambda}$, and hence $\mathcal{D}^{F^\star_\lambda + P_{\alpha,\beta}}$ perfectly simulates $\mathcal{A}^{H_{i-1,\lambda}}$.

Therefore, $\mathcal{D}$ distinguishes the oracle ensembles $F^\star_\lambda$ and $F^\star_\lambda + P_{\alpha,\beta}$ with the same non-negligible advantage as $\mathcal{A}$ distinguishes $H_{i,\lambda}$ from $H_{i-1,\lambda}$, contradicting (4). This proves the claim. □

Step 3: Conclude individual pseudorandomness. By a hybrid argument over $i = 0, 1, \ldots, Q(\lambda)$ and Claim 2.1, we obtain

$$\left\{H_{0,\lambda}\right\}_\lambda \approx_c \left\{H_{Q(\lambda),\lambda}\right\}_\lambda \quad \text{(as oracles)}$$

Observe that $H_{0,\lambda}$ answers every query as $F^\star_\lambda(\cdot)$, and therefore $H_{0,\lambda} \equiv F^\star_\lambda$. Moreover, $H_{Q(\lambda),\lambda}$ answers all (distinct) queries using the lazy-sampled table T, and hence is distributed identically to a truly random function oracle $R \leftarrow \mathsf{Rand}(\mathcal{X} \to \mathcal{Y})$. Therefore, we have for any $\alpha^\star \in \mathcal{X}$ and $\beta^\star \in \mathcal{Y}$:

$$\left\{F^\star_\lambda\right\}_\lambda \approx_c \left\{R \leftarrow \mathsf{Rand}(\mathcal{X} \to \mathcal{Y})\right\}_\lambda \quad \text{(as oracles)}$$

proving that the share-induced oracle ensemble is PRF-like. □

B.2 Proof of Theorem 6

Theorem 6. If $\Pi_{\mathsf{napPRF}} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ is a *correct* and *index-hiding* napPRF for a single point over domain $\mathcal{X}$ and range $\mathcal{Y}$, then the construction in Figure 1 yields a two-party DPF $\Pi_{\mathsf{DPF}} = (\mathsf{Gen}, \mathsf{Eval})$ over domain $\mathcal{X}$ and range $\mathcal{Y}$ that is *correct* and *secure* (defined in Section 3.2).

Proof. We prove correctness and security for our induced Π_{DPF}.

Correctness. Let $(k_1, k_2) \leftarrow \Pi_{\mathsf{DPF}}.\mathsf{Gen}(1^\lambda, \alpha, \beta)$, where $k_1 = \mathsf{sk}^{(1)}$ and $k_2 = \mathsf{sk}^{(2)}$ with $\mathsf{msk} \leftarrow \Pi_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda)$, $r \xleftarrow{\$} \mathcal{Y}$, and

$$\mathsf{sk}^{(1)} := \Pi_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha, r) \quad \text{and} \quad \mathsf{sk}^{(2)} := \Pi_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha, r \oplus \beta)$$

At $x = \alpha$, by correct programming of the napPRF,

$$\begin{aligned}&\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1, \alpha) \oplus \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2, \alpha)\\ =&\Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(1)}, \alpha) \oplus \Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(2)}, \alpha) = r \oplus (r \oplus \beta) = \beta\end{aligned}$$

For any $x \neq \alpha$, by off-point value independence of the napPRF applied to $\mathsf{sk}^{(1)}$ and $\mathsf{sk}^{(2)}$ (which share the same index α),

$$\begin{aligned}&\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1, x) \oplus \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2, x)\\ =&\Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(1)}, x) \oplus \Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(2)}, x) = 0\end{aligned}$$

Therefore, $\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1, \cdot) \oplus \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2, \cdot) = P_{\alpha,\beta}(\cdot)$.

Security. Fix $b \in \{1, 2\}$. We show that there exists a PPT simulator Sim_b such that for all $\alpha \in \mathcal{X}$ and $\beta \in \mathcal{Y}$,

$$\left\{k_b \;\middle|\; (k_1, k_2) \leftarrow \Pi_{\mathsf{DPF}}.\mathsf{Gen}(1^\lambda, \alpha, \beta)\right\}_\lambda \approx_c \left\{\mathsf{Sim}_b(1^\lambda)\right\}_\lambda$$

Define $\mathsf{Sim}_1 := \mathsf{Sim}_2 := \mathsf{Sim}$, where Sim is the simulator guaranteed by the napPRF index-hiding property. It remains to show that, for each b, the real distribution of k_b is exactly an instance covered by napPRF index hiding.

Case $b = 1$. By construction of $\Pi_{\mathsf{DPF}}.\mathsf{Gen}$, we have $k_1 = \mathsf{sk}^{(1)}$ where

$$\mathsf{msk} \leftarrow \Pi_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda), \;\; r \xleftarrow{\$} \mathcal{Y}, \;\; \mathsf{sk}^{(1)} := \Pi_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha, r)$$

Thus $\{k_1\}_\lambda$ is exactly the LHS ensemble in the napPRF index-hiding definition (with uniform programmed value), and hence $\{k_1\}_\lambda \approx_c \{\mathsf{Sim}(1^\lambda)\}_\lambda$.

Case $b = 2$. By construction, $k_2 = \mathsf{Prog}(\mathsf{msk}, \alpha, r \oplus \beta)$ for uniform $r \xleftarrow{\$} \mathcal{Y}$. Since $r \oplus \beta$ is uniform over $\mathcal{Y}$ for every fixed β, $\{k_2\}_\lambda$ is identically distributed to the LHS ensemble in the napPRF index-hiding definition (i.e., with a uniform programmed value), and hence $\{k_2\}_\lambda \approx_c \{\mathsf{Sim}(1^\lambda)\}_\lambda$.

Combining the two cases, we conclude that for each $b \in \{1, 2\}$ there exists a PPT simulator Sim_b satisfying the DPF security definition. □

B.3 Proof of Theorem 7

Theorem 7. If $\Pi_{\mathsf{DPF}} = (\mathsf{Gen}, \mathsf{Eval})$ is a *correct* and *secure* two-party DPF over domain $\mathcal{X}$ and range $\{0,1\}$, then the construction in Figure 2 yields a napPRF $\Pi_{\mathsf{napPRF}} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ over domain $\mathcal{X}$ and range $\mathcal{Y}$ that is *correct* and *index-hiding* (defined in Section 4).

Proof. We prove correctness and security (index-hiding) for our induced Π_{napPRF}.

Correctness. Fix any $\lambda \in \mathbb{N}$ and any $\mathsf{msk} \leftarrow \Pi_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda)$. Write $\mathsf{msk} = \mathsf{msk}_1 \| \cdots \| \mathsf{msk}_m$ with each $\mathsf{msk}_i \in \{0,1\}^R$.

(1) Correct programming at α. Fix any $\alpha \in \mathcal{X}$ and any $\beta \in \mathcal{Y} = \{0,1\}^m$, and write $\beta = \beta_1 \| \cdots \| \beta_m$. Let $\mathsf{sk} := \Pi_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha, \beta)$ and parse $\mathsf{sk} = k^{(1)} \| \cdots \| k^{(m)}$. For each $i \in [m]$, Prog computes

$$(k_1^{(i)}, k_2^{(i)}) := \Pi_{\mathsf{DPF}}.\mathsf{Gen}(1^\lambda, \alpha, 1; \mathsf{msk}_i)$$

and then sets $k^{(i)} \in \{k_1^{(i)}, k_2^{(i)}\}$ so that $\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k^{(i)}, \alpha) = \beta_i$. This choice is well-defined because DPF correctness for the point function $P_{\alpha,1}$ guarantees

$$\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1^{(i)}, \alpha) \oplus \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2^{(i)}, \alpha) = P_{\alpha,1}(\alpha) = 1,$$

hence exactly one of the two bits $\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1^{(i)}, \alpha)$ and $\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2^{(i)}, \alpha)$ equals β_i. Therefore,

$$\Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}, \alpha) = \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k^{(1)}, \alpha) \| \cdots \| \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k^{(m)}, \alpha) = \beta_1 \| \cdots \| \beta_m = \beta$$

(2) Off-point value independence. Fix any $\alpha \in \mathcal{X}$, any $\beta^{(1)}, \beta^{(2)} \in \mathcal{Y}$, and any $x \in \mathcal{X} \setminus \{\alpha\}$. Let $\mathsf{sk}^{(j)} := \Pi_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha, \beta^{(j)})$ for $j \in \{1,2\}$, and parse $\mathsf{sk}^{(j)} = k^{(1,j)} \| \cdots \| k^{(m,j)}$.

Fix any coordinate $i \in [m]$. Both executions of Prog (with $\beta^{(1)}$ and with $\beta^{(2)}$) invoke DPF key generation with the *same* inputs and *the same* fixed randomness:

$$(k_1^{(i)}, k_2^{(i)}) := \Pi_{\mathsf{DPF}}.\mathsf{Gen}(1^\lambda, \alpha, 1; \mathsf{msk}_i)$$

They may differ only in whether they output $k_1^{(i)}$ or $k_2^{(i)}$ as the programmed key for coordinate i. However, since $x \neq \alpha$, DPF correctness implies

$$\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1^{(i)}, x) \oplus \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2^{(i)}, x) = P_{\alpha,1}(x) = 0$$

and hence $\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1^{(i)}, x) = \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2^{(i)}, x)$. Therefore, regardless of whether $k^{(i,j)}$ equals $k_1^{(i)}$ or $k_2^{(i)}$, we have

$$\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k^{(i,1)}, x) = \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1^{(i)}, x) = \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2^{(i)}, x) = \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k^{(i,2)}, x)$$

This holds for every $i \in [m]$, so concatenating over i yields

$$\Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(1)}, x) = \Pi_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(2)}, x)$$

establishing off-point value independence.

Security (index-hiding). We show that when $\beta \xleftarrow{\$} \mathcal{Y}$ is uniform, the distribution of sk can be simulated without α.

Fix any $\alpha \in \mathcal{X}$. For each $i \in [m]$ and each λ, let $K_{1,\lambda}^{(i)}, K_{2,\lambda}^{(i)}$ denote the random variables distributed as the two DPF keys output by $\Pi_{\mathsf{DPF}}.\mathsf{Gen}(1^\lambda, \alpha, 1; \mathsf{msk}_i)$ for $\mathsf{msk}_i \xleftarrow{\$} \{0,1\}^R$, and let $K_\lambda^{(i)}$ denote the random variable output by Prog as $k^{(i)}$.

In the real experiment, for each $i \in [m]$, the napPRF scheme samples msk_i and computes $(k_1^{(i)}, k_2^{(i)}) := \Pi_{\mathsf{DPF}}.\mathsf{Gen}(1^\lambda, \alpha, 1; \mathsf{msk}_i)$. Let $s_i := \Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_1^{(i)}, \alpha) \in \{0,1\}$. By DPF correctness, $\Pi_{\mathsf{DPF}}.\mathsf{Eval}(k_2^{(i)}, \alpha) = s_i \oplus 1$. Since $\beta_i \xleftarrow{\$} \{0,1\}$ is uniform and independent of $(k_1^{(i)}, k_2^{(i)})$, the output key $k^{(i)}$ satisfies

$$\Pr\Big[k^{(i)} = k_1^{(i)} \mid k_1^{(i)}, k_2^{(i)}\Big] = \Pr[\beta_i = s_i] = 1/2 \qquad \Pr\Big[k^{(i)} = k_2^{(i)} \mid k_1^{(i)}, k_2^{(i)}\Big] = 1/2$$

and hence for every fixed λ,

$$\mathsf{Dist}\big(K_\lambda^{(i)}\big) = \tfrac{1}{2}\,\mathsf{Dist}\big(K_{1,\lambda}^{(i)}\big) + \tfrac{1}{2}\,\mathsf{Dist}\big(K_{2,\lambda}^{(i)}\big) \tag{2}$$

where $(K_{1,\lambda}^{(i)}, K_{2,\lambda}^{(i)}) \leftarrow \Pi_{\mathsf{DPF}}.\mathsf{Gen}(1^\lambda, \alpha, 1)$.

By DPF security, for each $b \in \{1,2\}$ there exists a PPT simulator Sim_b such that for all α, $\big\{K_{b,\lambda}^{(i)}\big\}_\lambda \approx_c \big\{\mathsf{Sim}_b(1^\lambda)\big\}_\lambda$. Combining with (2), we obtain

$$\big\{K_\lambda^{(i)}\big\}_\lambda \approx_c \Big\{\tfrac{1}{2}\,\mathsf{Sim}_1(1^\lambda) + \tfrac{1}{2}\,\mathsf{Sim}_2(1^\lambda)\Big\}_\lambda$$

That is, $K_\lambda^{(i)}$ is indistinguishable from the output of the PPT procedure that samples $c \xleftarrow{\$} \{1,2\}$ and returns $\mathsf{Sim}_c(1^\lambda)$, which is independent of α.

Define a simulator Sim for the napPRF experiment as follows: On input 1^λ, for each $i \in [m]$ sample $c_i \xleftarrow{\$} \{1,2\}$ and set $k^{(i)} \leftarrow \mathsf{Sim}_{c_i}(1^\lambda)$; finally output $\mathsf{sk} := k^{(1)} \| \cdots \| k^{(m)}$. A standard hybrid argument over $i \in [m]$ (replacing one coordinate at a time) shows that the distribution of sk in the real experiment is computationally indistinguishable from $\mathsf{Sim}(1^\lambda)$. This proves index hiding. □

B.4 Proof of Theorem 8

Theorem 8. Let G be a PRG. $\Pi_{\mathsf{napPRF}}^{(0)}$ (cf. Figure 3) is a correct and index-hiding offline-online napPRF over domain $\{0,1\}^0$ and range $\{0,1\}^m$.

Proof. Correctness is trivial. For security (index-hiding), since $\mathcal{X} = \{0,1\}^0$ has a single element (i.e., ϵ), a perfect simulator is: sample $(\mathsf{msk}, \mathsf{sk}_{\mathsf{off}}) \leftarrow \mathsf{KeyGen}^{(0)}(1^\lambda)$, sample $\beta \xleftarrow{\$} \{0,1\}^m$, set $\mathsf{sk}_{\mathsf{on}} := \mathsf{Prog}^{(0)}(\mathsf{msk}, \epsilon, \beta)$, and output $(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}})$.

We also note a simpler simulator that will be useful later: $\mathsf{Sim}^{(0)}(1^\lambda)$ runs $(\mathsf{msk}, \mathsf{sk}_{\mathsf{off}}) \leftarrow \mathsf{KeyGen}^{(0)}(1^\lambda)$ except it replaces $G(s_{\mathsf{master}})$ with $2m$ i.i.d. uniform λ-bit strings, samples $U^{(0)} \xleftarrow{\$} \{0,1\}^{m\lambda}$, and outputs $(\mathsf{sk}_{\mathsf{off}}, U^{(0)})$. This works because of the PRG security and the following simple fact. Fix any bit $p \in \{0,1\}$. Let $(\mathcal{L}^{(0)}, \mathcal{L}^{(1)})$ be sampled as two i.i.d. uniform λ-bit strings, except that we overwrite the last bits so that $\mathsf{Lsb}(\mathcal{L}^{(0)}) = p$ and $\mathsf{Lsb}(\mathcal{L}^{(1)}) = p \oplus 1$. Let $\zeta \xleftarrow{\$} \{0,1\}$ and define $\mathcal{L} := \mathcal{L}^{(\zeta)}$. Then $\mathcal{L}$ is *uniform over* $\{0,1\}^\lambda$ and independent of p. □

B.5 Proof of Theorem 9

Theorem 9. Let G be a PRG and let F be a PRF. For every $k \geq 1$, the scheme $\Pi_{\mathsf{napPRF}}^{(k)}$ (cf. Figures 4 and 5) is a correct and index-hiding offline-online napPRF over domain $\{0,1\}^k$ and range $\{0,1\}^m$.

Proof. We prove correctness and security (index-hiding) for $\{\Pi_{\mathsf{napPRF}}^{(k)}\}_{k \geq 0}$.

Correctness. We prove correctness by induction on k.

The base case $k = 0$ is proved in Theorem 8. Fix $k \geq 1$ and assume, inductively, that $\Pi^{(k-1)}_{\mathsf{napPRF}}$ is correct, i.e., for every $(\mathsf{msk}', \mathsf{sk}'_{\mathsf{off}}) \leftarrow \mathsf{KeyGen}^{(k-1)}(1^\lambda)$, every $\alpha' \in \{0,1\}^{k-1}$ and $\beta \in \{0,1\}^m$, letting $\mathsf{sk}'_{\mathsf{on}} := \mathsf{Prog}^{(k-1)}(\mathsf{msk}', \alpha', \beta)$, we have: (i) $\mathsf{Eval}^{(k-1)}(\mathsf{sk}'_{\mathsf{off}}, \mathsf{sk}'_{\mathsf{on}}, \alpha') = \beta$, and (ii) for every $x' \neq \alpha'$, the value $\mathsf{Eval}^{(k-1)}(\mathsf{sk}'_{\mathsf{off}}, \mathsf{sk}'_{\mathsf{on}}, x')$ is independent of β.

Now fix λ, sample $(\mathsf{msk}, \mathsf{sk}_{\mathsf{off}}) \leftarrow \mathsf{KeyGen}^{(k)}(1^\lambda)$, fix $\alpha^\star \in \{0,1\}^k$ and $\beta^\star \in \{0,1\}^m$, and let $\mathsf{sk}_{\mathsf{on}} := \mathsf{Prog}^{(k)}(\mathsf{msk}, \alpha^\star, \beta^\star)$. Write $\alpha^\star = \alpha_1 \| \cdots \| \alpha_k$, $\beta^\star = \beta_1 \| \cdots \| \beta_m$ and $x = x_1 \| x'$ where $\alpha_1, \ldots, \alpha_k, \beta_1, \ldots, \beta_m, x_1 \in \{0,1\}$.

(i) On-point ($x = \alpha^\star$). Since $\mathsf{sk}_{\mathsf{on}}$ contains $\mathcal{L}_{\alpha,1} = \mathcal{L}^{(\alpha_1)}_{\alpha,1}$, the $\mathsf{Eval}^{(k)}$ in Item 1 decrypts the seed-table row $r := \mathsf{Lsb}(\mathcal{L}_{\alpha,1}) = \mathsf{Lsb}(\mathcal{L}^{(\alpha_1)}_{\alpha,1})$ and obtains $s := \mathsf{Dec}^{(1)}_{\mathcal{L}_{\alpha,1}}(\tau^{(k)}_{\mathsf{seed},r}, \mathcal{T}_{\mathsf{seed}}[r])$. By the construction of $\mathcal{T}_{\mathsf{seed}}$, this seed is precisely the *real* child seed used by $\mathsf{KeyGen}^{(k)}$ to generate the child instance along the *inactive* branch $1 - \alpha_1$ (here branch 0 is "left" and branch 1 is "right").

Next, in Item 2, the $\mathsf{Eval}^{(k)}$ computes

$$(\widehat{\mathsf{msk}}, \widehat{\mathsf{sk}_{\mathsf{off}}}) := \Pi^{(k-1)}_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda; s) = \Pi^{(k-1)}_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda; s_{1-\alpha_1}).$$

Hence $\widehat{\mathsf{sk}_{\mathsf{off}}}$ equals the offline key $\mathsf{sk}_{1-\alpha_1,\mathsf{off}}$ of the *opposite/inactive* child instance generated in $\mathsf{KeyGen}^{(k)}$. Since $\mathsf{sk}_{\mathsf{stack}} = \mathsf{sk}_{\mathsf{left,off}} \oplus \mathsf{sk}_{\mathsf{right,off}}$, we have

$$\mathsf{sk}^{(\mathsf{next})}_{\mathsf{off}} := \mathsf{sk}_{\mathsf{stack}} \oplus \widehat{\mathsf{sk}_{\mathsf{off}}} = (\mathsf{sk}_{\mathsf{left,off}} \oplus \mathsf{sk}_{\mathsf{right,off}}) \oplus \mathsf{sk}_{1-\alpha_1,\mathsf{off}} = \mathsf{sk}_{\alpha_1,\mathsf{off}}$$

namely the offline key of the *active* child instance (branch α_1).

Moreover, in Item 3, for each $i \in [k-1]$ the $\mathsf{Eval}^{(k)}$ decrypts the unique entry of $\mathcal{T}_{\alpha,i}$ indexed by (r, c) where $c := \mathsf{Lsb}(\mathcal{L}_{\alpha,i+1}) = \mathsf{Lsb}(\mathcal{L}^{(\alpha_{i+1})}_{\alpha,i+1})$. By construction of $\mathcal{T}_{\alpha,i}$, the component selected by $x_1 = \alpha_1$ equals the child label $\mathcal{L}^{(\alpha_{i+1})}_{\alpha,\alpha_1,i}$, while the other component is junk. Similarly, for each $i \in [m]$, decrypting $\mathcal{T}_{\beta,i}[r][c]$ (with $c := \mathsf{Lsb}(\mathcal{L}_{\beta,i})$) yields a pair whose component selected by $x_1 = \alpha_1$ equals the child label $\mathcal{L}^{(\beta_i)}_{\beta,\alpha_1,i}$.

Therefore,

$$\mathsf{sk}^{(\mathsf{next})}_{\mathsf{on}} = \{\mathcal{L}_{\alpha,x_1,i}\}_{i\in[k-1]} \parallel \{\mathcal{L}_{\beta,x_1,i}\}_{i\in[m]} = \{\mathcal{L}^{(\alpha_{i+1})}_{\alpha,\alpha_1,i}\}_{i\in[k-1]} \parallel \{\mathcal{L}^{(\beta_i)}_{\beta,\alpha_1,i}\}_{i\in[m]}$$

which is exactly the online key that $\Pi^{(k-1)}_{\mathsf{napPRF}}$ would output for index $\alpha' = \alpha_2 \| \cdots \| \alpha_k$ and value $\beta^\star$ under the active child master key msk_{α_1}. Applying the inductive correctness of $\Pi^{(k-1)}_{\mathsf{napPRF}}$ gives

$$\mathsf{Eval}^{(k)}(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}}, \alpha^\star) = \mathsf{Eval}^{(k-1)}(\mathsf{sk}^{(\mathsf{next})}_{\mathsf{off}}, \mathsf{sk}^{(\mathsf{next})}_{\mathsf{on}}, \alpha') = \beta^\star$$

(ii) Off-point ($x \neq \alpha^\star$). If $x_1 \neq \alpha_1$, then $x_1 = 1 - \alpha_1$ and the decrypted seed is $s = s_{1-\alpha_1} = s_{x_1}$. Consequently, $(\widehat{\mathsf{msk}}, \widehat{\mathsf{sk}_{\mathsf{off}}})$ and thus $\mathsf{sk}^{(\mathsf{next})}_{\mathsf{off}} = \mathsf{sk}_{\mathsf{stack}} \oplus \widehat{\mathsf{sk}_{\mathsf{off}}}$ are independent of $\beta^\star$. Crucially, in Item 3 the component selected by x_1 from each decrypted translation-table entry is the *junk* label by construction, so $\mathsf{sk}^{(\mathsf{next})}_{\mathsf{on}}$ is independent of $\beta^\star$ as well. Thus, the final output is independent of $\beta^\star$.

Otherwise $x_1 = \alpha_1$ but $x' \neq \alpha' = \alpha_2 \| \cdots \| \alpha_k$. In this case, the argument in (i) shows that $\mathsf{sk}^{(\mathsf{next})}_{\mathsf{off}} = \mathsf{sk}_{\alpha_1,\mathsf{off}}$ and $\mathsf{sk}^{(\mathsf{next})}_{\mathsf{on}}$ is the correctly formed (offline and online) programmed key for index α' and value $\beta^\star$ in the active child instance. Since the recursive call evaluates at $x' \neq \alpha'$, the inductive hypothesis (off-point correctness for $\Pi^{(k-1)}_{\mathsf{napPRF}}$) implies that $\mathsf{Eval}^{(k-1)}(\mathsf{sk}^{(\mathsf{next})}_{\mathsf{off}}, \mathsf{sk}^{(\mathsf{next})}_{\mathsf{on}}, x')$ is independent of $\beta^\star$. Therefore $\mathsf{Eval}^{(k)}(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}}, x)$ is independent of $\beta^\star$ as well.

This completes the inductive proof of correctness for $\Pi^{(k)}_{\mathsf{napPRF}}$.

Security (index-hiding). We prove index-hiding by induction on k.

The base case $k = 0$ is proved in Theorem 8. Fix $k \geq 1$ and assume inductively that $\Pi^{(k-1)}_{\mathsf{napPRF}}$ is index-hiding.

Experiments. For $\alpha \in \{0,1\}^k$, let $\mathsf{REAL}^{(k)}_\alpha$ be the distribution of

$$(\mathsf{sk_{off}}, \mathsf{sk_{on}}) \mid (\mathsf{msk}, \mathsf{sk_{off}}) \leftarrow \mathsf{KeyGen}^{(k)}(1^\lambda), \mathsf{sk_{on}} := \mathsf{Prog}^{(k)}(\mathsf{msk}, \alpha, \beta \xleftarrow{\$} \{0,1\}^m)$$

For later use, we also define $\mathsf{REAL}^{(0)}_\epsilon$ in the same way.

Index-hiding asks for a PPT simulator $\mathsf{Sim}^{(k)}$ such that for each α, $\mathsf{REAL}^{(k)}_\alpha \approx_c \mathsf{Sim}^{(k)}(1^\lambda)$. Clearly, it suffices to show $\mathsf{REAL}^{(k)}_\alpha \approx_c \mathsf{REAL}^{(k)}_{0^k}$ for all α, and then take the simulator $\mathsf{Sim}^{(k)}(1^\lambda)$ as:

$$(\mathsf{msk}, \mathsf{sk_{off}}) \leftarrow \mathsf{KeyGen}^{(k)}(1^\lambda),\ \beta \xleftarrow{\$} \{0,1\}^m,\ \text{output } (\mathsf{sk_{off}}, \mathsf{Prog}^{(k)}(\mathsf{msk}, 0^k, \beta))$$

Step 0: PRG-to-uniform for the master expansion. In $\mathsf{KeyGen}^{(k)}$, all top-level randomness (the two seeds $s_{\mathsf{left}}, s_{\mathsf{right}}$, the k α-wire label pairs and the m β-wire label pairs in msk, and the top-level junk strings) is sampled via $G(s_{\mathsf{master}})$ for uniform s_{master}. By PRG security, we may replace $G(s_{\mathsf{master}})$ by uniform randomness of the same length, obtaining an experiment $\widetilde{\mathsf{REAL}}^{(k)}_\alpha$ such that

$$\mathsf{REAL}^{(k)}_\alpha \approx_c \widetilde{\mathsf{REAL}}^{(k)}_\alpha \qquad \text{for all } \alpha$$

Henceforth, we work in $\widetilde{\mathsf{REAL}}^{(k)}_\alpha$ where all these top-level values are jointly uniform subject only to the point-and-permute LSB constraints.

For later use, we also define $\widetilde{\mathsf{REAL}}^{(0)}_\epsilon \approx_c \mathsf{REAL}^{(0)}_\epsilon$ in the same way.

Step 1: Identify decryptable vs. non-decryptable entries. Let $\alpha = \alpha_1 \| \alpha'$ with $\alpha' = \alpha_2 \| \cdots \| \alpha_k \in \{0,1\}^{k-1}$ and write $\beta = \beta_1 \| \cdots \| \beta_m$. In $\mathsf{sk_{on}}$, the adversary sees exactly one α-label per coordinate, $\mathcal{L}_{\alpha,i} = \mathcal{L}^{(\alpha_i)}_{\alpha,i}$ for $i \in [k]$, and one β-label per output bit, $\mathcal{L}_{\beta,i} = \mathcal{L}^{(\beta_i)}_{\beta,i}$ for $i \in [m]$. Consequently, it can decrypt:

- Exactly one entry of $\mathcal{T}_{\mathsf{seed}}$ — the row indexed by $r := \mathsf{Lsb}(\mathcal{L}_{\alpha,1}) = \mathsf{Lsb}(\mathcal{L}^{(\alpha_1)}_{\alpha,1})$;
- For each $i \in [k-1]$, exactly one entry of $\mathcal{T}_{\alpha,i}$ — the cell indexed by (r, c_i) where $c_i := \mathsf{Lsb}(\mathcal{L}_{\alpha,i+1})$;
- For each $i \in [m]$, exactly one entry of $\mathcal{T}_{\beta,i}$ — the cell indexed by (r, c'_i) where $c'_i := \mathsf{Lsb}(\mathcal{L}_{\beta,i})$.

Every other entry in $\mathcal{T}_{\mathsf{seed}}$, $\{\mathcal{T}_{\alpha,i}\}$, and $\{\mathcal{T}_{\beta,i}\}$ is encrypted under at least one key that is *not* contained in $\mathsf{sk_{on}}$.

Step 2: PRF hybrids. Define $\mathsf{Hyb}^{(k)}_\alpha$ from $\widetilde{\mathsf{REAL}}^{(k)}_\alpha$ by replacing *every* table entry in $\mathcal{T}_{\mathsf{seed}}$ (and $\{\mathcal{T}_{\alpha,i}\}$, $\{\mathcal{T}_{\beta,i}\}$, resp.) that is encrypted under at least one key not in $\mathsf{sk_{on}}$ by an independent uniform string in $\{0,1\}^\lambda$ (and $\{0,1\}^{2\lambda}$, resp.). (All tweaks are unique by construction.) Then for each fixed α,

$$\widetilde{\mathsf{REAL}}^{(k)}_\alpha \approx_c \mathsf{Hyb}^{(k)}_\alpha$$

by a standard entry-by-entry hybrid argument using PRF security of F.

Thus, to prove index-hiding, it remains to show that for all α, $\mathsf{Hyb}^{(k)}_\alpha$ is computationally indistinguishable from $\mathsf{Hyb}^{(k)}_{0^k}$.

For later use, we define $\mathsf{Hyb}_{\epsilon}^{(0)} \equiv \widetilde{\mathsf{REAL}}_{\epsilon}^{(0)}$; note, there is no PRF in $\widetilde{\mathsf{REAL}}_{\epsilon}^{(0)}$.

Step 3a: Removing suffix α'. Fix $\alpha_1 \in \{0,1\}$. We show that $\forall \alpha' \in \{0,1\}^{k-1}$,

$$\mathsf{Hyb}_{\alpha_1\|\alpha'}^{(k)} \approx_c \mathsf{Hyb}_{\alpha_1\|0^{k-1}}^{(k)}$$

for $k \geq 1$. The case $k = 1$ is trivial, so we assume $k > 1$ below.

In $\mathsf{Hyb}_{\alpha_1\|\alpha'}^{(k)}$, all non-decryptable table entries have been replaced by independent uniform strings, hence they are independent of α'. Therefore, any dependence on α' can only arise from: (i) the revealed labels in $\mathsf{sk}_{\mathsf{on}}$ and (ii) the decryptable cells (one row of $\mathcal{T}_{\mathsf{seed}}$ and one cell per table in $\{\mathcal{T}_{\alpha,i}\}$ and $\{\mathcal{T}_{\beta,i}\}$).

First, the distribution of $\mathsf{sk}_{\mathsf{on}}$ is independent of α': for each $j \in \{2,\ldots,k\}$ the revealed label $\mathcal{L}_{\alpha,j} = \mathcal{L}_{\alpha,j}^{(\alpha_j)}$ is uniform in $\{0,1\}^\lambda$ subject only to the point-and-permute constraint, regardless of α_j. (The same holds for the revealed β-labels, and $\beta \xleftarrow{\$} \{0,1\}^m$ is uniform.)

Next, we define an efficient extraction algorithm Ext_{α_1} that, given $(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}})$, outputs a level-$(k-1)$ pair $(\mathsf{sk}_{\mathsf{act,off}}, \mathsf{sk}_{\mathsf{act,on}})$ as follows:

1. Decrypt the unique seed-row $\mathcal{T}_{\mathsf{seed}}[r]$ using $\mathcal{L}_{\alpha,1} \in \mathsf{sk}_{\mathsf{on}}$ and obtain s.
2. Compute $(\widehat{\mathsf{msk}}, \widehat{\mathsf{sk}}) := \Pi_{\mathsf{napPRF}}^{(k-1)}.\mathsf{KeyGen}(1^\lambda; s)$ and $\mathsf{sk}_{\mathsf{act,off}} := \mathsf{sk}_{\mathsf{stack}} \oplus \widehat{\mathsf{sk}}$.
3. For each $i \in [k-1]$ (resp. $i \in [m]$), decrypt the unique opened cell of $\mathcal{T}_{\alpha,i}$ (resp. $\mathcal{T}_{\beta,i}$) using the corresponding two keys in $\mathsf{sk}_{\mathsf{on}}$. From each decrypted 2λ-bit plaintext, take the *active half*: the left half if $\alpha_1 = 0$ and the right half if $\alpha_1 = 1$. Concatenating these active-half labels yields $\mathsf{sk}_{\mathsf{act,on}}$.

By construction of $\Pi_{\mathsf{napPRF}}^{(k)}$, extracted $(\mathsf{sk}_{\mathsf{act,off}}, \mathsf{sk}_{\mathsf{act,on}})$ is distributed *exactly* as

$$(\mathsf{sk}'_{\mathsf{off}}, \mathsf{sk}'_{\mathsf{on}}) \mid (\mathsf{msk}', \mathsf{sk}'_{\mathsf{off}}) \leftarrow \mathsf{KeyGen}^{(k-1)}(1^\lambda),\ \mathsf{sk}'_{\mathsf{on}} := \mathsf{Prog}^{(k-1)}(\mathsf{msk}', \alpha', \beta \xleftarrow{\$} \{0,1\}^m)$$

namely $\mathsf{REAL}_{\alpha'}^{(k-1)}$. Moreover, the remaining information in $(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}})$ outside of Ext_{α_1} is independent of α' in $\mathsf{Hyb}_{\alpha_1\|\alpha'}^{(k)}$ (it is either uniform filler or top-level values whose distribution does not depend on α').

Therefore, by the inductive index-hiding hypothesis for $\Pi_{\mathsf{napPRF}}^{(k-1)}$ and closure under efficient post-processing / taking products with α'-independent auxiliary information, we may replace $\mathsf{REAL}_{\alpha'}^{(k-1)}$ by $\mathsf{REAL}_{0^{k-1}}^{(k-1)}$ inside the extracted component, obtaining $\mathsf{Hyb}_{\alpha_1\|\alpha'}^{(k)} \approx_c \mathsf{Hyb}_{\alpha_1\|0^{k-1}}^{(k)}$.

Step 3b: Hiding α_1. It remains to show

$$\mathsf{Hyb}_{0^k}^{(k)} \approx_c \mathsf{Hyb}_{1\|0^{k-1}}^{(k)}$$

for $k \geq 1$. Now, fix $k \geq 1$ and consider the distribution $\mathsf{Hyb}_{0^k}^{(k)}$. We define $\mathsf{FCHyb}_{0^k}^{(k)}$ by further modifying $\mathsf{Hyb}_{0^k}^{(k)}$ *recursively down the leftmost path*:

Definition 8 ($\mathsf{FCHyb}_{0^k}^{(k)}$). *We define a distribution over $(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}})$ for $\alpha = 0^k$.*

1. ***(Generate $\mathsf{sk}_{\mathsf{off}}$ with cleaning on the 0^k spine).*** *Run $\mathsf{KeyGen}^{(k)}$ to obtain $\mathsf{sk}_{\mathsf{off}}$, except that for the left child instance seeded by s_{left} (and recursively its left child, etc., for k levels down to $\Pi_{\mathsf{napPRF}}^{(0)}$), replace the master expansion $G(s_{\mathsf{left}})$ used in that child $\mathsf{KeyGen}^{(\cdot)}$ by uniform randomness of the same length (followed by the usual point-and-permute LSB overwrite).*

2. ***(Sample β and form top-level $\mathsf{sk}_{\mathsf{on}}$).*** *Sample $\beta \xleftarrow{\$} \{0,1\}^m$ and form top-level $\mathsf{sk}_{\mathsf{on}}$ exactly as in $\mathsf{Hyb}^{(k)}_{0^k}$.*

3. ***(Hybridize unopened entries along the 0^k spine).*** *For each of the k instances on the left spine, apply Step 2 at that instance: within its $\mathsf{sk}_{\mathsf{off}}$, replace every table entry encrypted under at least one key that is* not revealed *by the corresponding (derived) programmed key at that level by an independent uniform λ-bit or 2λ-bit string.*

All other parts of the transcript are unchanged.

For later use, we define $\mathsf{FCHyb}^{(0)}_{\epsilon} \equiv \mathsf{Hyb}^{(0)}_{\epsilon}$.

Let $U^{(k)} \xleftarrow{\$} \{0,1\}^{(k+m)\lambda}$ be an independent uniform string with the same length/serialization as $\mathsf{sk}_{\mathsf{on}}$. We prove:

Lemma 3. *For every $k \geq 0$,*

$$(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}}) \mid (\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}}) \leftarrow \mathsf{FCHyb}^{(k)}_{0^k} \approx_c (\mathsf{sk}_{\mathsf{off}}, U^{(k)})$$

Proof. We prove by induction on k.

Base case $k = 0$. This follows directly from the proof of Theorem 8.

Inductive step. Assume the lemma holds for $k-1$. Consider $\mathsf{FCHyb}^{(k)}_{0^k}$. As in $\mathsf{Hyb}^{(k)}$, all non-decryptable *top-level* entries are uniform fillers already, so any dependence between $\mathsf{sk}_{\mathsf{off}}$ and $\mathsf{sk}_{\mathsf{on}}$ can only arise through the *decryptable* top-level cells: the opened seed-row and the opened cells of $\{\mathcal{T}_{\alpha,i}\}_{i\in[k-1]}$ and $\{\mathcal{T}_{\beta,i}\}_{i\in[m]}$.

- *Decryptable seed-row.* Its plaintext is s_{right}, an independent uniform λ-bit string (this is ensured already by the k-th level Step 0), hence its ciphertext is uniform even given the seed-row key $\mathcal{L}_{\alpha,1} \in \mathsf{sk}_{\mathsf{on}}$.
- *Decryptable translation cells.* Fix any opened translation cell. In $\alpha = 0^k$, its plaintext is of the form $(\mathcal{L}\|\mathcal{J})$, where $\mathcal{J}$ is a top-level junk string (uniform), and $\mathcal{L}$ is the corresponding *left-child* label (an entry of the next-level online programmed key that evaluation would recurse on). By definition of $\mathsf{FCHyb}^{(k)}_{0^k}$, the left child instance is distributed exactly as $\mathsf{FCHyb}^{(k-1)}_{0^{k-1}}$ (using the same random β), including the recursive Step 2 hybridization within that child. Therefore, by the inductive hypothesis, the left-child online key (hence each such label $\mathcal{L}$) is indistinguishable from an independent uniform λ-bit string even given the entire left-child offline key. Consequently, each plaintext $(\mathcal{L}\|\mathcal{J})$ is indistinguishable from two independent uniform λ-bit strings, and its ciphertext is uniform even given its decryption keys.

Thus, conditioned on $\mathsf{sk}_{\mathsf{off}}$, all decryptable ciphertexts that involve keys from $\mathsf{sk}_{\mathsf{on}}$ are (computationally) uniform and do not correlate with those keys; all other entries are uniform fillers. It follows that $\mathsf{sk}_{\mathsf{on}}$ is computationally independent of $\mathsf{sk}_{\mathsf{off}}$, and since it is uniform, we may replace it by an independent $U^{(k)}$, completing the induction. □

Similarly, define $\mathsf{FCHyb}^{(k)}_{1\|0^{k-1}}$ starting from $\mathsf{Hyb}^{(k)}_{1\|0^{k-1}}$ and recursively cleaning/hybridizing down the *active path* for $\alpha = 1\|0^{k-1}$, i.e., take the right child at the root and then the left child thereafter (always using the same β).

Claim 3b.1 (Cleaning both sides).

$$\mathsf{Hyb}^{(k)}_{0^k} \approx_c \mathsf{FCHyb}^{(k)}_{0^k} \qquad \text{and} \qquad \mathsf{Hyb}^{(k)}_{1\|0^{k-1}} \approx_c \mathsf{FCHyb}^{(k)}_{1\|0^{k-1}}$$

This holds naturally by PRG and PRF security.

Claim 3b.2 (Uniform $\mathsf{sk_{on}}$ given $\mathsf{sk_{off}}$ on both sides). Let $U^{(k)}$ be sampled as $(k+m)$ independent λ-bit strings. Then Lemma 3 gives

$$(\mathsf{sk_{off}}, \mathsf{sk_{on}}) \mid (\mathsf{sk_{off}}, \mathsf{sk_{on}}) \leftarrow \mathsf{FCHyb}^{(k)}_{0^k} \approx_c (\mathsf{sk_{off}}, U^{(k)})$$

and the symmetric argument yields

$$(\mathsf{sk_{off}}, \mathsf{sk_{on}}) \mid (\mathsf{sk_{off}}, \mathsf{sk_{on}}) \leftarrow \mathsf{FCHyb}^{(k)}_{1\|0^{k-1}} \approx_c (\mathsf{sk_{off}}, U^{(k)})$$

Claim 3b.3 (Once $\mathsf{sk_{on}}$ is uniform, α_1 disappears). Consider the two distributions obtained by replacing $\mathsf{sk_{on}}$ with $U^{(k)}$ in $\mathsf{FCHyb}^{(k)}_{0^k}$ and in $\mathsf{FCHyb}^{(k)}_{1\|0^{k-1}}$. We claim these are *identically distributed.* Indeed, conditioned on $\mathsf{sk_{off}}$ and $U^{(k)}$:

- All *non-decryptable* top-level table entries are uniform fillers by definition of $\mathsf{Hyb}^{(k)}$ (and remain so in FCHyb).
- The unique decryptable seed-row plaintext is an independent uniform λ-bit string (by top-level Step 0), so its ciphertext is uniform even given the revealed seed-row key; hence it carries no α_1-dependence.
- Each opened translation-cell plaintext at the top level is of the form

  $$(\text{child label}) \,\|\, (\text{junk}) \quad \text{if } \alpha_1 = 0 \qquad (\text{junk}) \,\|\, (\text{child label}) \quad \text{if } \alpha_1 = 1$$

  After replacing $\mathsf{sk_{on}}$ by $U^{(k)}$, the "child label" component is just an independent uniform λ-bit string, and the junk half is uniform as well. Therefore each opened plaintext is a concatenation of two independent uniform λ-bit strings, whose distribution is invariant under swapping left/right halves. Thus the fact that the "real" half appears on the left for $\alpha_1 = 0$ and on the right for $\alpha_1 = 1$ becomes distributionally irrelevant.

Since every component of $\mathsf{sk_{off}}$ that could reflect the choice of α_1 is either (i) a uniform filler, or (ii) an encryption of a uniform plaintext, the overall joint distribution does not depend on α_1. Hence, after replacing $\mathsf{sk_{on}}$ by $U^{(k)}$, the two FCHyb experiments are identically distributed.

Conclusion of Step 3b. Chaining the above,

$$\mathsf{Hyb}^{(k)}_{0^k} \approx_c \mathsf{FCHyb}^{(k)}_{0^k} \approx_c (\mathsf{sk_{off}}, U^{(k)}) \equiv (\mathsf{sk_{off}}, U^{(k)}) \approx_c \mathsf{FCHyb}^{(k)}_{1\|0^{k-1}} \approx_c \mathsf{Hyb}^{(k)}_{1\|0^{k-1}}$$

which completes Step 3b. □

C Formalism of Garbling-based Short-key Compiler

C.1 Garbling Schemes

Definition 9 (Garbling). *A **garbling scheme** is a pair of PPT algorithms:*

- $\mathsf{Gb}(1^\lambda, \mathcal{C})$ *takes as input a standard fan-in-2 Boolean circuit* $\mathcal{C} : \{0,1\}^n \to \{0,1\}^m$ *and outputs a garbled circuit* $\widehat{\mathcal{C}}$ *with input labels* $\{\mathcal{L}_i^{(0)}, \mathcal{L}_i^{(1)}\}_{i\in[n]}$.
- $\mathsf{Ev}(1^\lambda, \mathcal{C}, \widehat{\mathcal{C}}, \mathcal{L}_1, \ldots, \mathcal{L}_n)$ *takes* $\mathcal{C}$, $\widehat{\mathcal{C}}$, *and one label* $\mathcal{L}_i$ *for each input wire* $i \in [n]$, *and outputs a string* $y \in \{0,1\}^m$.

*The scheme is **correct** if for all* $\lambda \in \mathbb{N}$, *all Boolean circuits* $\mathcal{C} : \{0,1\}^n \to \{0,1\}^m$, *and all* $\boldsymbol{x} \in \{0,1\}^n$

$$\Pr\left[\mathsf{Ev}(1^\lambda, \mathcal{C}, \widehat{\mathcal{C}}, \mathcal{L}_1^{(x_1)}, \ldots, \mathcal{L}_n^{(x_n)}) = \mathcal{C}(\boldsymbol{x}) \;\middle|\; (\widehat{\mathcal{C}}, \{\mathcal{L}_i^{(0)}, \mathcal{L}_i^{(1)}\}_{i\in[n]}) \leftarrow \mathsf{Gb}(1^\lambda, \mathcal{C})\right] = 1$$

$$\Pi^{(\mathsf{new})}_{\mathsf{napPRF}}$$

- $\underline{\mathsf{KeyGen}^{(\mathsf{new})}(1^\lambda)}$. Sample $\mathsf{msk}^{(\mathsf{old})} \leftarrow \mathsf{KeyGen}^{(\mathsf{old})}(1^\lambda)$. Let $\ell_{\mathsf{msk}} := |\mathsf{msk}^{(\mathsf{old})}|$, and let $\mathcal{C}_{\mathsf{Prog}}$ be a Boolean circuit that on input $(\mathsf{msk}, \alpha, \beta) \in \{0,1\}^{\ell_{\mathsf{msk}}} \times \{0,1\}^n \times \{0,1\}^m$ outputs $\mathcal{C}_{\mathsf{Prog}}(\mathsf{msk}, \alpha, \beta) := \mathsf{Prog}^{(\mathsf{old})}(\mathsf{msk}, \alpha, \beta)$. Garble $\mathcal{C}_{\mathsf{Prog}}$:
$$\big(\widehat{\mathcal{C}}, \{\mathcal{L}_i^{(0)}, \mathcal{L}_i^{(1)}\}_{i \in [\ell_{\mathsf{msk}}+n+m]}\big) \leftarrow \mathsf{Gb}(1^\lambda, \mathcal{C}_{\mathsf{Prog}})$$
Define *fixed master-key labels* $\mathcal{L}_{\mathsf{msk}} := \big(\mathcal{L}_1^{(\mathsf{msk}^{(\mathsf{old})}[1])}, \ldots, \mathcal{L}_{\ell_{\mathsf{msk}}}^{(\mathsf{msk}^{(\mathsf{old})}[\ell_{\mathsf{msk}}])}\big)$. Output
$$\mathsf{msk}^{(\mathsf{new})} := \big(\{\mathcal{L}^{(0)}_{\ell_{\mathsf{msk}}+j}, \mathcal{L}^{(1)}_{\ell_{\mathsf{msk}}+j}\}_{j \in [n+m]}\big) \quad \text{and} \quad \mathsf{sk}^{(\mathsf{new})}_{\mathsf{off}} := \big(\widehat{\mathcal{C}}, \mathcal{L}_{\mathsf{msk}}\big)$$
- $\underline{\mathsf{Prog}^{(\mathsf{new})}(\mathsf{msk}^{(\mathsf{new})}, \alpha, \beta)}$. Parse $\mathsf{msk}^{(\mathsf{new})}$ as $\{\mathcal{L}^{(0)}_{\ell_{\mathsf{msk}}+j}, \mathcal{L}^{(1)}_{\ell_{\mathsf{msk}}+j}\}_{j \in [n+m]}$. Output
$$\mathsf{sk}^{(\mathsf{new})}_{\mathsf{on}} := \big(\mathcal{L}^{(\alpha_1)}_{\ell_{\mathsf{msk}}+1}, \ldots, \mathcal{L}^{(\alpha_n)}_{\ell_{\mathsf{msk}}+n}, \mathcal{L}^{(\beta_1)}_{\ell_{\mathsf{msk}}+n+1}, \ldots, \mathcal{L}^{(\beta_m)}_{\ell_{\mathsf{msk}}+n+m}\big)$$
- $\underline{\mathsf{Eval}^{(\mathsf{new})}(\mathsf{sk}^{(\mathsf{new})}_{\mathsf{off}}, \mathsf{sk}^{(\mathsf{new})}_{\mathsf{on}}, x)}$. Parse $\mathsf{sk}^{(\mathsf{new})}_{\mathsf{off}} = (\widehat{\mathcal{C}}, \mathcal{L}_{\mathsf{msk}})$. First, recover the old programmed key by evaluating the garbling:
$$\mathsf{sk}^{(\mathsf{old})} := \mathsf{Ev}(1^\lambda, \mathcal{C}_{\mathsf{Prog}}, \widehat{\mathcal{C}}, \mathcal{L}_{\mathsf{msk}}, \mathsf{sk}^{(\mathsf{new})}_{\mathsf{on}})$$
Output $y := \mathsf{Eval}^{(\mathsf{old})}(\mathsf{sk}^{(\mathsf{old})}, x)$.

Figure 6: Garbling-based short-key compiler.

Security. A garbling scheme $\Pi = (\mathsf{Gb}, \mathsf{Ev})$ is *secure* if there exists a PPT simulator Sim such that for all Boolean circuits $\mathcal{C} : \{0,1\}^n \to \{0,1\}^m$, and all $\boldsymbol{x} \in \{0,1\}^n$, the following ensembles are computationally indistinguishable:
$$\Big\{\big(\widehat{\mathcal{C}}, \mathcal{L}_1^{(x_1)}, \ldots, \mathcal{L}_n^{(x_n)}\big) \;\Big|\; \big(\widehat{\mathcal{C}}, \{\mathcal{L}_i^{(0)}, \mathcal{L}_i^{(1)}\}_{i \in [n]}\big) \leftarrow \mathsf{Gb}(1^\lambda, \mathcal{C})\Big\}_\lambda \approx_c \Big\{\mathsf{Sim}(1^\lambda, \mathcal{C}, \mathcal{C}(\boldsymbol{x}))\Big\}_\lambda$$

C.2 Formalism

Let $\Pi^{(\mathsf{old})}_{\mathsf{napPRF}} = (\mathsf{KeyGen}^{(\mathsf{old})}, \mathsf{Prog}^{(\mathsf{old})}, \mathsf{Eval}^{(\mathsf{old})})$ be a (standard) napPRF scheme with domain $\{0,1\}^n$ and range $\{0,1\}^m$, and let $\Pi_{\mathsf{GC}} = (\mathsf{Gb}, \mathsf{Ev})$ be the (Yao) garbling scheme. We construct an offline-online napPRF scheme $\Pi^{(\mathsf{new})}_{\mathsf{napPRF}} = (\mathsf{KeyGen}^{(\mathsf{new})}, \mathsf{Prog}^{(\mathsf{new})}, \mathsf{Eval}^{(\mathsf{new})})$ as depicted in Figure 6.

Theorem 13. *Let* $\Pi^{(\mathsf{old})}_{\mathsf{napPRF}} = (\mathsf{KeyGen}^{(\mathsf{old})}, \mathsf{Prog}^{(\mathsf{old})}, \mathsf{Eval}^{(\mathsf{old})})$ *be a* correct *and* index-hiding *napPRF over domain* $\mathcal{X} = \{0,1\}^n$ *and range* $\mathcal{Y} = \{0,1\}^m$. *Let* $\Pi_{\mathsf{GC}} = (\mathsf{Gb}, \mathsf{Ev})$ *be a* correct *and* secure *garbling scheme (cf. Appendix C.1). Then the construction in Figure 6 yields an offline-online napPRF* $\Pi^{(\mathsf{new})}_{\mathsf{napPRF}} = (\mathsf{KeyGen}^{(\mathsf{new})}, \mathsf{Prog}^{(\mathsf{new})}, \mathsf{Eval}^{(\mathsf{new})})$ *over the same domain and range that is* correct *and* index-hiding.

Moreover, in the resulting scheme the online key $\mathsf{sk}^{(\mathsf{new})}_{\mathsf{on}}$ *consists of exactly* $(n+m)$ *garbled input labels; in particular, under standard Yao garbling (with* λ*-bit labels) it has length* $(n+m)\lambda$ *bits.*

Proof sketch. Correctness and index hiding are both straightforward to verify. We therefore only briefly describe the simulator for the index-hiding experiment.

The goal is to simulate the joint distribution of $(\mathsf{sk}_{\mathsf{off}}^{(\mathsf{new})}, \mathsf{sk}_{\mathsf{on}}^{(\mathsf{new})})$. Recall that $\mathsf{sk}_{\mathsf{off}}^{(\mathsf{new})}$ consists of a garbled circuit $\widehat{\mathcal{C}}$ for the public circuit $\mathcal{C}_{\mathsf{Prog}}$ and a bundle of "fixed" garbled labels (corresponding to the $\mathsf{msk}^{(\mathsf{old})}$ input wires). The online key $\mathsf{sk}_{\mathsf{on}}^{(\mathsf{new})}$ consists of the remaining $(n+m)$ garbled labels (corresponding to the (α, β) input wires). Evaluating the garbled circuit on these labels yields the old programmed key $\mathsf{sk}^{(\mathsf{old})}$.

The simulator proceeds as follows. First, it invokes the index-hiding simulator for $\Pi_{\mathsf{napPRF}}^{(\mathsf{old})}$ to obtain a simulated programmed key $\widetilde{\mathsf{sk}}^{(\mathsf{old})}$. Next, since $\mathcal{C}_{\mathsf{Prog}}$ is public, it invokes the garbling simulator on input $(1^\lambda, \mathcal{C}_{\mathsf{Prog}}, \widetilde{\mathsf{sk}}^{(\mathsf{old})})$ to obtain a simulated garbled circuit $\widetilde{\mathcal{C}}$ together with a full set of active input labels consistent with output $\widetilde{\mathsf{sk}}^{(\mathsf{old})}$. Finally, it partitions these active labels into (i) the labels for the $\mathsf{msk}^{(\mathsf{old})}$ input wires, which it places into $\widetilde{\mathsf{sk}}_{\mathsf{off}}^{(\mathsf{new})}$ alongside $\widetilde{\mathcal{C}}$, and (ii) the labels for the (α, β) input wires, which it outputs as $\widetilde{\mathsf{sk}}_{\mathsf{on}}^{(\mathsf{new})}$. □

D Tailored Offline-online napPRF for LogStack

We formally present our offline-online napPRF constructions tailored to the LogStack application; Section 7.2 includes overview. We emphasize that we still define the schemes — and their security — at the level of a napPRF, rather than as outputting GC labels. Adapting the construction to the variant used inside SGC's demultiplexer is straightforward (it suffices to drop the decoding bits).

The constructions follow the paradigm from Section 6.2. The key difference is that we eliminate the seed table $\mathcal{T}_{\mathsf{seed}}$ entirely. Instead, KeyGen samples a full binary tree of i.i.d. dummy seeds; this tree is stored into msk and, in Prog, is modified (as SortingHat) so that the sibling roots along the active path carry the real GGM seeds while all remaining nodes carry dummy seeds.

As Section 6.2, we (re)define a family of offline-online napPRF schemes $\{\Pi_{\mathsf{napPRF}}^{(i)}\}_{i\in[n]}$, where each $\Pi_{\mathsf{napPRF}}^{(i)} : \{0,1\}^i \to \{0,1\}^m$. $\Pi_{\mathsf{napPRF}}^{(0)}$ is unchanged.

Recursive definitions. Fix $k \geq 1$. We (re)define $\Pi_{\mathsf{napPRF}}^{(k)} = (\mathsf{KeyGen}, \mathsf{Prog}, \mathsf{Eval})$ over domain $\mathcal{X} = \{0,1\}^k$ and range $\mathcal{Y} = \{0,1\}^m$. The construction of $\Pi_{\mathsf{napPRF}}^{(k)}$ makes *black-box* use of $\Pi_{\mathsf{napPRF}}^{(k-1)}$.

We write $\tau_{\mathsf{seed},r}^{(k)}$, $\tau_{\beta,i,r,c}^{(k)}$, and $\tau_{\alpha,i,r,c}^{(k)}$ for public, *unique* tweaks for the corresponding (garbled) table entries. For notational convenience, we write

$$(\mathsf{msk}, \mathsf{sk}_{\mathsf{off}}) := \Pi_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda; s)$$

to denote running $\Pi_{\mathsf{napPRF}}.\mathsf{KeyGen}$ with randomness set as $s \in \{0,1\}^\lambda$. In particular, we will ensure that each KeyGen uses only a λ-bit random tape. The redefined scheme $\Pi_{\mathsf{napPRF}}^{(k)}$ is depicted in Figures 7 and 8, and satisfies:

Theorem 14 (Main (tailored)). *Let G be a PRG and let F be a PRF. For every $k \geq 1$, the tailored scheme $\Pi_{\mathsf{napPRF}}^{(k)}$ (cf. Figures 7 and 8) is a correct and index-hiding offline-online napPRF over domain $\{0,1\}^k$ and range $\{0,1\}^m$.*

Proof sketch. The proof follows the structure of Theorem 9; the only changes are that we drop $\mathcal{T}_{\mathsf{seed}}$ and $\mathsf{sk}_{\mathsf{on}}$ additionally contains a seed tree s. The correctness is trivial, so we only outline the index-hiding argument.

Redefined $\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda)$

1. Sample a master seed $s_{\mathsf{master}} \xleftarrow{\$} \{0,1\}^\lambda$. Use $G(s_{\mathsf{master}})$ to sample

$$s_{\mathsf{left}}, s_{\mathsf{right}} \in \{0,1\}^\lambda \quad \{\mathcal{L}^{(0)}_{\alpha,i}, \mathcal{L}^{(1)}_{\alpha,i} \in \{0,1\}^\lambda\}_{i\in[k]} \quad \{\mathcal{L}^{(0)}_{\beta,i}, \mathcal{L}^{(1)}_{\beta,i} \in \{0,1\}^\lambda\}_{i\in[m]}$$

$$\{\mathcal{J}_{\alpha,i} \in \{0,1\}^\lambda\}_{i\in[k-1]} \quad \{\mathcal{J}_{\beta,i} \in \{0,1\}^\lambda\}_{i\in[m]}$$

$$\mathsf{seed}' := \{s'_{i,j} \in \{0,1\}^\lambda\}_{i\in[k], j\in\{0,1,\ldots,2^i-1\}}$$

Point-and-permute constraint: overwrite the last bits so that

$$\forall i \in [k], \mathsf{Lsb}(\mathcal{L}^{(1)}_{\alpha,i}) := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\alpha,i}) \oplus 1 \qquad \forall i \in [m], \mathsf{Lsb}(\mathcal{L}^{(1)}_{\beta,i}) := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\beta,i}) \oplus 1$$

(All other bits remain as sampled, so $\mathcal{L}^{(1)}$ is not derivable from $\mathcal{L}^{(0)}$.)

2. **Generate two child instances.** For each $j \in \{\mathsf{left}, \mathsf{right}\}$, compute $(\mathsf{msk}_j, \mathsf{sk}_{j,\mathsf{off}}) := \Pi^{(k-1)}_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda; s_j)$. By construction, msk_j includes the child wire labels $\{\mathcal{L}^{(0)}_{\alpha,j,i}, \mathcal{L}^{(1)}_{\alpha,j,i}\}_{i\in[k-1]}$ and $\{\mathcal{L}^{(0)}_{\beta,j,i}, \mathcal{L}^{(1)}_{\beta,j,i}\}_{i\in[m]}$ along with auxiliary seeds (i.e., the GGM-derived material from s_j and the level-$(k-1)$ dummy-seed tree). Note that this dummy-seed tree is irrelevant. $\mathsf{sk}_{j,\mathsf{off}}$ is the level-$(k-1)$ offline key output by the recursive call (in a fixed canonical serialization).

3. Let $\mathsf{seed} := \{s_{i,j} \in \{0,1\}^\lambda\}_{i\in[k],\, j\in\{0,1,\ldots,2^i-1\}}$ be the GGM seeds derived from s_{master}, i.e., the recursive expansion trace obtained by iterating from s_{left} and s_{right}.

   Output $\mathsf{msk} := \{\mathcal{L}^{(0)}_{\alpha,i}, \mathcal{L}^{(1)}_{\alpha,i}\}_{i\in[k]}, \{\mathcal{L}^{(0)}_{\beta,i}, \mathcal{L}^{(1)}_{\beta,i}\}_{i\in[m]}, \mathsf{seed}, \mathsf{seed}'$ and $\mathsf{sk}_{\mathsf{off}}$ containing:

   - **Stacked child offline keys:** $\mathsf{sk}_{\mathsf{stack}} := \mathsf{sk}_{\mathsf{left,off}} \oplus \mathsf{sk}_{\mathsf{right,off}}$.
   - **α-translation tables** $\{\mathcal{T}_{\alpha,i}\}_{i\in[k-1]}$**.** Let $r := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\alpha,1})$.

     For each $i \in [k-1]$: Let $j := i+1, c := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\alpha,j})$ and define

$$\mathcal{T}_{\alpha,i}[r][c] := \mathsf{Enc}^{(2)}_{\mathcal{L}^{(0)}_{\alpha,1}, \mathcal{L}^{(0)}_{\alpha,j}}(\tau^{(k)}_{\alpha,i,r,c},\ \mathcal{L}^{(0)}_{\alpha,\mathsf{left},i} \| \mathcal{J}_{\alpha,i})$$
$$\mathcal{T}_{\alpha,i}[r][c\oplus 1] := \mathsf{Enc}^{(2)}_{\mathcal{L}^{(0)}_{\alpha,1}, \mathcal{L}^{(1)}_{\alpha,j}}(\tau^{(k)}_{\alpha,i,r,c\oplus 1},\ \mathcal{L}^{(1)}_{\alpha,\mathsf{left},i} \| \mathcal{J}_{\alpha,i})$$
$$\mathcal{T}_{\alpha,i}[r\oplus 1][c] := \mathsf{Enc}^{(2)}_{\mathcal{L}^{(1)}_{\alpha,1}, \mathcal{L}^{(0)}_{\alpha,j}}(\tau^{(k)}_{\alpha,i,r\oplus 1,c},\ \mathcal{J}_{\alpha,i} \| \mathcal{L}^{(0)}_{\alpha,\mathsf{right},i})$$
$$\mathcal{T}_{\alpha,i}[r\oplus 1][c\oplus 1] := \mathsf{Enc}^{(2)}_{\mathcal{L}^{(1)}_{\alpha,1}, \mathcal{L}^{(1)}_{\alpha,j}}(\tau^{(k)}_{\alpha,i,r\oplus 1,c\oplus 1},\ \mathcal{J}_{\alpha,i} \| \mathcal{L}^{(1)}_{\alpha,\mathsf{right},i})$$

   - **β-translation tables** $\{\mathcal{T}_{\beta,i}\}_{i\in[m]}$**.** Let $r := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\alpha,1})$.

     For each $i \in [m]$: Let $c := \mathsf{Lsb}(\mathcal{L}^{(0)}_{\beta,i})$ and define

$$\mathcal{T}_{\beta,i}[r][c] := \mathsf{Enc}^{(2)}_{\mathcal{L}^{(0)}_{\alpha,1}, \mathcal{L}^{(0)}_{\beta,i}}(\tau^{(k)}_{\beta,i,r,c},\ \mathcal{L}^{(0)}_{\beta,\mathsf{left},i} \| \mathcal{J}_{\beta,i})$$
$$\mathcal{T}_{\beta,i}[r][c\oplus 1] := \mathsf{Enc}^{(2)}_{\mathcal{L}^{(0)}_{\alpha,1}, \mathcal{L}^{(1)}_{\beta,i}}(\tau^{(k)}_{\beta,i,r,c\oplus 1},\ \mathcal{L}^{(1)}_{\beta,\mathsf{left},i} \| \mathcal{J}_{\beta,i})$$
$$\mathcal{T}_{\beta,i}[r\oplus 1][c] := \mathsf{Enc}^{(2)}_{\mathcal{L}^{(1)}_{\alpha,1}, \mathcal{L}^{(0)}_{\beta,i}}(\tau^{(k)}_{\beta,i,r\oplus 1,c},\ \mathcal{J}_{\beta,i} \| \mathcal{L}^{(0)}_{\beta,\mathsf{right},i})$$
$$\mathcal{T}_{\beta,i}[r\oplus 1][c\oplus 1] := \mathsf{Enc}^{(2)}_{\mathcal{L}^{(1)}_{\alpha,1}, \mathcal{L}^{(1)}_{\beta,i}}(\tau^{(k)}_{\beta,i,r\oplus 1,c\oplus 1},\ \mathcal{J}_{\beta,i} \| \mathcal{L}^{(1)}_{\beta,\mathsf{right},i})$$

Figure 7: Redefined $\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{KeyGen}$ **over** $\{0,1\}^k \to \{0,1\}^m$**.**

Redefined $\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{Prog}(\mathsf{msk}, \alpha \in \{0,1\}^k, \beta \in \{0,1\}^m)$

Parse $\alpha = \alpha_1 \| \cdots \| \alpha_k$ and $\beta = \beta_1 \| \cdots \| \beta_m$. Let $s := \mathsf{seed}'$ then overwrite s as follows: $\forall i \in [k], s[i][\alpha_1 \| \cdots \| \alpha_{i-1} \| (1-\alpha_i)] := \mathsf{seed}[i][\alpha_1 \| \cdots \| \alpha_{i-1} \| (1-\alpha_i)]$. That is, we start from the all-dummy seed tree seed' and replace the k sibling roots along the path to α by the corresponding *real* GGM seeds from seed. Output

$$\mathsf{sk}_{\mathsf{on}} := \{\mathcal{L}^{(\alpha_i)}_{\alpha,i}\}_{i\in[k]} \parallel \{\mathcal{L}^{(\beta_i)}_{\beta,i}\}_{i\in[m]} \parallel s$$

Redefined $\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}}, x \in \{0,1\}^k)$

Parse $\mathsf{sk}_{\mathsf{off}} = (\mathsf{sk}_{\mathsf{stack}}, \{\mathcal{T}_{\alpha,i}\}_{i\in[k-1]}, \{\mathcal{T}_{\beta,i}\}_{i\in[m]})$, $\mathsf{sk}_{\mathsf{on}} = (\{\mathcal{L}_{\alpha,i}\}_{i\in[k]}, \{\mathcal{L}_{\beta,i}\}_{i\in[m]}, s)$, and $x = x_1 \| \cdots \| x_k$.

1. **Re-derive inactive offline key via the top-level seed.** Compute

$$(\widehat{\mathsf{msk}}, \widehat{\mathsf{sk}_{\mathsf{off}}}) := \Pi^{(k-1)}_{\mathsf{napPRF}}.\mathsf{KeyGen}(1^\lambda; s[1][1-x_1])$$

2. **Decrypt translation for α and β.** Let $r := \mathsf{Lsb}(\mathcal{L}_{\alpha,1})$.
   For each $i \in [k-1]$: Let $c := \mathsf{Lsb}(\mathcal{L}_{\alpha,i+1})$ and compute

$$\mathcal{L}_{\alpha,0,i} \| \mathcal{L}_{\alpha,1,i} := \mathsf{Dec}^{(2)}_{\mathcal{L}_{\alpha,1},\mathcal{L}_{\alpha,i+1}}(\tau^{(k)}_{\alpha,i,r,c},\ \mathcal{T}_{\alpha,i}[r][c])$$

   For each $i \in [m]$: Let $c := \mathsf{Lsb}(\mathcal{L}_{\beta,i})$ and compute

$$\mathcal{L}_{\beta,0,i} \| \mathcal{L}_{\beta,1,i} := \mathsf{Dec}^{(2)}_{\mathcal{L}_{\alpha,1},\mathcal{L}_{\beta,i}}(\tau^{(k)}_{\beta,i,r,c},\ \mathcal{T}_{\beta,i}[r][c])$$

3. **Derive next-level keys and recurse.** Let $s^{(\mathsf{next})}$ be defined as follows:

$$\forall i \in [k-1], \forall j \in \{0,1,\ldots,2^i-1\}, s^{(\mathsf{next})}[i][j] := s[i+1][x_1 \cdot 2^i + j]$$

   Define

$$\mathsf{sk}^{(\mathsf{next})}_{\mathsf{off}} := \mathsf{sk}_{\mathsf{stack}} \oplus \widehat{\mathsf{sk}_{\mathsf{off}}} \quad \text{and} \quad \mathsf{sk}^{(\mathsf{next})}_{\mathsf{on}} := \{\mathcal{L}_{\alpha,x_1,i}\}_{i\in[k-1]} \parallel \{\mathcal{L}_{\beta,x_1,i}\}_{i\in[m]} \parallel s^{(\mathsf{next})}$$

   Output $\Pi^{(k-1)}_{\mathsf{napPRF}}.\mathsf{Eval}(\mathsf{sk}^{(\mathsf{next})}_{\mathsf{off}}, \mathsf{sk}^{(\mathsf{next})}_{\mathsf{on}}, x_2 \| \cdots \| x_k)$.

Figure 8: Redefined $\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{Prog}$ and $\Pi^{(k)}_{\mathsf{napPRF}}.\mathsf{Eval}$ over $\{0,1\}^k \to \{0,1\}^m$.

Let $\mathsf{REAL}_{\alpha}^{(k)}$ be the distribution of $(\mathsf{sk}_{\mathsf{off}}, \mathsf{sk}_{\mathsf{on}})$ for uniform $\beta \xleftarrow{\$} \{0,1\}^m$. The simulator $\mathsf{Sim}^{(k)}$ runs $(\mathsf{msk}, \mathsf{sk}_{\mathsf{off}}) \leftarrow \mathsf{KeyGen}^{(k)}(1^\lambda)$, samples $\beta \xleftarrow{\$} \{0,1\}^m$, and outputs $(\mathsf{sk}_{\mathsf{off}}, \mathsf{Prog}^{(k)}(\mathsf{msk}, 0^k, \beta))$; it suffices to show $\mathsf{REAL}_{\alpha}^{(k)} \approx_c \mathsf{REAL}_{0^k}^{(k)}$.

We use the same hybrid sequence as Theorem 9. First, replace the top-level master expansion $G(s_{\mathsf{master}})$ by uniform randomness (PRG security). Second, apply a PRF hybrid (using unique tweaks) to replace every non-decryptable translation-table entry by an independent uniform string. Hence, as before, the view can depend on α only through the revealed labels and the unique decryptable cells of the translation tables.

Then, Step 3a (hiding suffix $\alpha_2 \| \cdots \| \alpha_k$) proceeds exactly as in Theorem 9.

Step 3b (hiding α_1) is the same "swap-the-halves" argument: after cleaning/hybridizing down the candidate active path, each opened translation-cell plaintext is indistinguishable from two independent uniform λ-bit strings, whose distribution is invariant under swapping left/right halves.

At this point, the new seed tree s is handled similarly. Although s may be correlated with $\mathsf{sk}_{\mathsf{off}}$ through the recursive keys, it is α-*invariant*: it is obtained by starting from the dummy tree seed' and overwriting a fixed number of positions by seeds derived from s_{master}; after the standard PRG "cleaning" along the hidden active path (as in the base case), the overwritten seeds are indistinguishable from fresh uniform strings, so the marginal distribution of s does not depend on α. □

E The Improved LogStack Garbling Scheme

For completeness, following prior SGC schemes [HK20a, HK21], we formalize the improved LogStack — where the demultiplexer is instantiated with our tailored offline-online napPRFs (see Appendix 11) — as a *garbling scheme* [BHR12]. Here, we use a tailored napPRF variant in which the decoding bits are omitted, so evaluations output GC labels directly. We use $\Pi^{(n)} = (\mathsf{KeyGen}^{(n)}, \mathsf{Prog}^{(n)}, \mathsf{Eval}^{(n)})$ to denote the napPRF over domain $\{0,1\}^n$, where each evaluation returns I λ-bit labels. The parameters n and I are fixed by the garbling scheme.

Our garbling scheme ILS is the tuple of procedures defined in Figures 9 to 12:

$$(\mathsf{ILS.ev},\ \mathsf{ILS.Ev},\ \mathsf{ILS.Gb},\ \mathsf{ILS.En},\ \mathsf{ILS.De})$$

For simplicity, we omit any subprocedures that are identical to those in [HK21]. We make the following remarks.

- **Circuit syntax and semantics.** The circuit supports three recursive forms:
  1. Netlist: a standard Boolean circuit given as a netlist of AND/XOR gates.
  2. Seq: sequential composition of two components, i.e., $\mathcal{C}_1 \circ \mathcal{C}_0$.
  3. Cond: an n-bit conditional/dispatch over a vector $\vec{\mathcal{C}}$ of 2^n subcircuits, where the input is parsed as $\alpha \mid \vec{x}'$ and α selects the active branch.
- **The base Yao garbling scheme.** We model standard Yao garbling as the Base scheme, instantiated with an OWF-based point-and-permute construction. Alternatively, Base can use *free XOR* [KS08] and *half gates* [ZRE15] under the additional assumption of a *circular correlation-robust hash function* [CKKZ12].
- **Garble a conditional vector.** The improved algorithm GbCond for garbling a conditional vector is defined in Figure 10. Relative to GbCond in LogStack [HK21], the key change is how we implement the demultiplexer: we invoke our tailored offline-online napPRF to generate the branch input labels. In

particular, the offline key $\mathsf{sk}_{\mathsf{off}}$ is included in the garbling material, while the master key msk contains the garbling encodings (i.e., the wire label pairs).

We introduce a new subprocedure ComputeGarbageInput (defined in Figure 11) that helps G generate all dummy input labels by evaluating the napPRF to enumerate all off-point outputs, stored in Λ_{in}. Its running time is $\mathcal{O}(B \log^2 B)$, which is lower order compared to evaluating $\mathcal{O}(B \log B)$ dummy GCs.

- **Evaluate a conditional vector.** The improved algorithm EvCond for evaluating a conditional vector is defined in Figure 10. Relative to EvCond in LogStack [HK21], the key change is how we implement the demultiplexer: we invoke our tailored offline-online napPRF to generate the branch input labels. In particular, it invokes our offline-online napPRF's FullEval procedure.

Finally, we obtain the following theorem.

Theorem 15 (Improved LogStack). *Assuming one-way functions (via black-box use), the garbling scheme* ILS *is* correct*,* authentic*,* oblivious*, and* private *in the sense of standard definitions in [BHR12].*

Proof sketch. The proof follows the analysis of LogStack and reuses its simulators essentially verbatim. The only change is in the demultiplexer: wherever LogStack samples values derived from its PRF-based routing gadget, we instead sample them using the index-hiding simulators for our tailored offline-online napPRFs. All other components are unchanged, so correctness, authenticity, obliviousness, and privacy reduce to the corresponding guarantees established in [HK21] together with index-hiding of the napPRF. □

```
ILS.ev(C, x⃗) :
  ▷ What are the circuit semantics?
  switch C :
    case Netlist(·) : return Base.ev(C, x⃗)
    case Seq(C_0, C_1) : return ILS.ev(C_1, ILS.ev(C_0, x⃗))
    case Cond(C⃗) :
      ▷ Split branch index from input.
      α | x⃗' ← x⃗
      ▷ Run the active branch.
      return ILS.ev(C⃗[α], x⃗')

ILS.Ev(C, M, X⃗) :
  ▷ How does E evaluate the GC?
  switch(C) :
    case Netlist(·) : return Base.Ev(C, M, X⃗)
    case Seq(C_0, C_1) :
      M_0 | M_tr | M_1 ← M
      return ILS.Ev(C_1, M_1, trans.Ev(M_tr, ILS.Ev(C_0, M_0, X⃗)))
    case Cond(C⃗) : return EvCond(C⃗, M, X⃗)

ILS.En(e, x⃗) :
  ▷ How do inputs map to labels?
  ▷ This works for all projective schemes:
  X⃗ ← {}
  for i ∈ 0..InpSize(C)−1 :
    (X^(0), X^(1)) ← e[i]
    if x⃗[i] = 0 : {X⃗[i] ← X^(0)} else : {X⃗[i] ← X^(1)}
  return X⃗

ILS.Gb(1^λ, C, S)
  ▷ How does G garble the GC?
  ▷ S is an explicit seed.
  switch C :
    case Netlist(·) :
      return Base.Gb(1^λ, C, S)
    case Seq(C_0, C_1) :
      ▷ Derive seeds for two circuits.
      S_0 ← F_S(0)
      S_1 ← F_S(1)
      (M_0, e_0, d_0) ← ILS.Gb(1^λ, C_0, S_0)
      (M_1, e_1, d_1) ← ILS.Gb(1^λ, C_1, S_1)
      ▷ Labels out of C_0 must be translated
      ▷ to labels into C_1.
      M_tr ← trans.Gb(d_0, e_1)
      M ← M_0 | M_tr | M_1
      return (M, e_0, d_1)
    case Cond(C⃗) : return GbCond(C⃗, S)

ILS.De(d, Y⃗) :
  ▷ How do labels map to outputs?
  ▷ This works for all projective schemes:
  y⃗ ← {}
  for i ∈ 0..OutSize(C)−1 :
    (Y^(0), Y^(1)) ← d[i]
    if Y⃗[i] = Y^(0) : y⃗[i] ← 0
    else if Y⃗[i] = Y^(1) : y⃗[i] ← 1
    else : ABORT
  return y⃗
```

Figure 9: ILS's garbling scheme. The included algorithms are typical except for the handling of conditionals Ev and Gb delegate the core of our approach: GbCond (Figure 10) and EvCond (Figure 12). This level matches LogStack [HK21].

$\mathsf{GbCond}(\vec{\mathcal{C}}, S)$:

$B \leftarrow |\vec{\mathcal{C}}|, n \leftarrow \log B, I \leftarrow \mathsf{InpSize}(\mathsf{Cond}(\vec{\mathcal{C}})), O \leftarrow \mathsf{OutSize}(\mathsf{Cond}(\vec{\mathcal{C}}))$

▷ Sample output encodings for the conditional.

$d \leftarrow \mathsf{GenProjection}(S, O)$

▷ Derive all 'good' and 'bad' seeds for the entire tree from tailored napPRF.

$s_{\mathsf{master}} \xleftarrow{\$} \{0,1\}^\lambda, (\mathsf{msk}, \mathsf{sk}_{\mathsf{off}}) \leftarrow \mathsf{KeyGen}^{(n)}(1^\lambda; s_{\mathsf{master}})$

▷ Parse msk into input encodings for the conditional, 'good' and 'bad' seeds.

$e \mid \mathsf{seed} \mid \mathsf{seed}' \leftarrow \mathsf{msk}$

▷ Parse encoding into encoding of α and encoding of rest of I-bit input.

$e_\alpha \mid e' \leftarrow e$

▷ Garble SortingHat based on the encoding of α, 'good' and 'bad' seeds.

$M_{\mathsf{SortingHat}} \leftarrow \mathsf{SortingHat.Gb}(e_\alpha, \mathsf{seed}, \mathsf{seed}')$

▷ Construct the stacked material and input encodings for each branch.

$M_{\mathsf{cond}}, e_{\mathsf{cond}}, d_{\mathsf{cond}} \leftarrow \mathsf{GbSubtreeFromSeed}(\vec{\mathcal{C}}, 0, B-1, s_{\mathsf{master}})$

▷ Compute the dummy inputs by evaluating the napPRF over the entire domain.

▷ Here G would try every programmed point α, and evaluate on every $x' \neq \alpha$.

▷ The suffix invariance ensures that many cross α evaluations would coincide.

$\Lambda_{\mathsf{in}} \leftarrow \mathsf{ComputeGarbageInput}(n, I, \mathsf{sk}_{\mathsf{off}}, \mathsf{seed}, \mathsf{seed}', s_{\mathsf{master}})$

▷ Compute all possible garbage outputs.

$\Lambda_{\mathsf{out}} \leftarrow \mathsf{ComputeGarbage}(\vec{\mathcal{C}}, M_{\mathsf{cond}}, \Lambda_{\mathsf{in}}, \mathsf{seed}, \mathsf{seed}')$

▷ The multiplexer collects garbage outputs.

$M_{\mathsf{mux}} \leftarrow \mathsf{Mux.Gb}(e_\alpha, d, d_{\mathsf{cond}}, \Lambda_{\mathsf{out}})$

return $(M_{\mathsf{SortingHat}} \mid \mathsf{sk}_{\mathsf{off}} \mid M_{\mathsf{cond}} \mid M_{\mathsf{mux}}, e, d)$

Figure 10: The improved algorithm for garbling a conditional vector. Relative to GbCond in LogStack [HK21], the key change is how we implement the demultiplexer: we invoke our tailored offline-online napPRF (over $\{0,1\}^n \to \{0,1\}^I$, with decoding bits omitted so that Eval outputs wire labels) to generate the branch input labels. In particular, the offline key $\mathsf{sk}_{\mathsf{off}}$ is included in the returned garbled material. W.l.o.g., we assume B is a power of two. The procedures GenProjection, SortingHat, GbSubtreeFromSeed, and Mux are exactly as in LogStack. We introduce a new subroutine, ComputeGarbageInput (defined in Figure 11), which computes the dummy input labels for each branch (i.e., the off-point evaluation outputs of our napPRF). Consequently, LogStack's subroutine ComputeGarbage [HK21] requires a minor corresponding modification: when evaluating the dummy garbled circuit, the evaluator supplies the branch-specific dummy labels stored in Λ_{in}. We omit this change from the formalization since it is straightforward. We remark that KeyGen of napPRF and GbSubtreeFromSeed are invoked using the same master seed s_{master}, ensuring that the napPRF output at the programmed point coincides with the corresponding real GC labels.

$\mathsf{ComputeGarbageInput}(n, I, \mathsf{sk_{off}}, \mathsf{seed}, \mathsf{seed}', s)$:

▷ We first define a recursive subprocedure.

$\mathsf{ComputeGarbageInput}'(\ell, i, j, \mathsf{sk_{off}}, \mathsf{seed}, \mathsf{seed}', s)$:

if $i = j$:

▷ Base case: do nothing.

else :

$k \leftarrow \lfloor (i+j)/2 \rfloor, \quad d \leftarrow j - i + 1$

▷ Pased the stacked next-level offline key.

$\mathsf{sk_{stack}} \mid _ \leftarrow \mathsf{sk_{off}}$

▷ Compute the real next-level offline keys.

$(_, \mathsf{sk_{off,left}}) \leftarrow \mathsf{KeyGen}^{(n-\ell)}(1^\lambda; \mathsf{seed}[\ell][2i/d])$

$\mathsf{sk_{off,right}} \leftarrow \mathsf{sk_{stack}} \oplus \mathsf{sk_{off,left}}$

▷ Compute the dummy next-level offline keys.

$(_, \mathsf{sk}'_{\mathsf{off,left}}) \leftarrow \mathsf{KeyGen}^{(n-\ell)}(1^\lambda; \mathsf{seed}'[\ell][2i/d])$

$(_, \mathsf{sk}'_{\mathsf{off,right}}) \leftarrow \mathsf{KeyGen}^{(n-\ell)}(1^\lambda; \mathsf{seed}'[\ell][2i/d+1])$

$\mathsf{sk}''_{\mathsf{off,left}} \leftarrow \mathsf{sk_{stack}} \oplus \mathsf{sk}'_{\mathsf{off,right}}, \quad \mathsf{sk}''_{\mathsf{off,right}} \leftarrow \mathsf{sk_{stack}} \oplus \mathsf{sk}'_{\mathsf{off,left}}$

▷ Re-derive the dummy labels as in napPRF KeyGen.

$\vec{\mathcal{J}} \xleftarrow{\$_s} \{0,1\}^{(n-\ell+I)\lambda}$

▷ Setup dummy seeds for napPRF Eval.

$\mathsf{seed}''_{\mathsf{left}}[\chi][\gamma] \leftarrow \mathsf{seed}'[\ell+\chi][i/2^{n-\ell-\chi} + \gamma], \forall \chi \in [n-\ell], \gamma \in \{0, 1, \ldots, 2^\chi - 1\}$

$\mathsf{seed}''_{\mathsf{right}}[\chi][\gamma] \leftarrow \mathsf{seed}'[\ell+\chi][(k+1)/2^{n-\ell-\chi} + \gamma], \forall \chi \in [n-\ell], \gamma \in \{0, 1, \ldots, 2^\chi - 1\}$

▷ Fill dummy input labels using napPRF FullEval.

$\Lambda_{\mathsf{in}}[i \ldots k][n-\ell] \leftarrow \mathsf{FullEval}^{(n-\ell)}(\mathsf{sk}''_{\mathsf{off,left}}, \vec{\mathcal{J}}, \mathsf{seed}''_{\mathsf{left}})$

$\Lambda_{\mathsf{in}}[k{+}1 \ldots j][n-\ell] \leftarrow \mathsf{FullEval}^{(n-\ell)}(\mathsf{sk}''_{\mathsf{off,right}}, \vec{\mathcal{J}}, \mathsf{seed}''_{\mathsf{right}})$

▷ Recurse into left and right.

$\mathsf{ComputeGarbageInput}'(\ell+1, i, k, \mathsf{sk_{off,left}}, \mathsf{seed}, \mathsf{seed}', \mathsf{seed}[\ell][2i/d])$

$\mathsf{ComputeGarbageInput}'(\ell+1, k+1, j, \mathsf{sk_{off,right}}, \mathsf{seed}, \mathsf{seed}', \mathsf{seed}[\ell][2i/d+1])$

▷ Start recursive process from the top level.

$\mathsf{ComputeGarbageInput}'(1, 0, 2^n - 1, \mathsf{sk_{off}}, \mathsf{seed}, \mathsf{seed}', s_{\mathsf{master}})$

return Λ_{in}

Figure 11: A new subroutine ComputeGarbageInput**.** It is invoked within GbCond (Figure 10) to compute the dummy input labels for each branch (i.e., the off-point evaluation outputs of our napPRF). For each branch $t \in \{0, \ldots, 2^n - 1\}$, the resulting dummy labels are stored in $\Lambda_{\mathsf{in}}[t]$; moreover, by suffix invariance, each $\Lambda_{\mathsf{in}}[t]$ falls into one of only n possibilities, denoted $\Lambda_{\mathsf{in}}[t][0], \ldots, \Lambda_{\mathsf{in}}[t][n-1]$. Note that the dummy labels $\vec{\mathcal{J}}$ must be derived from the seed s according to the corresponding $\mathsf{KeyGen}^{(\cdot)}$ procedure; we omit the details for simplicity.

$\mathsf{EvCond}(\vec{\mathcal{C}}, M, X)$:

$B \leftarrow |\vec{\mathcal{C}}|, n \leftarrow \log B, I \leftarrow \mathsf{InpSize}(\mathsf{Cond}(\vec{\mathcal{C}})), O \leftarrow \mathsf{OutSize}(\mathsf{Cond}(\vec{\mathcal{C}}))$

▷ Parse the active branch index from the rest of the input.

$X_\alpha \mid X' \leftarrow X$

▷ Parse material for gadgets and body of conditional.

$M_{\mathsf{SortingHat}} \mid \mathsf{sk}_{\mathsf{off}} \mid M_{\mathsf{cond}} \mid M_{\mathsf{mux}} \leftarrow M$

▷ Run SortingHat to compute all of E's seeds.

$\mathsf{es} \leftarrow \mathsf{SortingHat.Ev}(X_\alpha, M_{\mathsf{SortingHat}})$

▷ Run the napPRF FullEval to compute input for each branch $\mathcal{C}_i$.

$\vec{X}_{\mathsf{cond}} \leftarrow \mathsf{FullEval}^{(n)}(\mathsf{sk}_{\mathsf{off}}, X_\alpha, X', \mathsf{es})$

▷ We define a recursive subprocedure that evaluates $\mathcal{C}_i$ to $\mathcal{C}_j$ using material M.

$\mathsf{EvCond}'(i, j, M)$:

if $i = j$:

▷ Base case: compute output by evaluating the branch normally.

▷ This base case corresponds to guess $= i$.

▷ Accumulate output labels into the vector $\vec{Y}_{\mathsf{cond}}$ (for later garbage collection).

$\vec{Y}_{\mathsf{cond}}[i] \leftarrow \mathsf{Base.Ev}(\mathcal{C}_i, M, \vec{X}_{\mathsf{cond}}[i])$

else :

$k \leftarrow \lfloor (i+j)/2 \rfloor, \quad d \leftarrow j - i + 1$

▷ Garble the right subtree using the available seed,

▷ unstack, and recursively evaluate the left subtree.

$M_{k+1,j}, _, _ \leftarrow \mathsf{GbSubtreeFromSeed}(\vec{\mathcal{C}}, k+1, j, \mathsf{es}[B - \log(d) + 1][2i/d + 1])$

$\mathsf{EvCond}'(i, k, M \oplus M_{k+1,j})$

▷ Symmetrically evaluate the right subtree.

$M_{i,k}, _, _ \leftarrow \mathsf{GbSubtreeFromSeed}(\vec{\mathcal{C}}, i, k, \mathsf{es}[B - \log(d) + 1][2i/d])$

$\mathsf{EvCond}'(k+1, j, M \oplus M_{i,k})$

▷ Start recursive process from the top level.

$\mathsf{EvCond}'(0, B-1, M_{\mathsf{cond}})$

▷ Eliminate garbage and propagate $\vec{Y}_\alpha$ via the multiplexer.

return $\mathsf{Mux.Ev}(X_\alpha, \vec{Y}_{\mathsf{cond}}, M_{\mathsf{mux}})$

Figure 12: The improved algorithm for evaluating a conditional vector. Relative to EvCond in LogStack [HK21], the key change is how we implement the demultiplexer: we invoke our tailored offline-online napPRF (over $\{0,1\}^n \to \{0,1\}^I$, with decoding bits omitted so that Eval outputs wire labels) to generate the branch input labels. In particular, the offline key $\mathsf{sk}_{\mathsf{off}}$ is included in the input garbled material. W.l.o.g., we assume B is a power of two. The procedures SortingHat, Base, GbSubtreeFromSeed, and Mux are exactly as in LogStack.